CODEX APP · PROMPT · CONTEXT · HARNESS | 실습 중심 강의 교안

# Coding Agent 기본 :

# AI를 활용한 SW 개발 기초

Codex App 으로 PRD → GitHub → 배포까지 — 브라우저 게임(Tetris)을 만들고 인터넷에 공개한다

ChatGPT / OpenAI 기반 | 프롬프트 · 컨텍스트 · 하네스 | Tetris → GitHub Pages

하루 과정 · 08:30–17:30 (점심 12:00–13:00) · 대상: 프로그래밍 경험이 없어도 완주

2026 · 김대경 Ph.D.

# 과정 개요 — PRD에서 배포까지

| 항목 | 내용 |
|---|---|
| **교육 대상** | 프로그래밍 경험이 없어도 완주 — 누구나 참여 |
| **교육 시간** | 하루 08:30–17:30 (점심 12:00–13:00) · 강의와 실습을 번갈아 진행 |
| **교육 구성** | 1장 준비 → 2–4장 Codex 개관 · App · 사이드바 → 5–8장 프롬프트 · 컨텍스트 · 지시/권한 · 하네스 → 9–10장 Tetris 실습 → 11장 회고 |
| **교육 방식** | Codex App(데스크톱)을 메인으로 '직접 손으로' 실습, Codex CLI 는 보조 |
| **도구 스택** | 순수 HTML/CSS/JavaScript · Node.js · Git · GitHub · Codex App |
| **사전 과제** | 계정: ChatGPT/OpenAI · GitHub / 설치: Node.js 18+ (20 LTS 권장) · Git |
| **관통 예제** | linksnip(URL 단축기)으로 기법을 익히고, Tetris 를 만들어 GitHub Pages 에 배포한다 |

**핵심** 핵심 목표 — 프롬프트 · 컨텍스트 · 하네스로 PRD → GitHub → 배포까지, 내 손으로 제품을 출시 · 운영한다.

# 학습 목표 — 과정을 마치면 할 수 있는 것

## 1 전체 기술 지도를 설명 2–4장

Codex 의 사용 방식 · 실행 환경 · 지시/권한 체계 등 전체 기술 지도를 설명할 수 있다.

## 2 규칙 · 검증 체계 구성 5–8장

Prompt · Context · Harness 와 AGENTS.md 로 프로젝트 규칙과 검증 체계를 구성할 수 있다.

## 3 안전한 실행 설정 7장

Sandbox · Approvals 로 안전하게 실행하고 config.toml 기초를 설정할 수 있다.

## 4 제품 출시 · 운영 9–10장

PRD → GitHub → 배포까지 제품을 출시 · 운영할 수 있다.

강의 · 개념 이해 과정 안내 | DAY FLOW

# 하루 흐름 — 강의와 실습을 번갈아 진행한다

08:30–17:30 · 점심 12:00–13:00 · 교시별 시각은 진행 상황에 맞춰 강사가 조정한다

| 오전 | | | 오후 | | |
|---|---|---|---|---|---|
| 1장 | 시작 전 준비 — 계정과 환경 | 실습 | 7장 | 지시 · 권한 체계 — AGENTS.md · config.toml · Sandbox · Skill | 강의+실습 |
| 2장 | Codex 개관 — 기술 지도와 사용 방식 | 강의 | 8장 | 하네스 엔지니어링 — 빌드 · 테스트 · 배포 · 로그 | 강의+실습 |
| 3장 | Codex App 시작하기 | 실습 | 9장 | PRD 작성 · To-Do 도출 | 실습 |
| 4장 | 사이드바 — 사이트 · 예약 · 플러그인 | 실습 | 10장 | Tetris 만들고 GitHub Pages 에 배포 | 실습 |
| 5장 | 프롬프트 엔지니어링 | 강의+실습 | 11장 | 마무리 회고 | 강의 |
| 6장 | 컨텍스트 엔지니어링 | 강의+실습 | | | |

**점심 12:00–13:00 · 50분 진행 후 10분 휴식을 기본으로 한다**

강의 · 개념 이해 과정 안내 | HOW TO READ

# 이 교안 보는 법 — 강의와 실습을 색으로 구분한다

슬라이드 왼쪽 띠와 제목 위 배지를 보면 '지금 듣는 시간인지, 직접 하는 시간인지' 바로 알 수 있다

강의 · 개념 이해

### 듣고 이해한다

개념 · 원리 · 사례를 설명한다. 노트북은 잠시 내려 둔다.

실습 · 직접 따라 하기

### 직접 따라 한다

하단 초록 띠의 '지금 할 일'을 순서대로 실행한다.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

```
Codex 응답
✦ hello.txt 를 만들었다.
  Hello Codex
```

결과 예시 = 프롬프트를 실행했을 때 돌아오는 응답 · 출력의 예

▶ 프롬프트 — Codex App

이 폴더에 hello.txt 파일을 만들어줘.

파란 상자 = Codex 작성 칸에 그대로 붙여 넣을 프롬프트

✕ 나쁜 예

로그인 기능 만들어줘.

빨간 상자 = 일부러 보여 주는 나쁜 프롬프트 (따라 하지 않는다)

**실행 결과 (화면)** 워크북에서 실제로 실행한 화면

스크린숏 = 실제 실행 결과 화면

@파일 · $skill · /명령 은 굵은 고정폭 글꼴로 표시한다

**실습 슬라이드 하단 초록 띠 = '지금 할 일' — 이 줄만 따라가면 실습이 끝난다**

01

CHAPTER 1 · 실습

# 시작 전 준비 — 계정과 환경

실습 중 '계정이 없어서' 막히지 않도록, 본격적인 실습 전에 계정과 도구를 먼저 준비한다 (약 20–30분).

1.1 계정과 로컬 도구

1.2 사전 점검

# 만들어 둘 계정과 설치할 도구

모두 무료로 시작한다 — 필요한 설치 프로그램은 Node.js 와 Git 뿐이다

### ChatGPT / OpenAI

- Codex 로그인 · 모델 사용
- chatgpt.com 계정 하나로 App · CLI 모두 로그인

### GitHub

- 소스 코드 저장 · 배포 트리거
- github.com — 10장에서 GitHub Pages 배포

### Codex App

- 데스크톱 앱 — 3장에서 설치
- openai.com/ko-KR/ 에서 내려받기

**로컬 개발 도구**

| 도구 | 버전 · 용도 |
|---|---|
| Node.js | 18 이상 (20 LTS 권장) · CLI · npm |
| Git | 최신 버전 · 버전 기록 · GitHub 업로드 |

**설치 확인 명령**

```
# 설치 여부 확인 (터미널)
node -v
git --version
```

참고 Windows 는 '명령 프롬프트(cmd)' 또는 PowerShell, macOS 는 '터미널' 앱에서 실행한다.

# 사전 점검 — 다섯 가지를 확인하고 시작한다

## 사전 점검 체크리스트

- ✅ ChatGPT/OpenAI 계정으로 로그인할 수 있다
- ✅ GitHub 계정이 있고 로그인된다
- ✅ 터미널에서 node -v 가 버전 숫자를 출력한다
- ✅ 터미널에서 git --version 이 버전 숫자를 출력한다
- ✅ 작업 폴더 C:\DEV 를 만들어 두었다 (macOS 는 ~/DEV)

**TIP** 점검이 끝나면 강사에게 '준비 완료'를 알린다 — 3장부터는 모든 실습이 이 환경을 전제로 한다.

**결과 예시** 버전 숫자는 설치 시점에 따라 다르다

명령 프롬프트

```
C:\DEV> node -v
v20.15.1
C:\DEV> git --version
git version 2.45.2.windows.1
```

## 버전이 나오지 않으면

- '인식할 수 없는 명령' → 설치 후 터미널을 새로 연다
- Node.js: nodejs.org 에서 LTS 설치
- Git: git-scm.com 에서 설치 (기본 옵션 그대로)

02

CHAPTER 2 · 강의

# Codex 개관 — 기술 지도와 사용 방식

Codex 는 터미널 · 에디터에서 실제로 코드를 읽고, 고치고, 명령을 실행하는 코딩 에이전트다. 무엇을 시키고(프롬프트) · 무엇을 보여 주고(컨텍스트) · 어떤 검증 틀에서 일하게 하느냐(하네스)가 결과를 좌우한다.

2.1 네 개의 축 | 2.2 핵심 용어 | 2.3 App · IDE · CLI

# Codex 전체 기술 지도 — 네 개의 큰 축

단순히 코드를 보여 주는 챗봇이 아니라, 프로젝트 폴더 안에서 파일을 직접 수정하고 테스트까지 돌린다

1

## 모델

코드를 생성 · 추론하는 AI 엔진 (gpt-5.x 계열 · 예: GPT-5.6 Sol)

다루는 곳 3.5 · 7.5 · 8.8

2

## 사용 방식

Codex 를 '어디서' 쓰는가 — App · IDE 확장 · CLI

다루는 곳 2.3 · 3장 · 4장

3

## 지시 · 권한

AGENTS.md(규칙) · Sandbox(범위) · Approvals(승인)

다루는 곳 7장

4

## 하네스

빌드 · 테스트 · 배포 · 로그로 결과를 '검증'하는 틀

다루는 곳 8장

**핵심** 이 네 축이 결과 품질을 좌우한다 — 무엇을 시키는가(프롬프트 · 5장) · 무엇을 보여 주는가(컨텍스트 · 6장) · 어떤 검증 틀에서 일하게 하는가(하네스 · 8장).

**참고** 지금 모든 용어를 외울 필요는 없다 — '프롬프트 → 컨텍스트 → 하네스'가 이 과정의 뼈대라는 점만 기억한다.

# 핵심 용어 미리 보기

각 용어는 해당 장에서 직접 만들어 보며 익힌다

### Prompt (프롬프트) 5장

Codex 에게 시키는 자연어 지시. 작게 쪼개고, 검증 방법을 함께 준다.

### Context (컨텍스트) 6장

Codex 가 참고하는 정보(파일 · 이미지 · 이전 대화 · AGENTS.md). 모델의 '작업 기억'.

### AGENTS.md 7.1

프로젝트의 상시 규칙 문서. 모든 작업 전에 Codex 가 먼저 읽는다.

### Harness (하네스) 8장

작업을 감싸는 검증 · 안전 장치 (빌드 / 테스트 / 린트 / 샌드박스).

### Sandbox (샌드박스) 7.6

Codex 가 파일을 쓰거나 네트워크를 쓸 수 있는 '기술적 범위'.

### Approvals (승인) 7.6

위험한 행동 전에 사람에게 물어보게 하는 '정책'.

### Thread (스레드) 3.3

하나의 작업 세션. 여러 프롬프트가 이어진 대화.

### MCP 부록 D

Codex 에 외부 도구(GitHub 등)를 연결하는 표준 규약.

# 사용 방식 — App · IDE · CLI

어떤 방식으로 쓰든 실제로 일하는 AI 모델은 똑같다 — 이 과정은 Codex App 을 메인으로 실습한다

## App

MAIN

독립 데스크톱 앱. 인앱 브라우저 · 변경 미리보기(diff) · 승인 UI · 클라우드 작업

**언제 쓰나** 이 과정의 메인 실습 (3장 ~ 10장)

## IDE 확장

VS Code 등 에디터에 통합. 열린 파일 · 선택 영역을 자동으로 컨텍스트에 넣는다

**언제 쓰나** 코드 작성 중 즉시 도움

## CLI

터미널 기반. 스크립트 / CI 자동화에 적합 (Rust 로 작성되어 가볍고 빠름)

**언제 쓰나** 자동화 · 보조 (3.5 · 8.8)

**참고** 왜 App 을 메인으로? — Codex 가 제안한 파일 변경을 diff(전/후 비교)로 눈으로 확인하고 버튼으로 승인할 수 있어 입문자에게 가장 안전하다. 인앱 브라우저로 만든 웹을 바로 열어 보고, 오래 걸리는 작업은 클라우드에 맡길 수도 있다.

**TIP** CLI 는 같은 AI 모델을 쓰므로, 자동화가 필요할 때 보조로 함께 익힌다 (3.5 설치 · 8.8 비대화형 실행).

03

CHAPTER 3 · 실습

# Codex App 시작하기

Codex App 을 설치하고, 작업 폴더를 열고, 첫 작업을 시킨 뒤 업무에서 자주 쓰는 12가지 유형으로 대화해 본다. CLI 설치도 참고로 익힌다.

3.1 설치 · 로그인 | 3.2 첫 프로젝트 | 3.3 첫 작업 | 3.4 유형별 대화 | 3.5 CLI

# Codex App 설치 — 내려받기부터 실행까지

### 1 Codex 페이지 열기

openai.com/ko-KR/ 상단 메뉴 '제품' → Codex

### 2 데스크톱 앱 내려받기

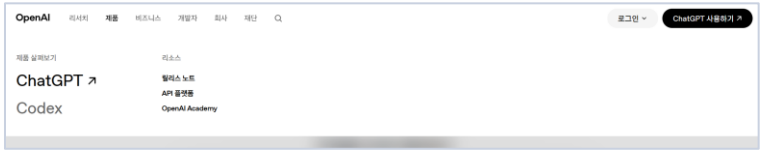
'Windows 버전 다운로드'(macOS 는 Mac 버전)

### 3 설치 프로그램 실행

ChatGPT Installer.exe 를 실행한다

### 4 앱 실행 → 로그인 화면

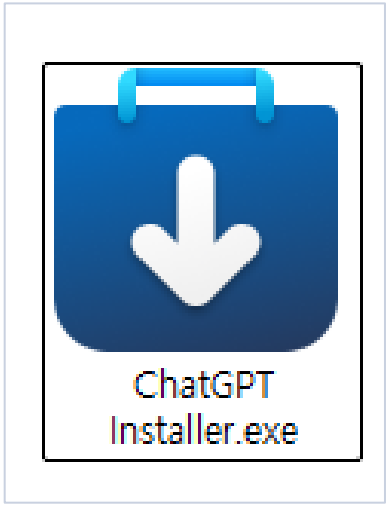
'ChatGPT에 로그인' 창이 열린다




① openai.com — 제품 메뉴의 Codex




② Codex 페이지 — Windows 버전 다운로드

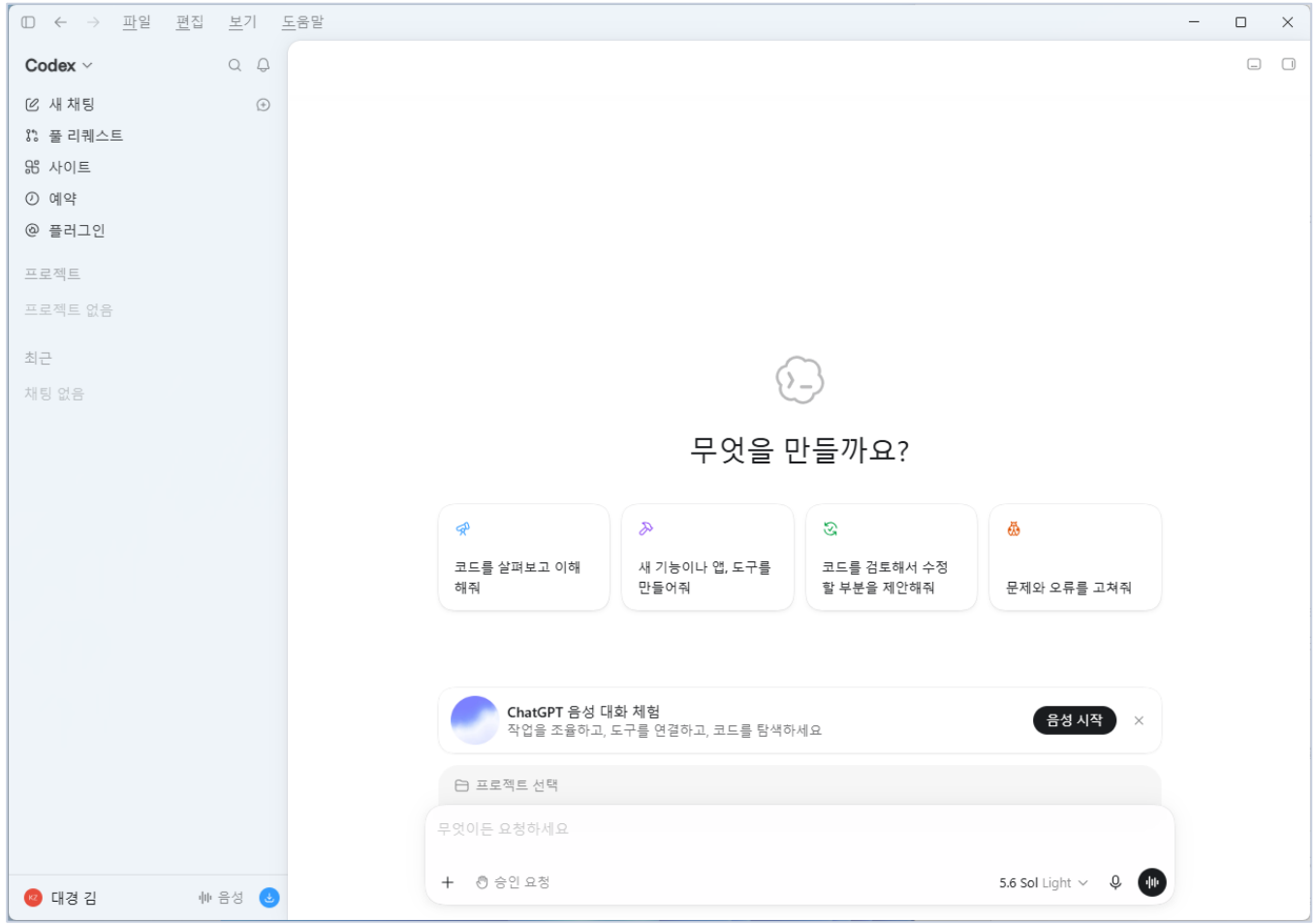



③ 설치 프로그램




④ 첫 실행 — ChatGPT 에 로그인

# 로그인과 첫 화면 — 같은 ChatGPT 계정으로




로그인 후 첫 화면 — 왼쪽 사이드바(새 채팅 · 풀 리퀘스트 · 사이트 · 예약 · 플러그인)와 가운데 작성 칸

**1** **ChatGPT 계정으로 로그인**

구독에 포함된 최신 모델을 그대로 쓸 수 있다

**2** **권한 · 보안 설정 확인**

기본값은 '작업 폴더 안에서만 변경, 위험한 동작은 승인' — 안전하다 (7.6)

## TIP — 계정 로그인 방식 두 가지

- 입문자는 ChatGPT 계정 로그인을 권장 (최신 모델 즉시 사용, 구독에 포함)
- 자동화 · CI 가 필요하면 OpenAI 플랫폼의 API 키도 가능 — 사용량만큼 과금, 신규 모델 반영이 늦을 수 있다

지금 할 일 '로그인 계속하기' → ChatGPT 계정으로 로그인 ▸ 첫 화면 '무엇을 만들까요?' 확인 ▸ 권한 기본값 확인

# 첫 프로젝트 열기 — 하나의 작업 폴더를 기준으로 일한다

**1 빈 폴더 만들기**

예: C:\DEV\codex-hello (macOS 는 ~/DEV/codex-hello)

**2 App 에서 폴더 열기**

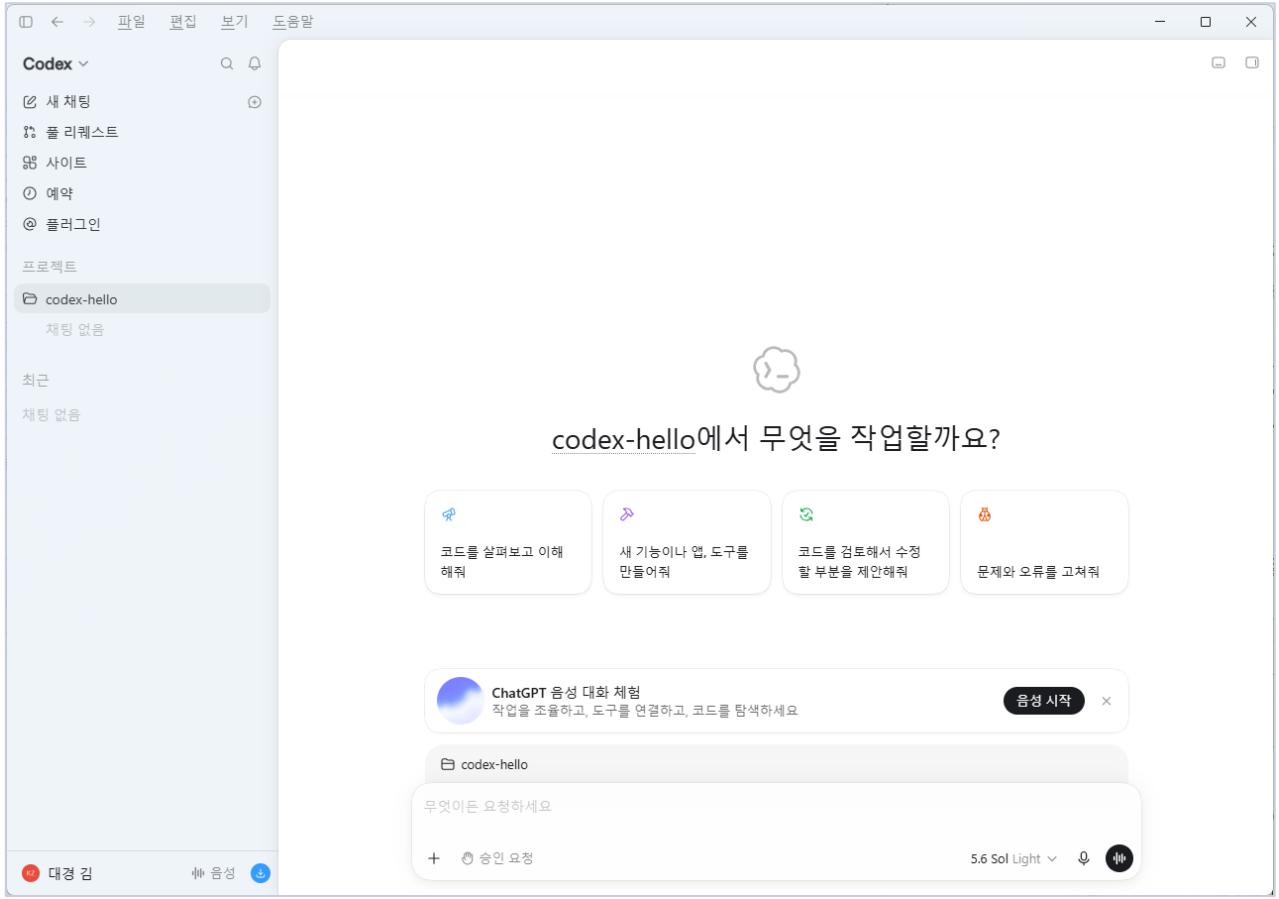
'파일 / 폴더 열기'(또는 새 작업 만들기) → 방금 만든 폴더 선택. 이 폴더가 Codex 의 작업 공간이 된다

**3 작성 칸 확인**

화면 아래 '작성 칸(프롬프트 입력창)'이 보이면 준비 완료

**TIP** 폴더를 열어 두면 Codex 가 그 안의 파일을 바로 참고한다. 바탕화면 전체처럼 관련 없는 큰 폴더는 열지 않는다 (6장).

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다

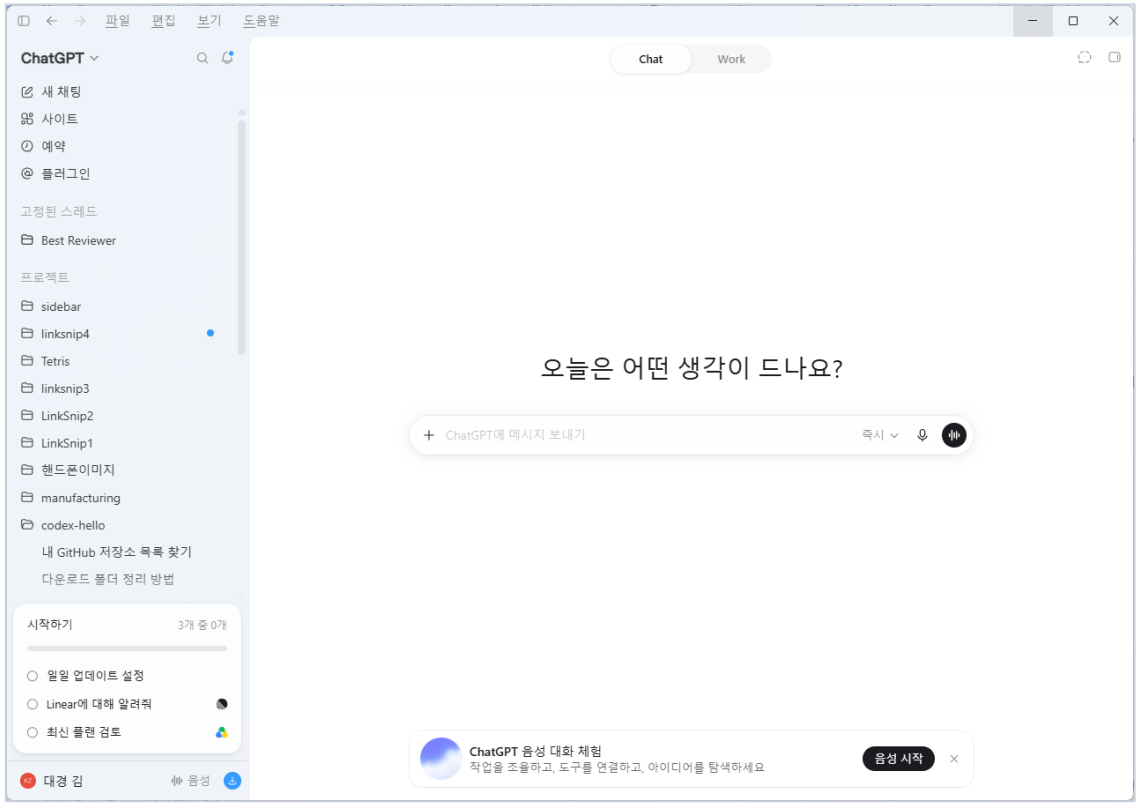



codex-hello 를 연 상태 — 'codex-hello에서 무엇을 작업할까요?'

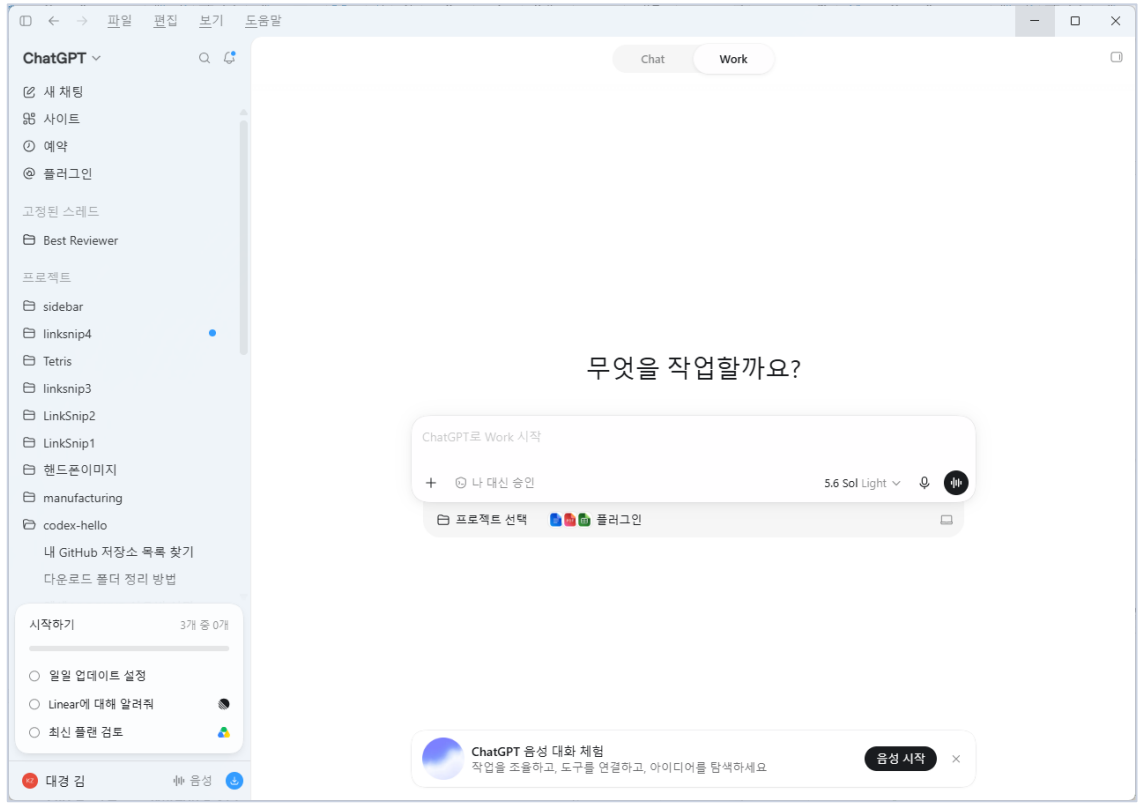
**지금 할 일** 탐색기: C:\DEV\codex-hello 폴더 만들기 ▸ Codex App: '폴더 열기' → codex-hello 선택 ▸ 작성 칸 확인

# 같은 앱의 ChatGPT 화면 — Chat 과 Work

앱 왼쪽 위 메뉴에서 Codex ↔ ChatGPT 를 오간다. 프로젝트 목록 · 사이트 · 예약 · 플러그인은 함께 쓴다




ChatGPT — Chat 탭: '오늘은 어떤 생각이 드나요?'



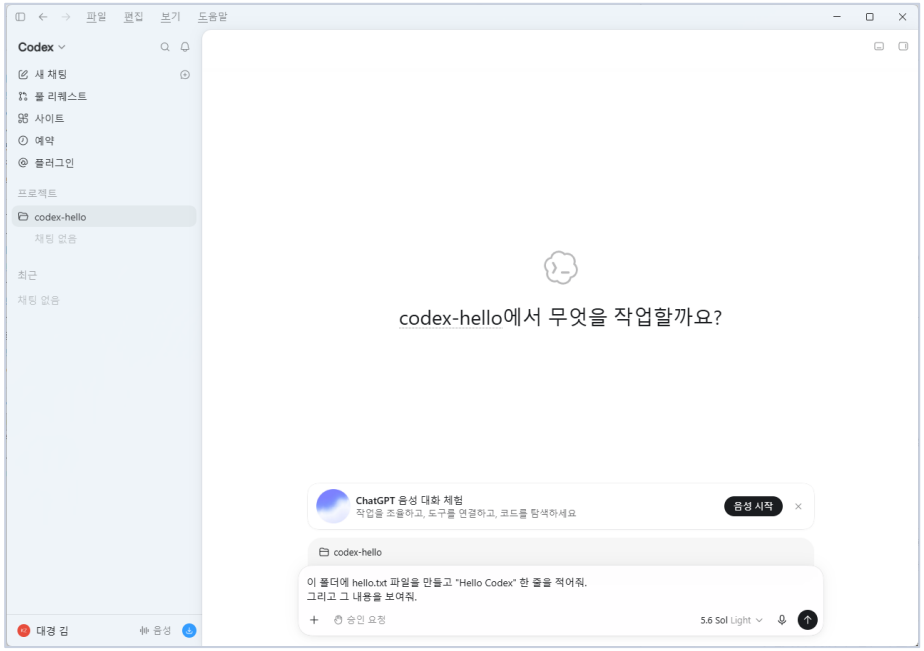

ChatGPT — Work 탭: 프로젝트를 선택해 작업

**참고** 코드 · 파일 작업은 Codex 화면에서 한다 — 이 과정의 모든 프롬프트는 Codex 작성 칸에 붙여 넣는다.

# 첫 작업 — Codex 와 인사하기

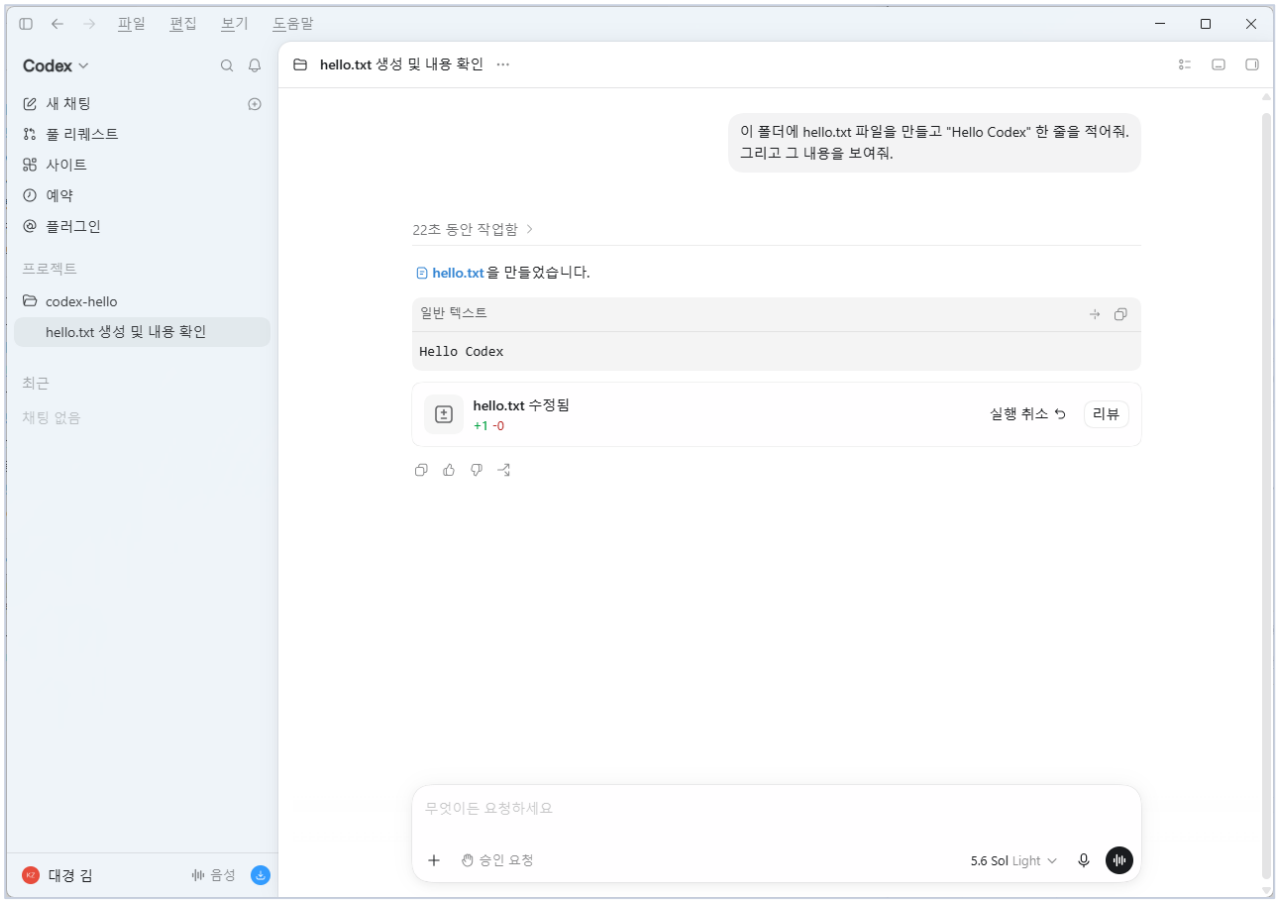
**▶ 프롬프트 — Codex App**

이 폴더에 hello.txt 파일을 만들고 "Hello Codex" 한 줄을 적어줘.
그리고 그 내용을 보여줘.




① 작성 칸에 프롬프트 입력

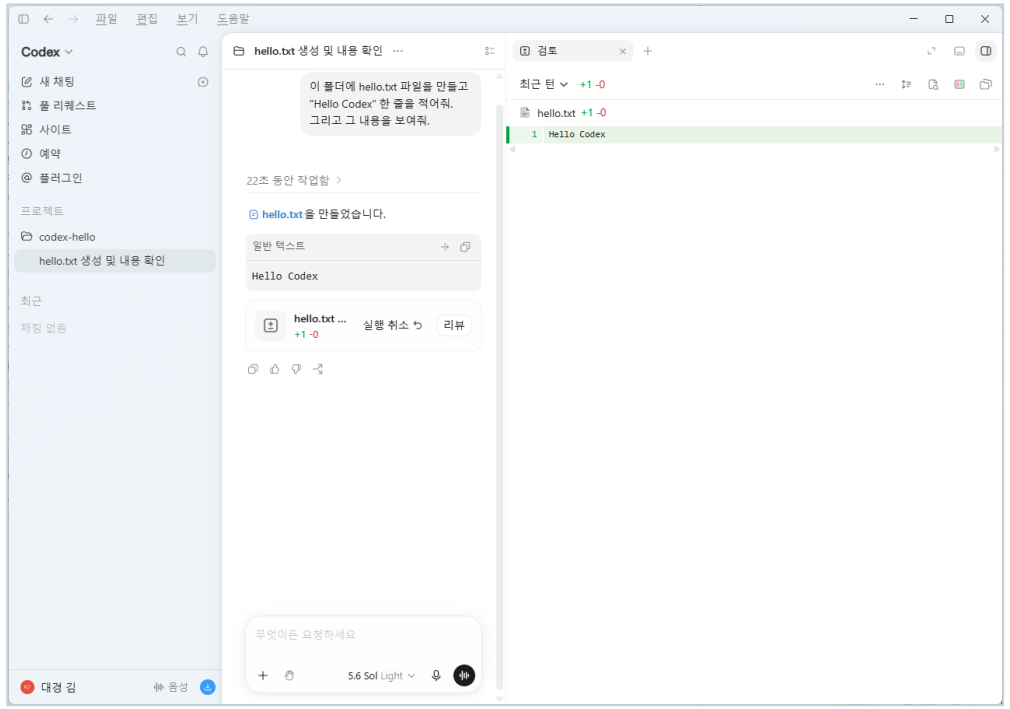
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



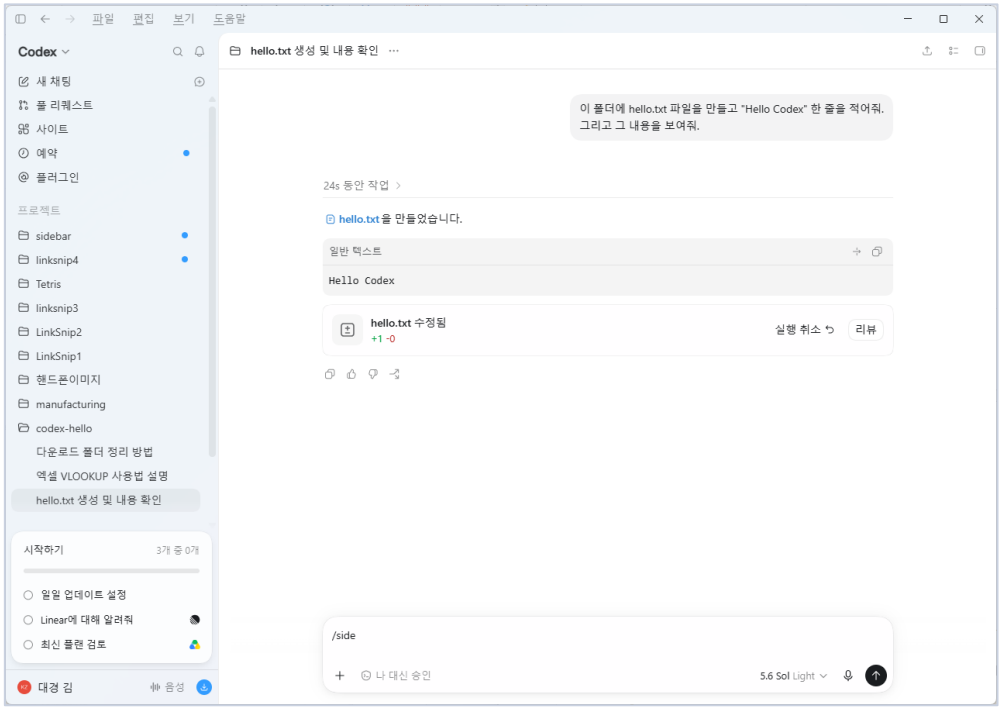

② 결과 — hello.txt 생성 · 내용 'Hello Codex' · 변경 파일 카드(+1)

**지금 할 일** Codex 작성 칸: 아래 프롬프트 붙여 넣기 → 전송 ▸ 파일 목록에서 hello.txt 열기 ▸ 내용이 'Hello Codex' 인지 확인

# 화면 다루기 — 사이드 패널과 사이드 채팅(/side)




① 사이드 패널 표시 / 숨기기 — 변경 내용(diff) 확인




② 작성 칸에 /side — 본 대화와 별개인 임시 사이드 채팅

**▶ 프롬프트 — 사이드 채팅**

지금까지 무슨 대화를 하고 있었는지 알려줘.

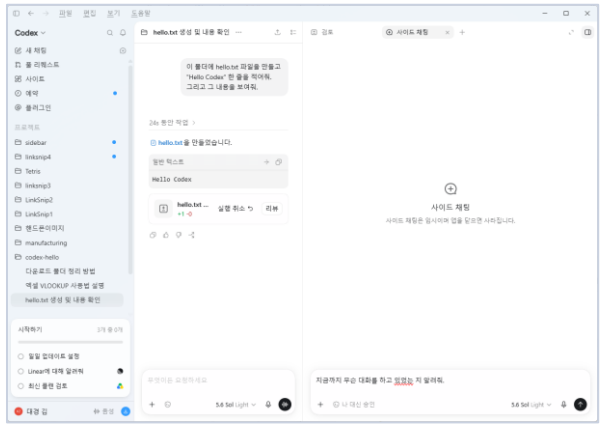
## /side 란?

- 본 작업 흐름을 건드리지 않고 잠깐 묻는 임시 채팅 — 결과는 다음 슬라이드

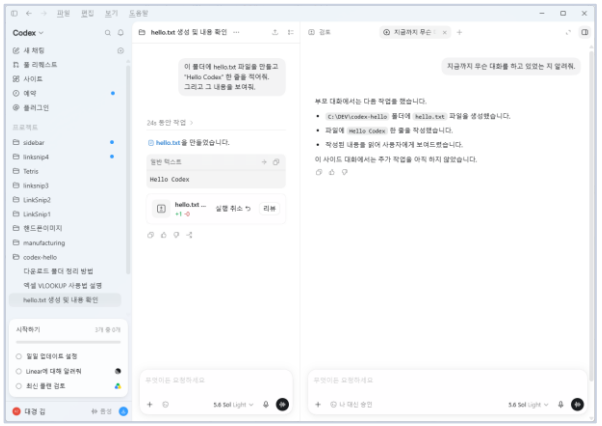
**지금 할 일** 오른쪽 위 버튼으로 사이드 패널 열기 · 닫기 ▸ 작성 칸에 /side 입력 ▸ 사이드 채팅에 아래 프롬프트

# 사이드 채팅 결과 · 터미널 · 파일 트리

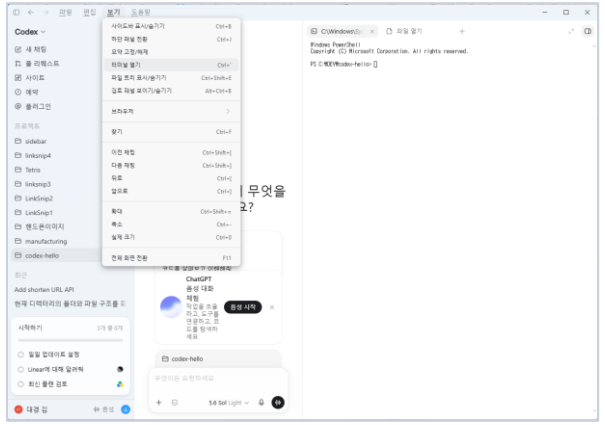
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



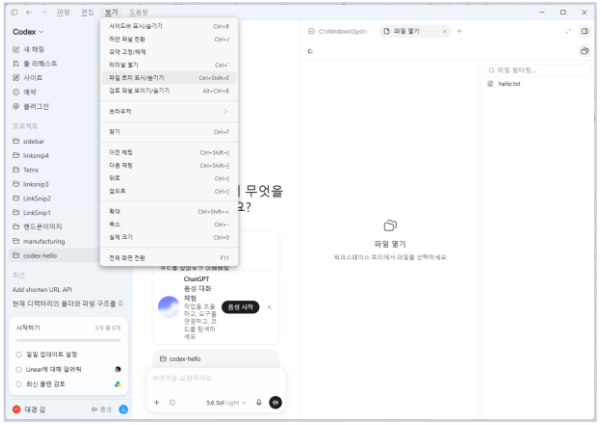
① 사이드 채팅 열림



② 결과 — 지금까지의 대화 요약



③ 보기 → 터미널 열기



④ 보기 → 파일 트리 표시

## NOTE — 한 번에 말고 대화로

- 한 스레드(작업 세션) 안에서 후속 지시를 받는다
- '파일 만들기 → 테스트 추가 → 설명 추가'처럼 작은 단계로

## TIP — 새 주제 = 새 스레드

- 사이드바 '새 채팅' — 한 작업 = 한 스레드 (6.4 격리)

## 3.1–3.3 확인

- ✅ App 로그인
- ✅ 폴더 열기
- ✅ hello.txt 확인

**지금 할 일** 사이드 채팅 응답 확인 ▸ 보기 메뉴 → 터미널 열기 ▸ 보기 메뉴 → 파일 트리 표시 / 숨기기

# 업무에서 자주 쓰는 AI 활용 유형 12가지

예시의 폴더 경로 · 파일명은 내 컴퓨터의 실제 경로로 바꿔서 사용한다 (실습 폴더: C:\DEV\manufacturing · C:\DEV\meeting)

### 1 질의응답 (Q&A)
개념 · 사용법을 묻고 바로 답을 얻는다

### 2 문장 생성
이메일 · 공지문 등 목적에 맞는 글

### 3 요약
긴 문서 · 회의록의 핵심만

### 4 번역
문서 · 문장을 다른 언어로

### 5 분석
데이터를 모아 집계 · 표 · 차트로

### 6 프로그램(자동화)
반복 작업을 처리하는 스크립트 · 매크로

### 7 채팅(상담)
정답 없는 주제를 대화로 정리

### 8 텍스트 → 이미지
장면 · 스타일을 글로 설명해 생성

### 9 이미지 변형
기존 이미지의 상태 · 스타일만 변경

### 10 유사도 검색
새 사례와 가장 비슷한 과거 사례

### 11 음성 → 텍스트
녹음 파일을 글자로 받아 적기

### 12 음성 → 영어 번역
듣고 번역해 파일로 저장

**핵심** 목표 — 12가지 중 3가지 이상을 직접 실행하고, 결과물(문서 · 표 · 이미지 · 텍스트 파일)을 직접 열어 확인한다.

# ① 질의응답 · ② 문장 생성

## ① 질의응답 (Q&A) — 개념이나 사용법을 묻는다

**▶ 프롬프트 — Codex App**

엑셀에서 VLOOKUP 함수 어떻게 쓰는지 알려줘. 예시 데이터를 만들어서 설명해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ VLOOKUP 은 표의 첫 열에서 값을 찾아 같은 행의 다른 열 값을
  가져오는 함수이다.
  =VLOOKUP(찾을값, 범위, 열번호, FALSE)

  예시 데이터  A열 사번 | B열 이름 | C열 부서
  A001  김민수  생산1팀
  A002  이서연  품질팀
  =VLOOKUP("A002", A2:C10, 3, FALSE)  →  품질팀
  · 마지막 인수 FALSE = 정확히 일치하는 값만 찾기
```

## ② 문장 생성 — 목적에 맞는 글을 새로 쓴다

**▶ 프롬프트 — Codex App**

다음 내용으로 팀 전체에 보낼 회의 공지 이메일을 작성해줘.
- 일시: 다음 주 화요일(9/9) 오후 2시
- 장소: 3층 대회의실
- 안건: 2분기 실적 리뷰

정중하고 간결한 톤으로, 5문장 이내로 써줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 제목: [공지] 9/9(화) 2분기 실적 리뷰 회의 안내

  팀원 여러분, 안녕하세요.
  다음 주 화요일(9/9) 오후 2시, 3층 대회의실에서
  2분기 실적 리뷰 회의를 진행한다.
  각 파트별 실적 자료를 지참해 주시기 바란다.
  일정 변경이 필요하면 미리 알려 주세요. 감사한다.
```

**지금 할 일** 두 프롬프트 중 하나 이상 실행 ▸ ①은 예시 표가 들어갔는지, ②는 5문장 이내인지 확인

# ③ 요약 · ④ 번역

## ③ 요약 — 긴 문서의 핵심만 짧게

**▶ 프롬프트 — Codex App**

[meeting_notes.txt](C:\DEV\manufacturing\meeting_notes.txt) 파일 내용을 5줄 이내로 요약해줘. 결정된 사항과 담당자는 따로 구분해서 정리해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 요약 (5줄)
  1. 3라인 불량률이 지난주 대비 0.8%p 상승
  2. 원인 후보: 접착 공정 온도 편차
  3. 금형 점검 일정 앞당김
  4. 협력사 자재 로트 추적 요청
  5. 다음 회의에서 개선 결과 공유

  결정 사항 · 담당자
  · 금형 점검 → 생산기술팀 박OO
  · 로트 추적 → 구매팀 이OO
```

## ④ 번역 — 서식은 그대로 다른 언어로

**▶ 프롬프트 — Codex App**

[notice_kr.docx](C:\DEV\manufacturing\notice_kr.docx) 파일 내용을 영어로 번역해서 같은 폴더에 notice_en.docx로 저장해줘. 굵게 표시나 글머리 기호 같은 서식은 그대로 유지해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ notice_kr.docx 를 영어로 번역해 notice_en.docx 로 저장했다.
  · 굵게 표시 7곳, 글머리 기호 12개 유지
  · 제목: "Safety Inspection Notice for Line A"

  수정됨  notice_en.docx  (+1 파일)
```

# ⑤ 분석 — 데이터를 모으고 집계해 표 · 차트로

## ⑤-1 먼저 폴더에 무엇이 있는지 묻는다

**▶ 프롬프트 — Codex App**

폴더에 있는 파일이 어떤 파일인지 알려줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ C:\DEV\manufacturing 에는 다음 파일이 있다.
  · defect_prediction_data.csv — 공정 센서값과 불량 여부(0/1)
  · 2020-01-21_A_Line.xlsx ~ 01-25 (5개) — A라인 기능검사 기록
  · meeting_notes.txt — 품질 회의록
  · notice_kr.docx — 안전 점검 공지(한국어)

  분석에 바로 쓸 수 있는 것은 CSV 와 검사 기록 5개이다.
```

## ⑤-2 합산표와 차트를 만든다

**▶ 프롬프트 — Codex App**

[defect_prediction_data.csv](C:\DEV\manufacturing\defect_prediction_data.csv)에서 불량에 대한 합산표를 만들어줘. 결과는 새 시트에 정리하고 차트도 함께 만들어줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ defect_summary.xlsx 를 만들었다.
  시트 '불량 합산'
  구분        건수     비율
  정상       9,412   94.1%
  불량         588    5.9%
  · 라인별 · 일자별 불량 건수 피벗 표 추가
  · 막대 차트 '라인별 불량 건수' 삽입
```

# ⑥ 프로그램(자동화) 생성 — 다중 파일 병합

A라인 제품 기능검사 기록 5개 파일을 하나로 합친다 — 요구사항을 번호로 적을수록 결과가 정확하다

**▶ 프롬프트 — Codex App**

C:\DEV\manufacturing 폴더에 있는 다음 Excel 파일 5개를 하나의 Excel 통합문서로 합쳐줘.
- 2020-01-21_A_Line.xlsx ~ 2020-01-25_A_Line.xlsx (5개)

작업 요구사항:

1. 각 파일의 `RC` 시트 데이터를 하나의 `통합 데이터` 시트에 세로로 이어 붙여줘.
2. 첫 번째 파일의 헤더만 사용하고, 나머지 파일의 헤더 행은 제외해줘.
3. 1월 21일과 22일 파일에만 있는 `BLE DEVICENAME`, `BLE RSSI` 열도 통합 열 구조에 포함해줘.
4. 특정 파일에 없는 열은 빈 셀로 두고, 열 이름을 기준으로 정확하게 배치해줘. 열 위치만 보고 합치지 마.
5. `원본 파일` 열을 새로 추가하여 각 행이 어느 파일에서 왔는지 기록해줘.
6. 날짜와 시간, 숫자, 제품 시리얼 및 MAC 주소의 데이터 형식을 보존해줘.
7. 전체 데이터에 필터를 적용하고 첫 번째 행을 고정해줘.
8. 중복 데이터는 임의로 삭제하지 말고 모두 유지해줘.
9. 별도의 `통합 요약` 시트에 원본 파일별 · 날짜별 · 검사 결과(`Result`)별 · 전체 데이터 건수를 정리해줘.
10. 원본 파일별 데이터 건수를 비교하는 막대 차트를 `통합 요약` 시트에 추가해줘.
11. 수식 오류와 누락 여부를 검사하고, 원본 파일별 행 수의 합계가 통합 데이터 행 수와 일치하는지 확인해줘.
12. 원본 파일은 변경하지 말고 결과를 C:\DEV\manufacturing\A_Line_통합_검사기록.xlsx 로 저장해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ A_Line_통합_검사기록.xlsx 를 만들었다.

통합 데이터 1,250행
· 01-21 250 · 01-22 250
· 01-23 250 · 01-24 250
· 01-25 250
BLE 열 2개 포함
(01-23~25 는 빈 셀)

검증
· 원본 합계 1,250
= 통합 1,250 ✓
· 수식 오류 0건
· 원본 파일 변경 없음

지금 할 일 C:\DEV\manufacturing 열기 ▸ 아래 프롬프트 전송 ▸ A_Line_통합_검사기록.xlsx 의 '통합 요약' 시트와 행 수 일치 여부 확인

# ⑦ 채팅(상담) · ⑧ 텍스트에서 이미지 생성

## ⑦ 채팅(상담) — 정답 없는 주제를 함께 정리

**▶ 프롬프트 — Codex App**

다운로드 폴더 정리하는 방법 알려줘. 카테고리 분류 기준과 중복 파일 처리 원칙도 함께 알려줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 1) 카테고리: 문서 · 이미지 · 설치파일 · 압축 · 기타
  2) 기준: 확장자 + 최근 사용일(90일)
  3) 중복: 이름이 같아도 크기 · 해시가 같을
     때만 중복으로 본다 → 최신 1개만 남기고
     나머지는 '_중복확인' 폴더로 이동
  4) 바로 지우지 말고 2주 보관 후 삭제
  원하면 이 규칙대로 정리하는 스크립트도
  만들어 드릴까요?
```

## ⑧ 텍스트 → 이미지 — 글로 설명해 새 이미지

**▶ 프롬프트 — Codex App**

다음 내용의 이미지 5장을 만들어서 핸드폰이미지 폴더에 저장해줘.
- 깨져서 불량인 스마트폰 이미지 5장
- 깨진 부분 없는 정상인 스마트폰 이미지 5장

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 핸드폰이미지 폴더에 10장을 저장했다.
  · 불량_1.png ~ 불량_5.png  (액정 균열 · 모서리 파손)
  · 정상_1.png ~ 정상_5.png  (같은 구도 · 손상 없음)
  불량 판정 모델 학습용이라면 배경과 각도를
  다양하게 더 만들 수 있다.
```

**지금 할 일** ⑦ 은 답을 읽고 내 상황으로 후속 질문 ▸ ⑧ 은 '핸드폰이미지' 폴더에 이미지 10장이 저장됐는지 확인

# ⑨ 기존 이미지 변형 · ⑩ 유사도 검색

## ⑨ 기존 이미지 변형 — 상태 · 스타일만 바꾼다

**▶ 프롬프트 — Codex App**

[정상_1.png] 이미지를 바탕으로, 같은 스마트폰 기종에 액정이 깨진 모습만 다르게 표현한 이미지를 새로 만들어서 같은 폴더에 저장해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ 정상_1.png 를 바탕으로 액정 균열만 표현한 정상_1_깨짐.png 를 같은 폴더에 저장했다.
· 기종 · 각도 · 배경은 원본과 동일
· 균열은 화면 오른쪽 위에서 대각선으로

## ⑩ 유사도 검색 — 가장 비슷한 과거 사례 찾기

**▶ 프롬프트 — Codex App**

[A_Line_통합_검사기록.xlsx](C:\DEV\manufacturing\A_Line_통합_검사기록.xlsx)에서, 아래 새로 접수된 불량 현상과 설명이 가장 비슷한 과거 사례를 상위 5건 찾아서 발생일과 함께 표로 정리해줘.
[새 불량 현상] 액정 하단부에 얼룩이 번지듯 나타나고, 터치 반응이 느려짐

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ 유사 사례 상위 5건

| 순위 | 발생일 | 현상 요약 | 유사도 |
|---|---|---|---|
| 1 | 2020-01-22 | 하단 얼룩 · 터치 지연 | 0.91 |
| 2 | 2020-01-24 | 화면 번짐 · 반응 느림 | 0.87 |
| 3 | 2020-01-21 | 하단 변색 | 0.79 |

…
공통 원인 후보: 하단 커넥터 접촉 불량

# ⑪ 음성 → 텍스트 · ⑫ 음성 → 영어 번역 저장

## ⑪ 음성 → 텍스트 — 녹음을 받아 적는다

**▶ 프롬프트 — Codex App**

[0905_회의.m4a](C:\DEV\meeting\0905_회의.m4a) 파일 내용을 텍스트로 받아 적어줘. 화자가 여러 명이면 발언자 A, B로 구분해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

```
Codex 응답
✦ 0905_회의.m4a 받아쓰기 (약 6분)
  A: 이번 주 라인 점검 결과부터 볼게요.
  B: 3라인 접착 온도가 기준보다 2도 높았다.
  A: 그럼 금형 점검을 목요일로 당기죠.
  …
  0905_회의.txt 로 저장할까요?
```

## ⑫ 음성 → 영어 번역 — 결과를 파일로 남긴다

**▶ 프롬프트 — Codex App**

[0905_회의.m4a](C:\DEV\meeting\0905_회의.m4a) 파일 내용을 듣고, 영어로 번역한 텍스트를 같은 폴더에 0905_meeting_en.txt로 저장해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

```
Codex 응답
✦ 0905_meeting_en.txt 를 저장했다.
  A: Let's start with this week's line inspection.
  B: Line 3's bonding temperature was 2°C above spec.
  A: Then let's move the mold check up to Thursday.
  …
```

TIP 유형은 섞어 써도 된다 — "이 녹음을 텍스트로 바꾸고(음성→텍스트), 영어로 번역한 뒤(번역), 5줄로 요약해줘(요약)" 처럼 이어 붙이면 순서대로 처리한다.

**지금 할 일** C:\DEV\meeting 의 0905_회의.m4a 로 실행 ▸ 12가지 중 3가지 이상 실행했는지 스스로 확인

# 참고 — CLI 도 같은 AI 모델을 쓴다 (설치 · 로그인)

App 실습에는 필수가 아니다 — 스크립트 · CI 자동화(8.8)를 대비해 '이런 것도 있다' 정도로 익힌다

터미널 (선택)

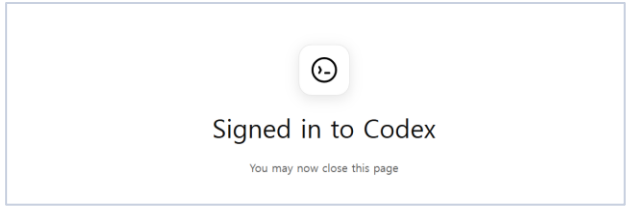
```
# Codex CLI 전역 설치 (Node.js 18+ 필요)
$ npm install -g @openai/codex

# 설치 확인 / 로그인
$ codex --version
$ codex login      # 브라우저로 ChatGPT 로그인

# macOS Homebrew: brew install codex 도 가능
```




① codex --version → codex-cli 0.144.5 · codex login




④ 'Signed in to Codex' — 이 페이지는 닫아도 된다



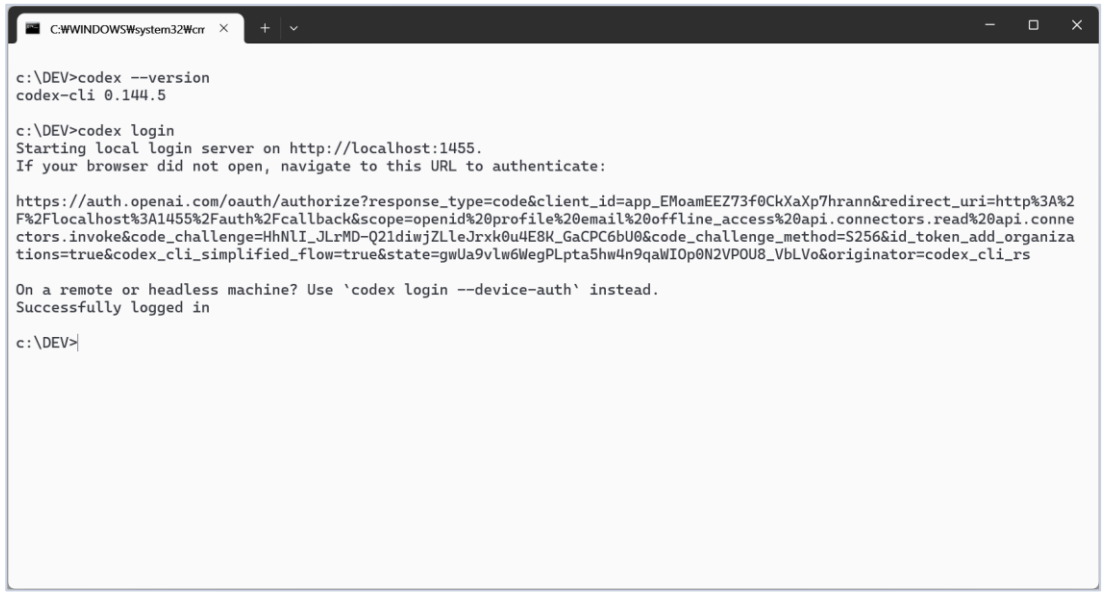

② 브라우저 로그인 창 — 이메일 또는 Google · Apple · Microsoft 계정




③ 계정 확인 → '계속'

# CLI 로그인 결과와 업데이트

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다

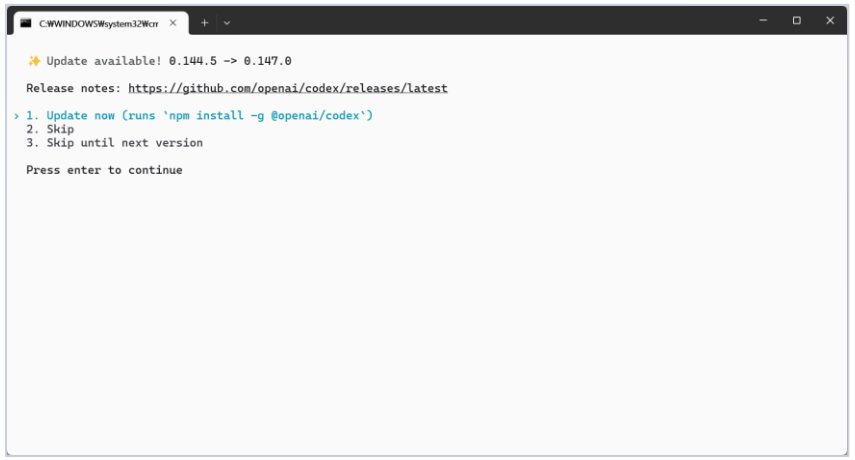



codex login 결과 — 로컬 로그인 서버 → Successfully logged in

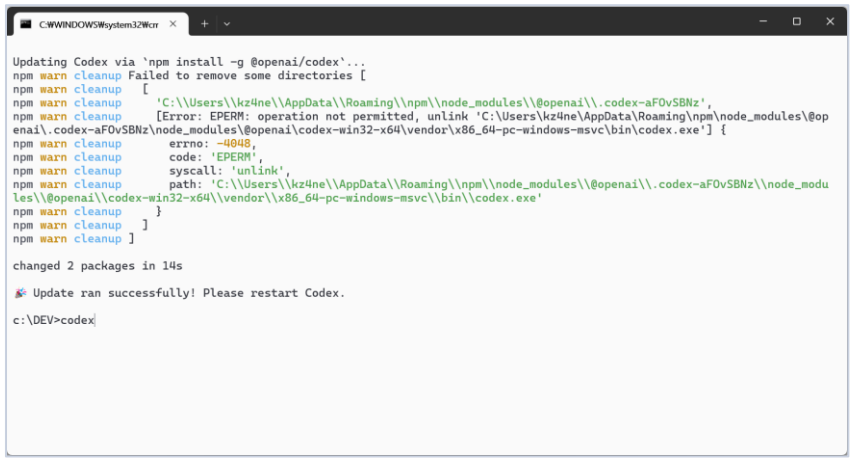



codex 실행

원격 · 헤드리스 환경이면 codex login --device-auth 를 쓴다.



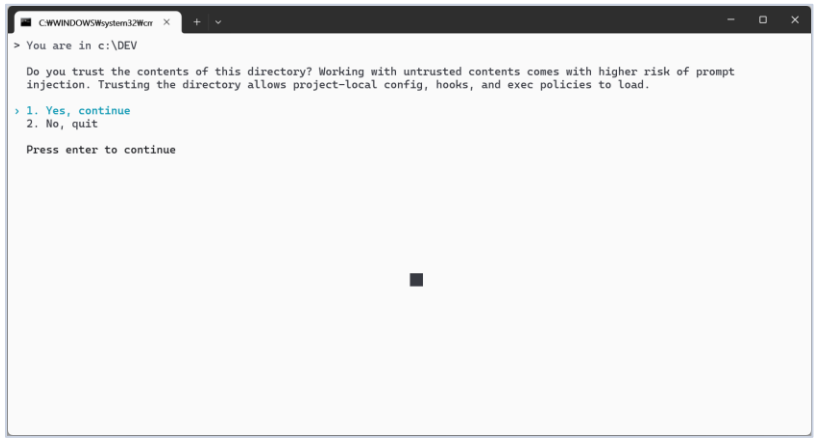

업데이트 안내 — 1. Update now (npm install -g @openai/codex)



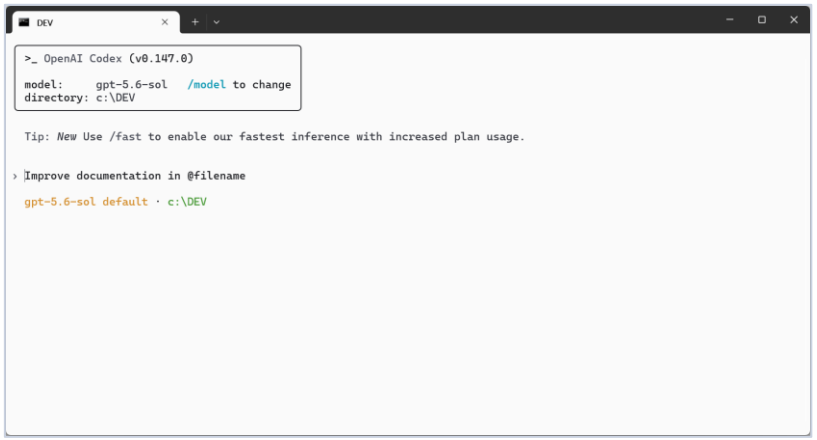

업데이트 완료 — 'Update ran successfully! Please restart Codex.'

**지금 할 일** (선택) 터미널에 'Successfully logged in' 확인 ▸ codex 실행 ▸ 업데이트 안내가 나오면 1. Update now

# CLI 첫 실행 — 폴더 신뢰 확인과 첫 질문




① 이 폴더를 신뢰할까요? → 1. Yes, continue




② 시작 화면 — model: gpt-5.6-sol · /model to change

**▶ 프롬프트 — Codex CLI**

현재 디렉터리의 폴더와 파일 구조를 확인하고 이 프로젝트가 무엇인지 설명해줘.



③ CLI 작성 줄(›)에 입력한 모습

**결과 예시** 응답 문구는 실행마다 다를 수 있다

```
Codex CLI

• Explored
  └ List C:\DEV
✦ C:\DEV 는 여러 실습 프로젝트를 모아 둔 작업 폴더이다.
  · codex-hello  — hello.txt 하나 (3.3 첫 작업)
  · manufacturing — 검사 기록 xlsx · CSV · 회의록
  · meeting      — 회의 녹음 m4a
  하나의 앱이 아니라 실습별 폴더 묶음이다.
```

04

CHAPTER 4 · 실습

# Codex App 사이드바 — 사이트 · 예약 · 플러그인

사이드바의 세 메뉴는 각각 '웹을 만들어 공유하기', '반복 작업을 자동으로 돌리기', 'Codex 의 기능을 늘리기'에 해당한다. 짧은 실습으로 하나씩 눈으로 확인한다.

4.1 한눈에 보기 | 4.2 사이트 | 4.3 예약 | 4.4 플러그인

# 사이드바 핵심 기능 — 사이트 · 예약 · 플러그인

왼쪽 사이드바에는 새 채팅 · 풀 리퀘스트 외에도 세 가지 메뉴가 있다 — 실습 폴더: C:\DEV\sidebar

## 사이트

Sites

프롬프트 한 줄로 웹페이지 · 앱 · 게임을 만들어 OpenAI 서버에 바로 호스팅. 미리보기 → 공개 범위 선택 → 링크 공유

Codex 에게 맡기는 배포

## 예약

Scheduled

'나중에 · 정기적으로' 할 일을 맡긴다. 매일 / 매주 반복, 변화 감시. 실행 결과는 예약 화면에 수신함처럼 모인다

반복 작업 자동화

## 플러그인

Plugins

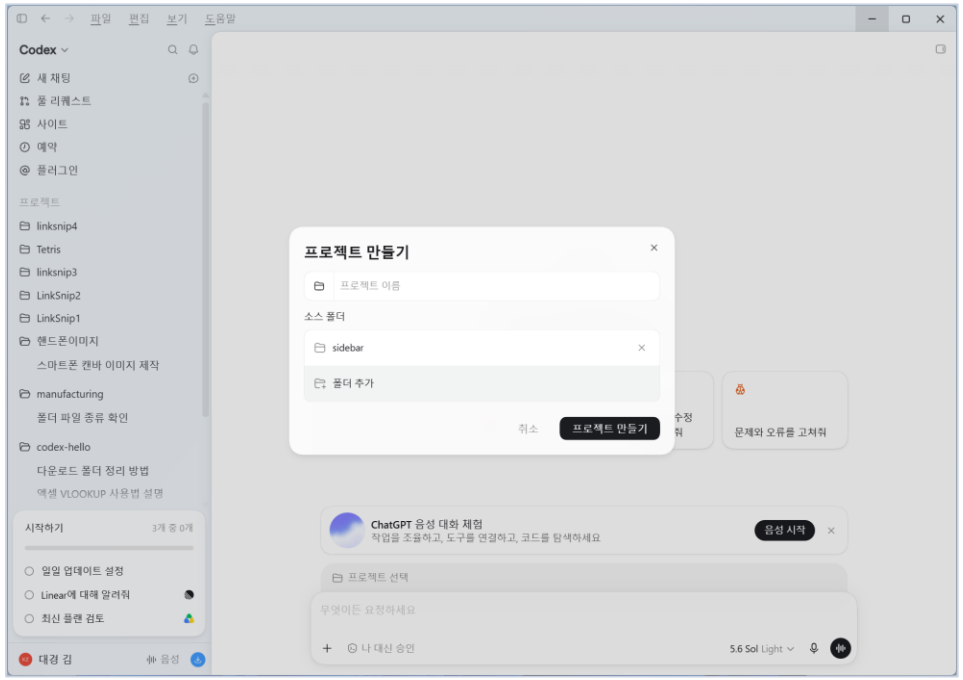
새 기능(스킬 · 앱 · 템플릿)을 더하는 확장 장터. 문서 · PDF · 스프레드시트 · 프레젠테이션이 기본 포함

기능 확장

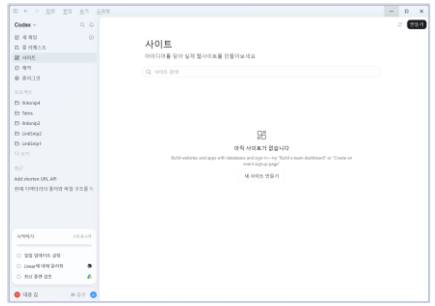
**TIP** 세 기능은 서로 연결된다 — 예약 작업이 플러그인 · 스킬을 함께 호출할 수 있다 (예: 매주 금요일 변경사항을 대시보드 사이트로 정리).

**참고** 메뉴 이름 · 위치는 앱 버전 · 요금제에 따라 다르다. 메뉴가 안 보이면 새 채팅 프롬프트에 서비스 이름(예: @Sites)을 직접 적어 호출한다.

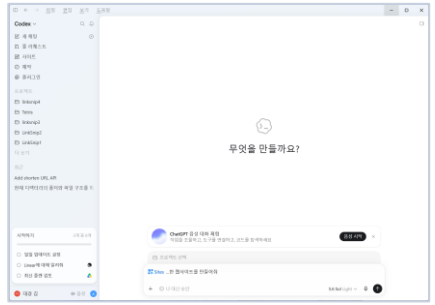
# 사이트 ① — 실습 프로젝트를 만들고 사이트 화면 열기




① 프로젝트 만들기 — 소스 폴더 sidebar




② 사이드바 → 사이트 (아직 사이트가 없음)




③ 새 채팅 — 작성 칸에 Sites 표시

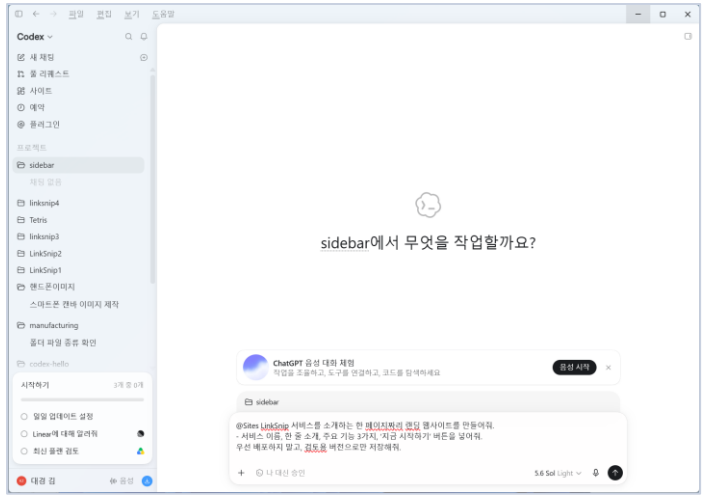
1. 왼쪽 사이드바에서 '사이트(Sites)'를 클릭한다. 메뉴가 안 보이면 새 채팅을 열고 프롬프트에 @Sites 를 쓴다.

2. 요청에 'website' 또는 '@Sites' 가 들어가면 Sites 작업이 시작된다 — 다음 슬라이드 프롬프트를 붙여 넣는다.

# 사이트 ② — 프롬프트로 랜딩 페이지 만들기

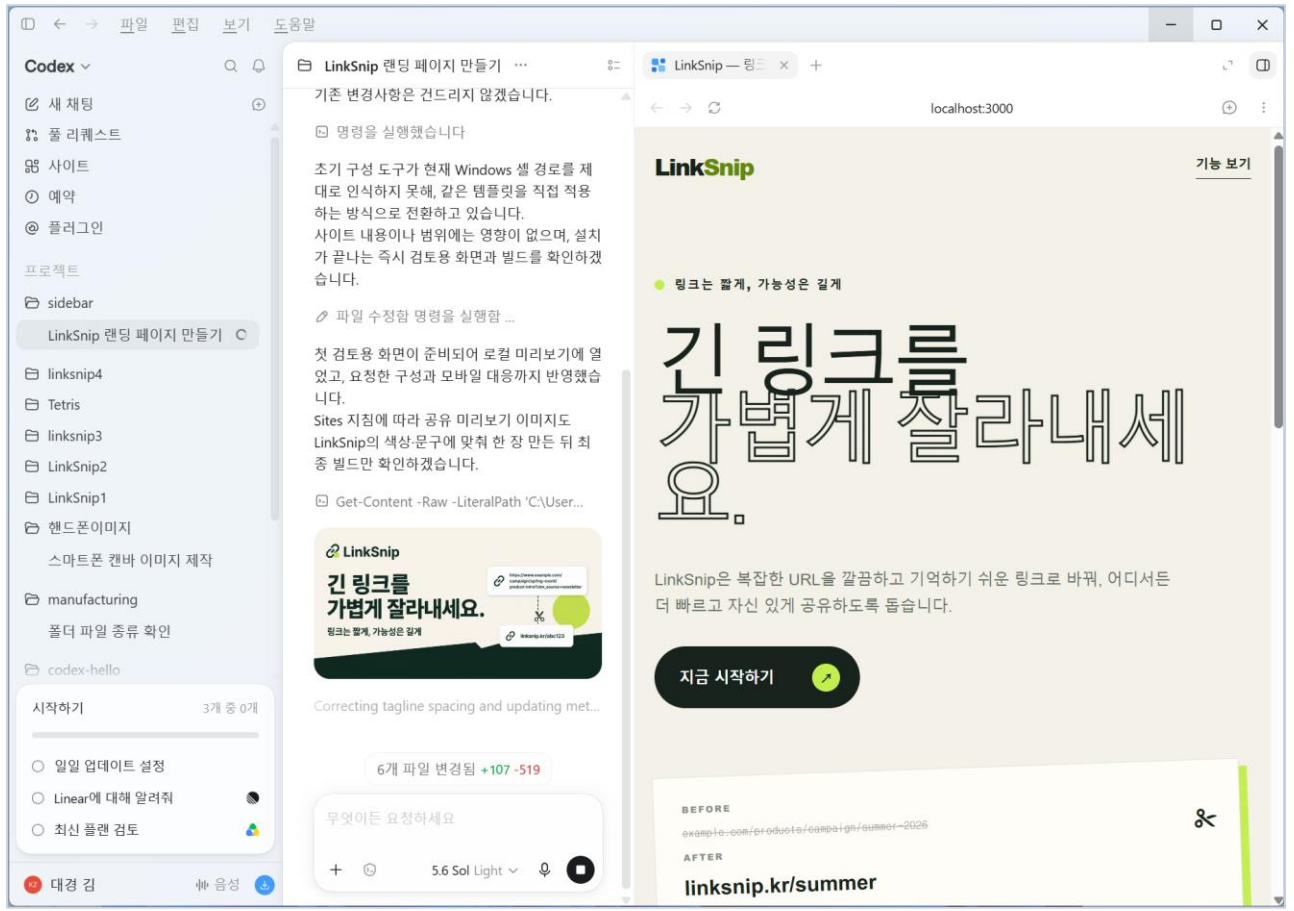
**▶ 프롬프트 — Codex App**

`@Sites` linksnip 서비스를 소개하는 한 페이지짜리 랜딩 웹 사이트를 만들어줘.
- 서비스 이름, 한 줄 소개, 주요 기능 3가지, '지금 시작하기' 버튼을 넣어줘.
우선 배포하지 말고, 검토용 버전으로만 저장해줘.




작성 칸에 붙여 넣은 모습

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




결과 — 검토용 버전과 미리보기 ('긴 링크를 가볍게 잘라내세요')

# 사이트 ③ — 수정 · 저장 · 배포 · 통계

Sites 공개는 '버전 저장 → 배포' 두 단계다

1. **미리보기로 확인**
   내용 · 링크 · 버튼이 의도대로 동작하는지 본다

2. **자연어로 수정 요청**
   "버튼 색을 파란색으로 바꾸고, 소개 문구를 더 간결하게 해줘."

3. **공개 범위를 골라 배포**
   나만 / 워크스페이스 / 인터넷 전체 → 받은 링크 공유

**▶ 프롬프트 — 수정 요청**

버튼 색을 파란색으로 바꾸고, 소개 문구를 더 간결하게 해줘.

**결과 예시: 미리보기의 버튼 색 · 문구가 바뀌고 새 '버전'이 저장된다.**

### NOTE — '저장'과 '배포'는 다르다

- 배포 URL 은 실제 공개 주소 — 공개 전에 개인정보 · 비밀값이 없는지 반드시 확인
- 새 Sites 는 기본적으로 만든 사람과 관리자에게만 보인다

### TIP — 방문 통계도 자동으로

- 배포 후 '사이트' → 해당 사이트 → 더 보기 → 분석(Analytics) 에서 방문자 수 · 조회 수 확인

### 주의 — 이용 가능 범위

- 프리뷰(베타) 기능 — 요금제(Plus · Pro · Business · Enterprise · Edu) · 지역 · 워크스페이스 설정에 따라 사용 가능 여부와 한도가 다르다
- 메뉴가 없거나 한도에 걸리면 이 실습은 읽기만 하고 넘어간다

# 예약 — 정해진 시간 · 주기에 작업을 맡긴다

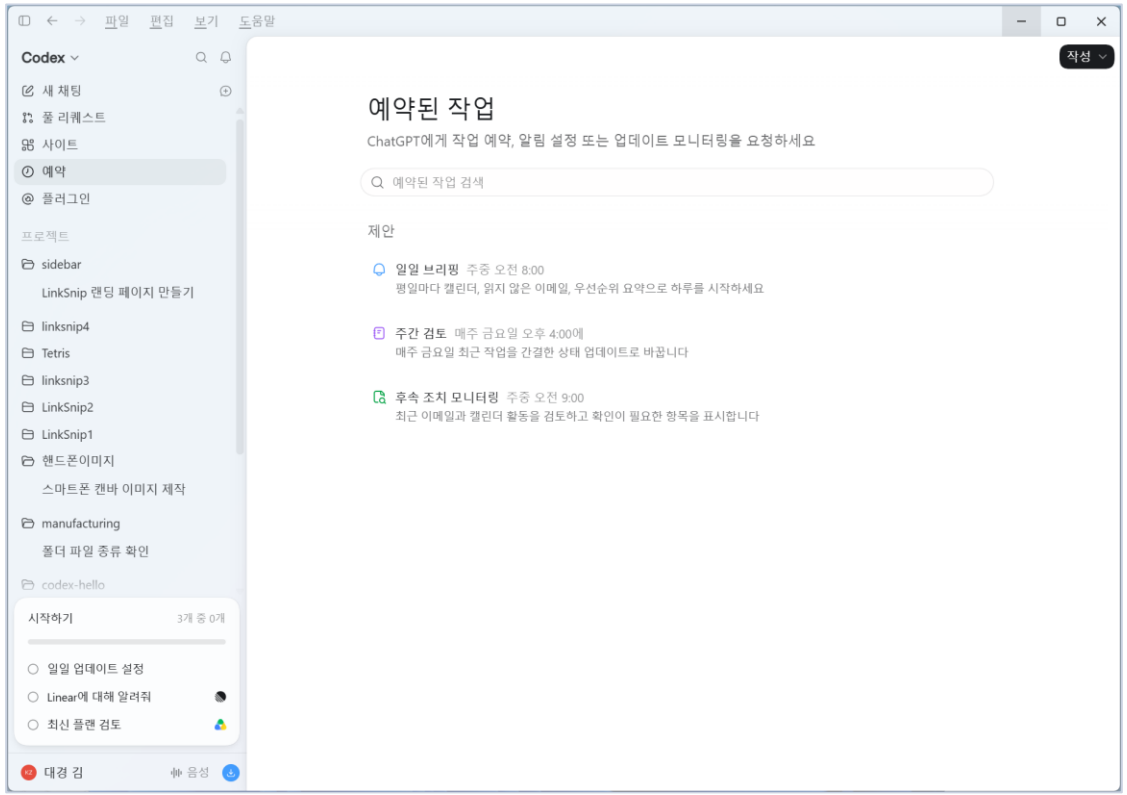
예: 매일 오전 8시 AI 뉴스 요약 · 매주 월요일 할 일 알림 · 조건이 충족되면 알림(모니터링)

## 예약 방식 세 가지

- ① 한 번만 실행
- ② 매일 · 매주 · 매월 반복 실행
- ③ 조건이 충족될 때까지 주기적으로 확인

## 이렇게 적는다

- 무엇을: AI 에이전트 최신 뉴스 핵심 5개 요약
- 언제: 매일 오전 8시 · 매주 월요일 9시
- 조건(선택): 중요한 뉴스가 있을 때만




사이드바 → 예약 — 예약된 작업과 지난 실행 결과가 모이는 곳




'작성' → Codex로 만들기 / 수동으로 설정

한 문장으로 적어도 된다:
"매일 오전 8시에 생성형 AI와 AI Agent 주요 뉴스를 찾아서 한국어로 요약해줘"

# 예약 실습 ① — 주간 업계 뉴스로 게시글 아이디어

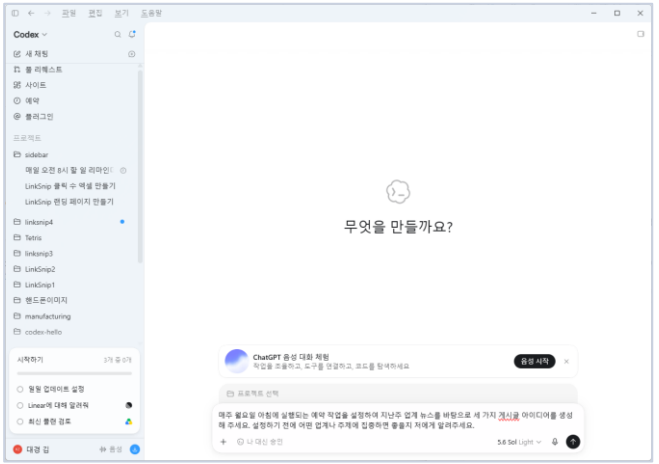
**▶ 프롬프트 — Codex App**

매주 월요일 아침에 실행되는 예약 작업을 설정하여 지난주 업계 뉴스를 바탕으로 세 가지 게시글 아이디어를 생성해 주세요. 설정하기 전에 어떤 업계나 주제에 집중하면 좋을지 저에게 알려주세요.

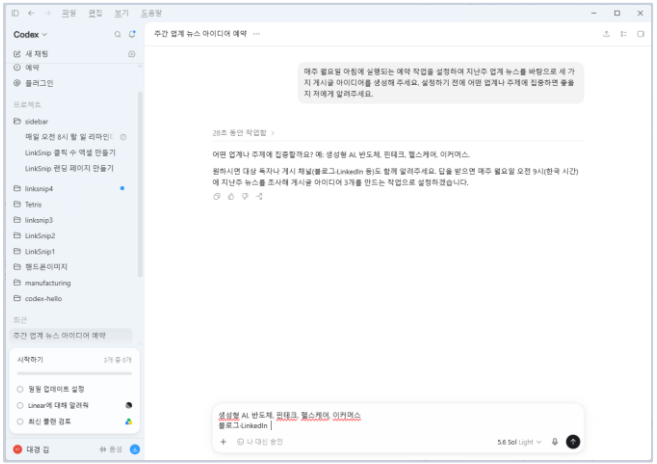
## 포인트

- '설정하기 전에 저에게 알려주세요' — 모델이 먼저 질문하게 한다 (5.4 원칙 14)
- 답한 내용이 예약 작업의 프롬프트가 된다
- 실행 주기 · 대상 채널(블로그 · LinkedIn)도 함께 알려 준다

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다

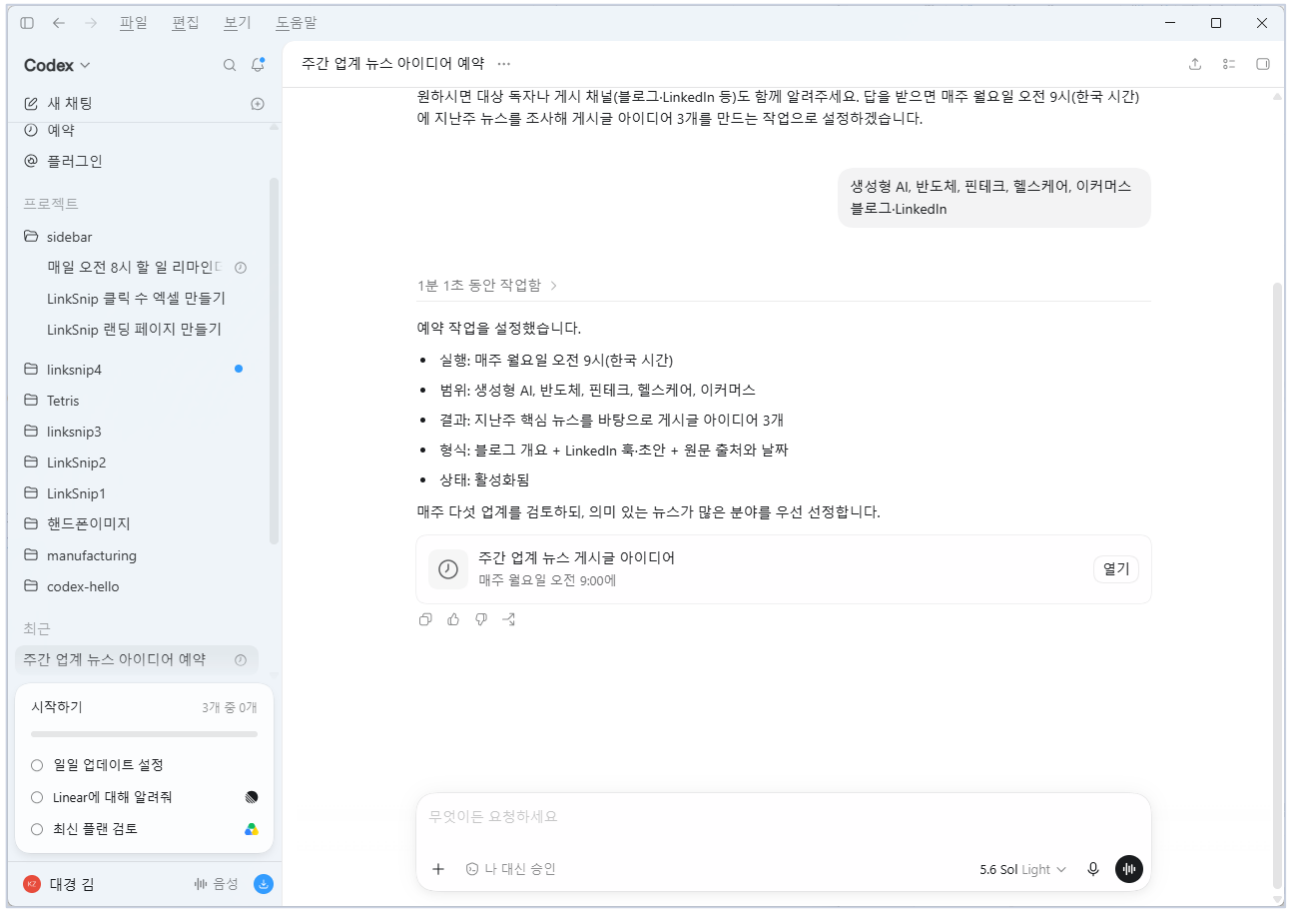


작성 칸에 입력

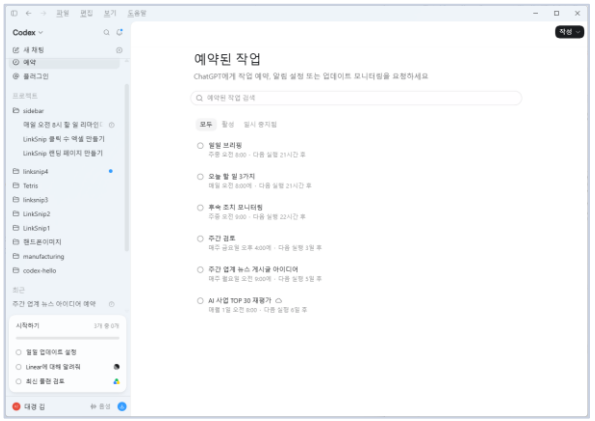

Codex 가 먼저 묻는다 → 업계 · 채널로 답한다

# 예약 실습 ① 결과 — 예약 작업이 등록된다

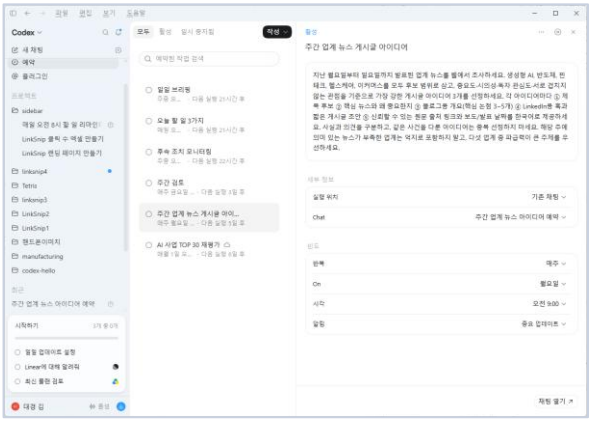
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




채팅 — '예약 작업을 설정했다' (매주 월요일 9:00)




예약 화면 — 등록된 작업 목록

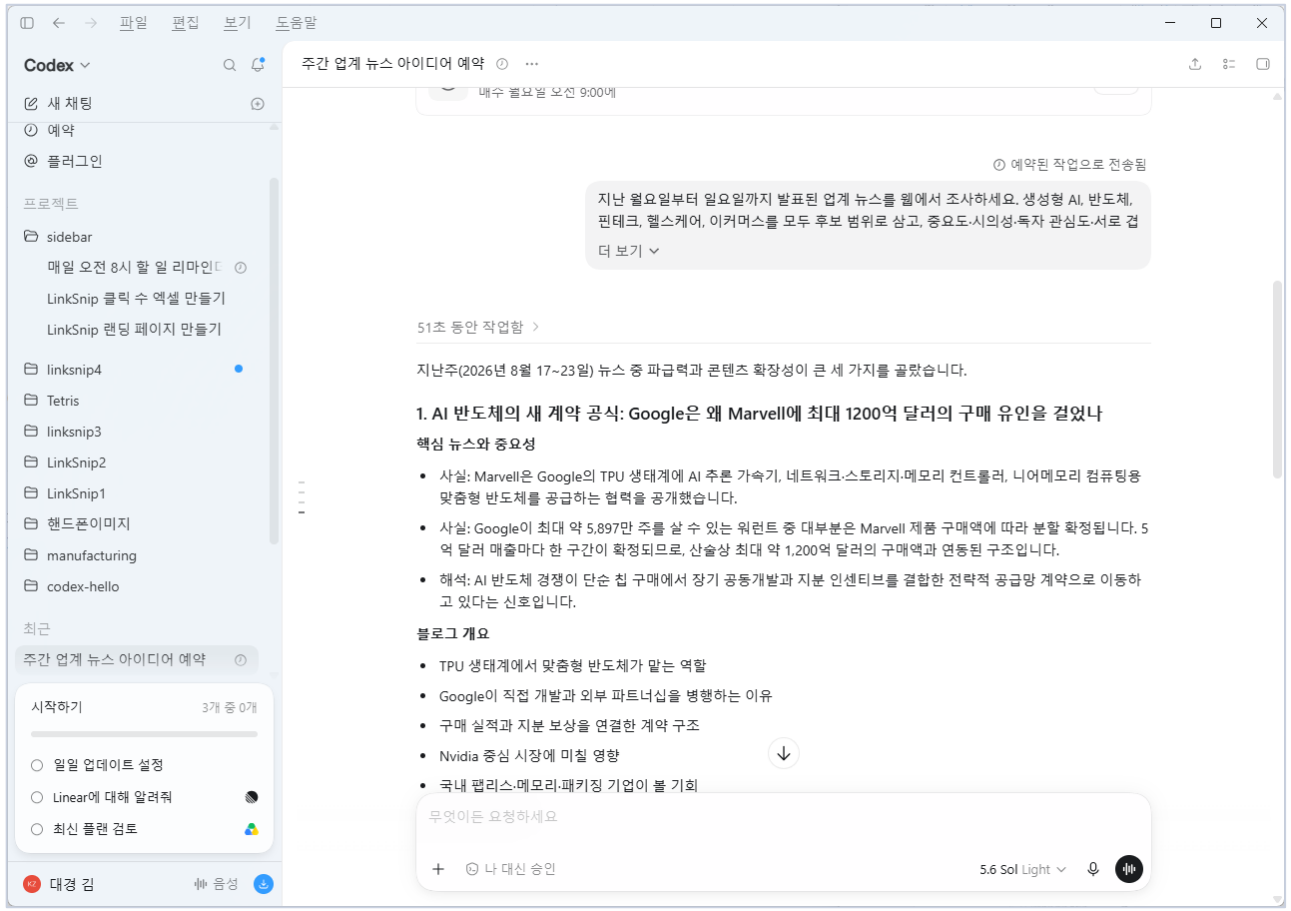


작업 상세 — 반복 매주 · 월요일 · 오전 9:00

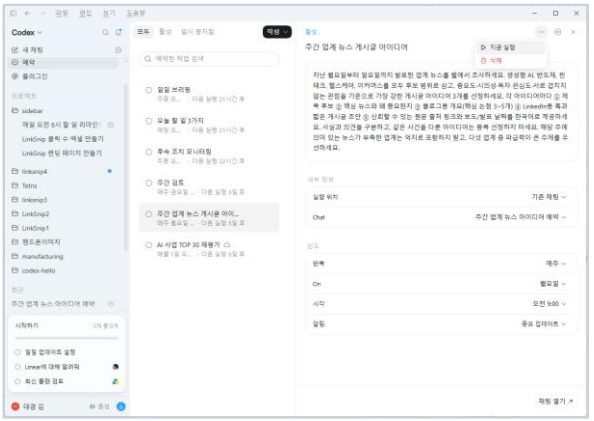
**지금 할 일** 채팅의 예약 카드 확인 ▸ 사이드바 '예약'에서 새 작업 확인 ▸ 작업을 눌러 반복 · 요일 · 시각 확인

# 예약 실습 ① — 지금 실행해 보고 결과 채팅 열기

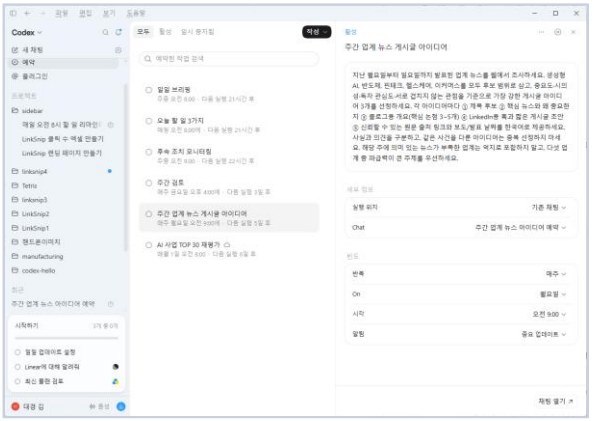
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




③ 결과 — 지난주 뉴스 기반 게시글 아이디어



① … 메뉴 → 지금 실행



② 실행 기록 → 채팅 열기

**지금 할 일** 작업 상세에서 '지금 실행' 클릭 ▸ 실행이 끝나면 '채팅 열기' ▸ 게시글 아이디어 3개 확인

# 예약 실습 ②③ — 키워드 모니터링과 프로젝트 반복 작업

## ② 관심 주제 매일 모니터링

**▶ 프롬프트 — Codex App**

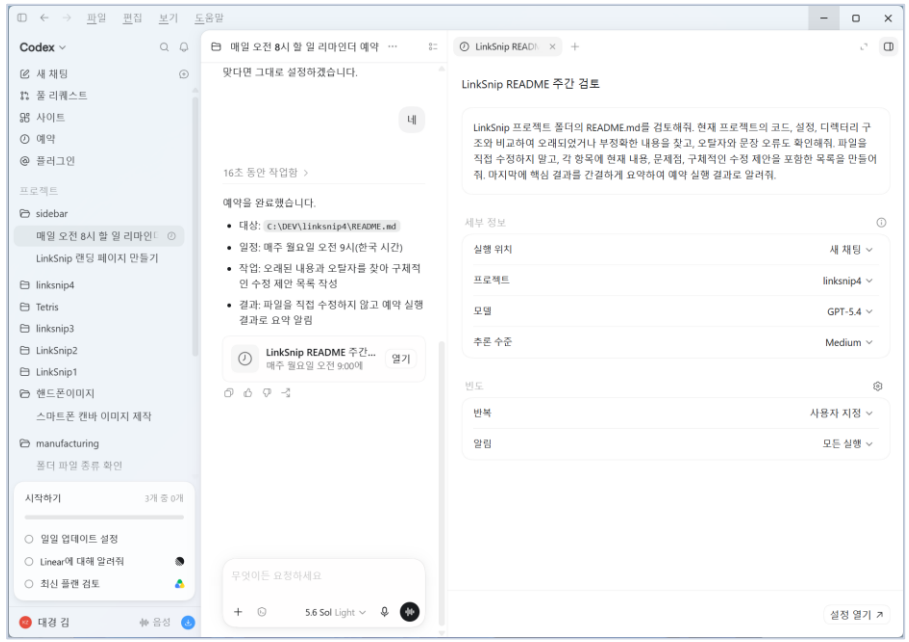
관심 있는 주제에 대한 뉴스나 언급이 있는지 매일 확인하고 새로운 내용을 요약해주는 예약 작업을 설정해주세요. 설정하기 전에 어떤 주제, 경쟁사 또는 키워드를 주시해야 하는지 저에게 알려주세요.

## ③ 프로젝트와 연결된 반복 작업

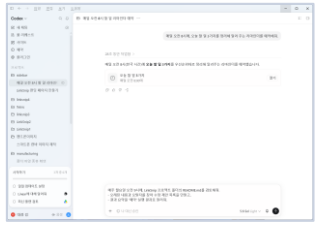
**▶ 프롬프트 — Codex App**

매주 월요일 오전 9시에, linksnip(sidebar) 프로젝트 폴더의 README.md를 검토해줘.
- 오래된 내용과 오탈자를 찾아 수정 제안 목록을 만들고,
- 결과 요약을 '예약' 실행 결과로 알려줘.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




결과 — 'linksnip README 주간 검토' 예약 등록 · 상세



③ 입력 — 예약 요청

예약 카드의 '대상 · 일정 · 작업 · 결과' 네 줄을 확인한다. 대상 경로가 내 폴더와 다르면 대화로 바로 고친다.

# 독립 예약 vs 대화 속 예약 — 관리와 활용

예정된 시간이 되면 Codex 가 스스로 실행하고, 결과는 '예약' 화면에 모인다

**1 실행 주기 확인 · 저장**

매일 / 매주 / 사용자 지정 — 더 세밀한 주기는 사용자 지정 스케줄

**2 결과 확인 · 관리**

'예약' 화면에서 결과 확인, 필요하면 멈춤(pause) · 재개 · 삭제

| 구분 | 독립형 예약 | 대화 속 예약 |
|---|---|---|
| 실행 방식 | 매번 새 채팅에서 저장된 프롬프트를 처음부터 | 진행 중인 대화의 맥락을 이어서 |
| 언제 | 매번 독립적으로 실행해야 할 때 | 맥락 유지가 중요할 때 |

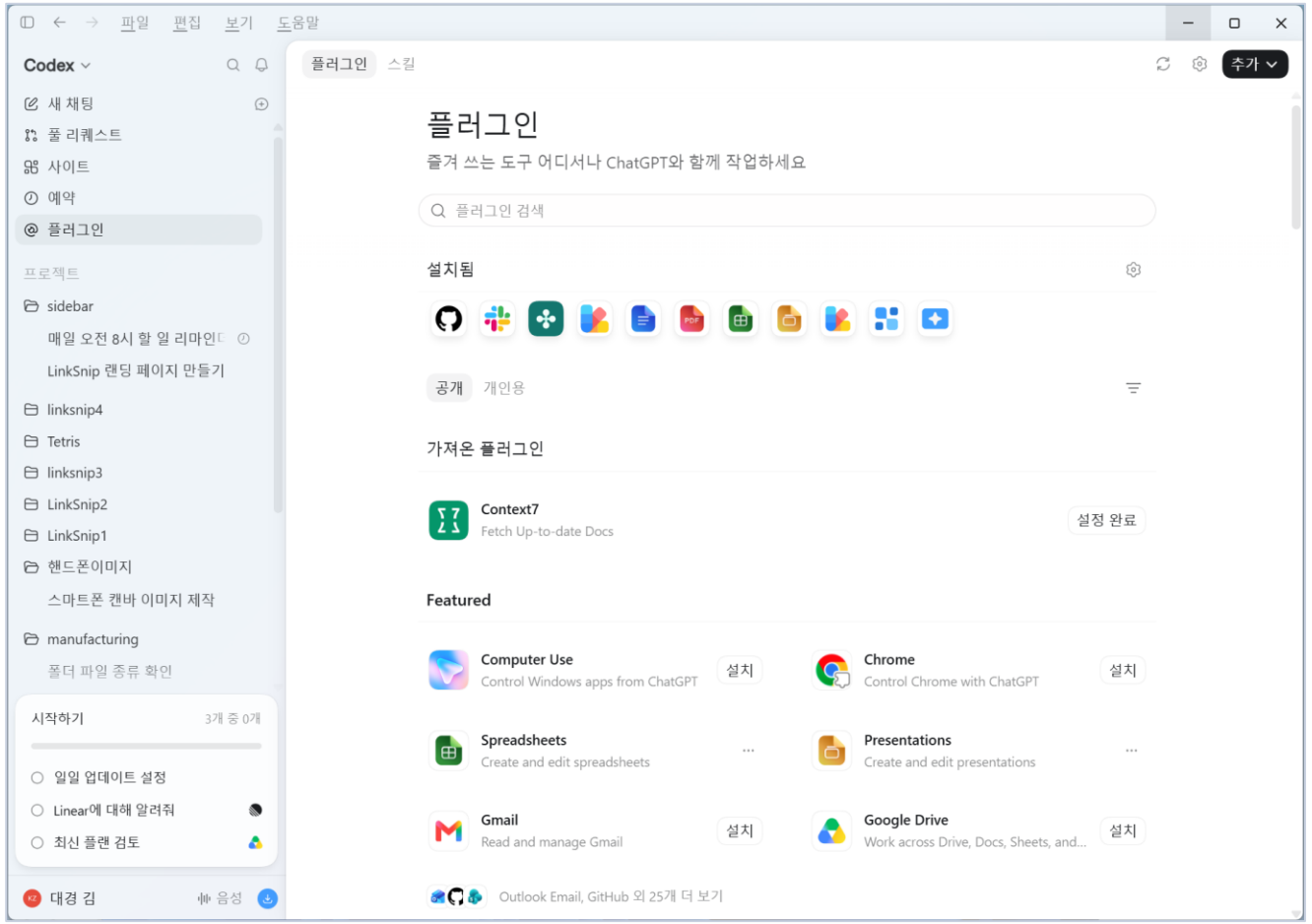
**TIP** 예약 작업은 설치한 플러그인 · 스킬 · 업로드한 파일 · 연결된 도구를 함께 쓸 수 있다 — '매주 금요일에 이번 주 변경사항을 대시보드 사이트로 정리해줘'처럼 사이트 · 플러그인과 엮는다.

### 4.3 확인

- ✅ 사이드바의 '예약' 화면을 열어 봤다
- ✅ 간단한 예약을 하나 만들어 '지금 실행'까지 해 봤다
- ✅ 독립 예약과 대화 속 예약의 차이를 이해했다

# 플러그인 ① — 디렉터리 둘러보기

ChatGPT 와 Codex 는 같은 플러그인 카탈로그를 공유한다




플러그인 디렉터리 — 설치됨 · 가까운 플러그인 · Featured

1. 사이드바에서 '플러그인'을 클릭해 디렉터리(카탈로그)를 연다.

2. 기본 플러그인을 확인하고, 필요한 것은 찾아 설치한다.

### NOTE — 플러그인은 어디서 오나

- OpenAI 제공 · 워크스페이스(회사) 제공 · 개인 마켓플레이스
- 설치한 플러그인의 스킬은 새 채팅(새 세션)부터 사용

### 주의 — 로그인 방식에 따른 제한

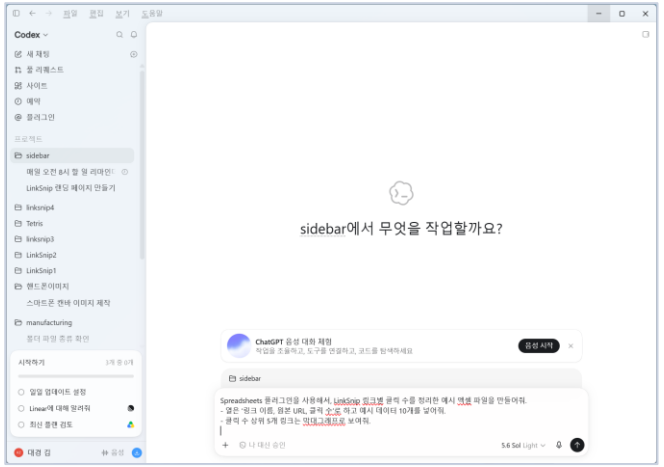
- API 키 로그인은 일부 플러그인 미지원
- 회사 관리자 정책으로 설치가 제한될 수 있다

**지금 할 일** 사이드바 '플러그인' 클릭 ▸ 설치된 기본 플러그인(Documents · PDF · Spreadsheets · Presentations) 확인

# 플러그인 ② — Spreadsheets 로 예시 엑셀 만들기

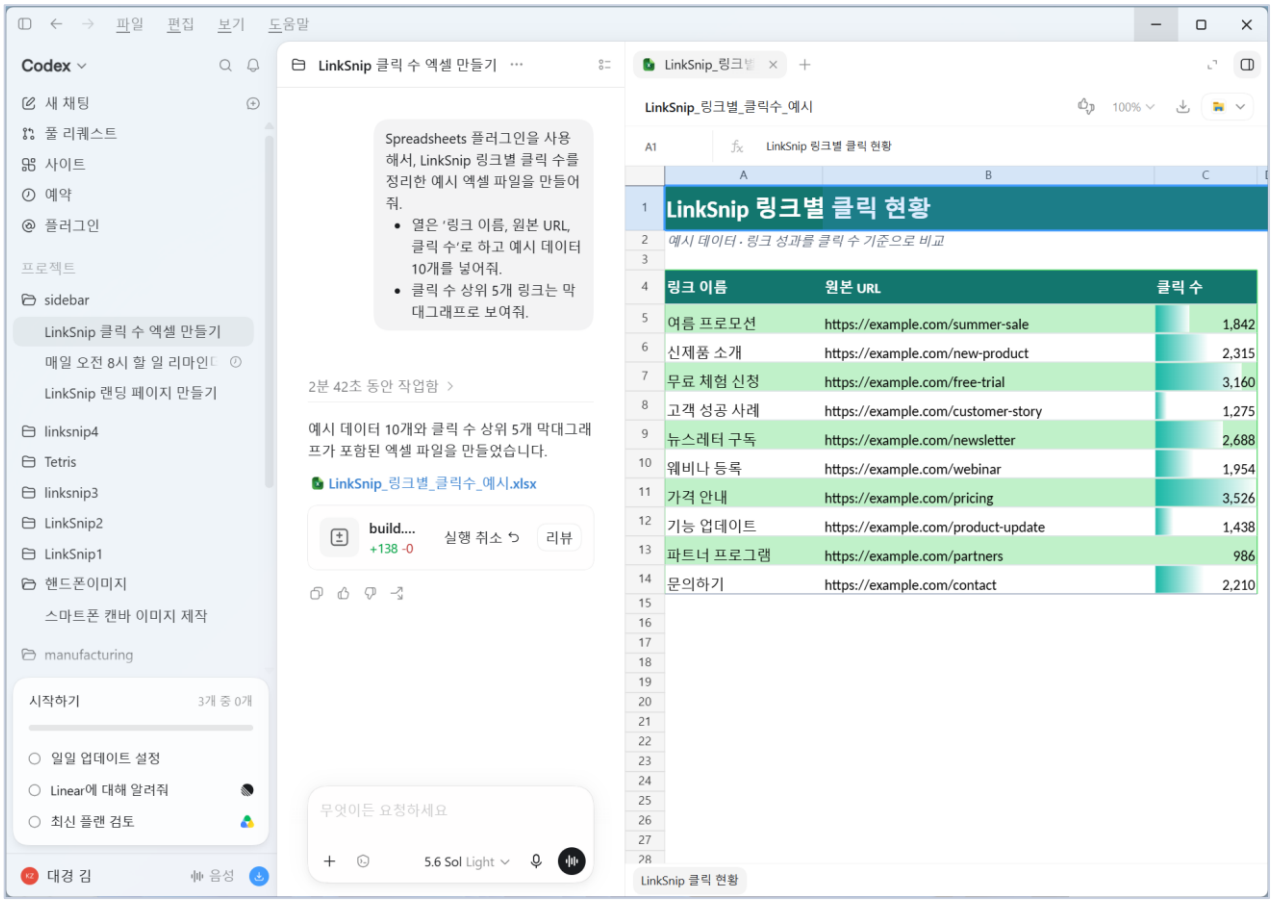
**▶ 프롬프트 — Codex App**

Spreadsheets 플러그인을 사용해서, linksnip 링크별 클릭 수를 정리한 예시 엑셀 파일을 만들어줘.
- 열은 '링크 이름, 원본 URL, 클릭 수'로 하고 예시 데이터 10개를 넣어줘.
- 클릭 수 상위 5개 링크는 막대그래프로 보여줘.




작성 칸에 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다

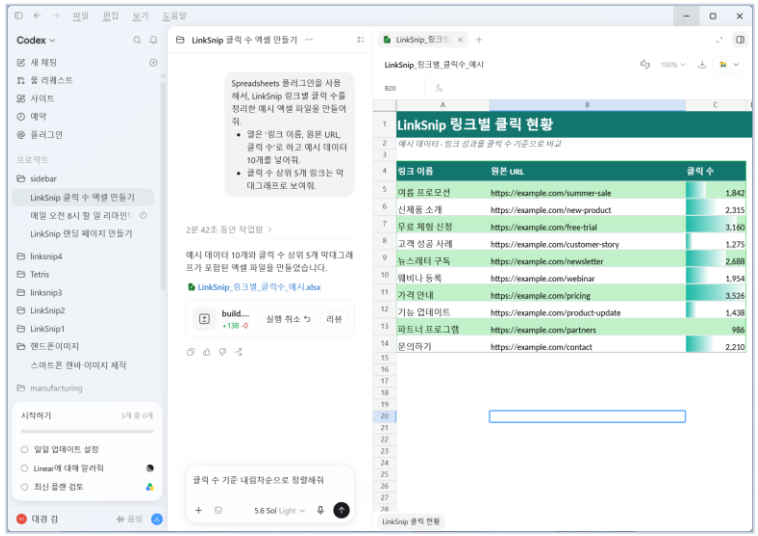



결과 — 'linksnip 링크별 클릭 현황' 시트 (상위 5개 강조)

# 플러그인 ③ — 후속 지시로 다듬기

**▶ 프롬프트 — 후속 지시**

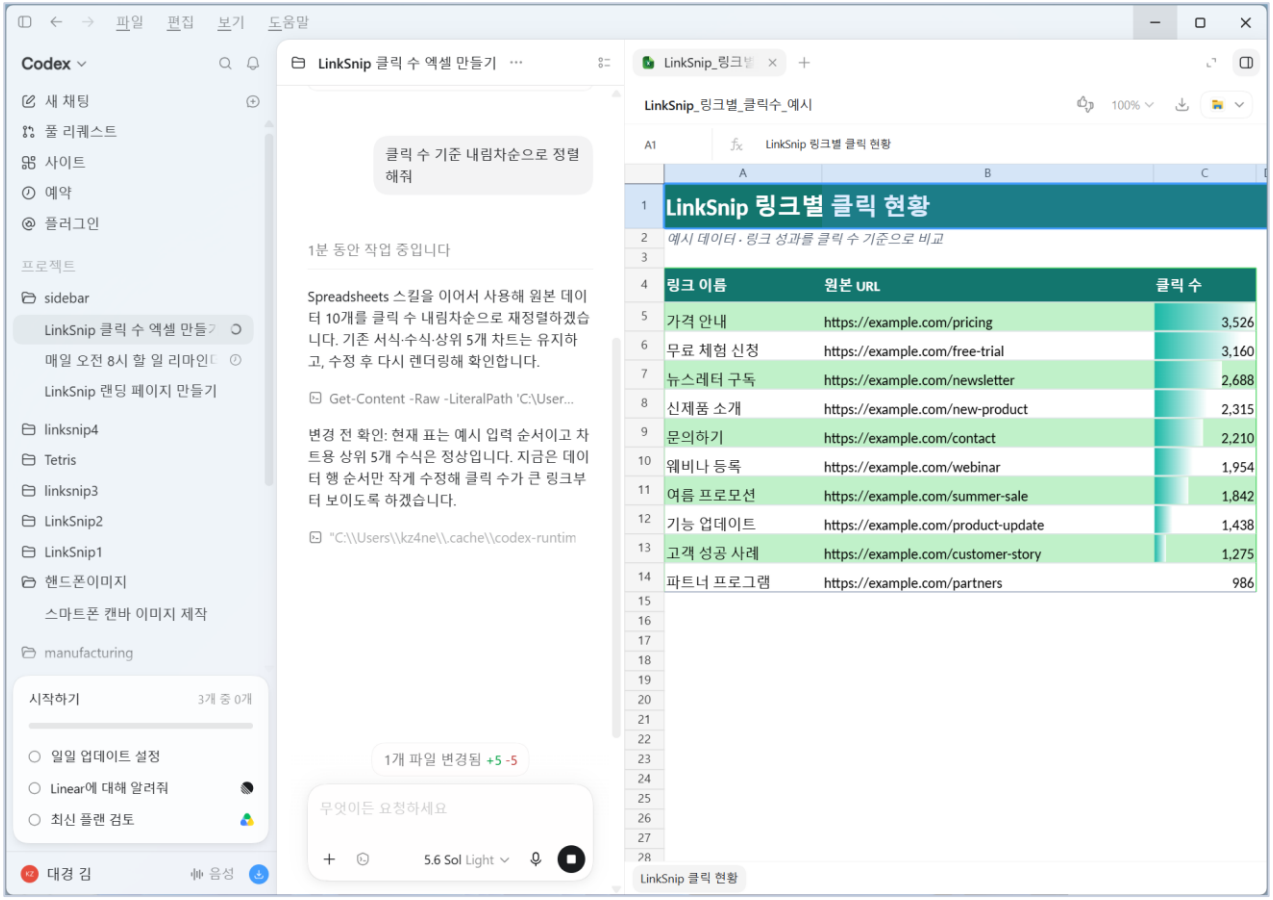
클릭 수 기준 내림차순으로 정렬해줘.



후속 지시 입력

한 번에 완벽하게 시키지 않는다 — 같은 스레드라 앞의 파일 · 열 구조를 기억하므로 짧은 후속 지시로 다듬는다.

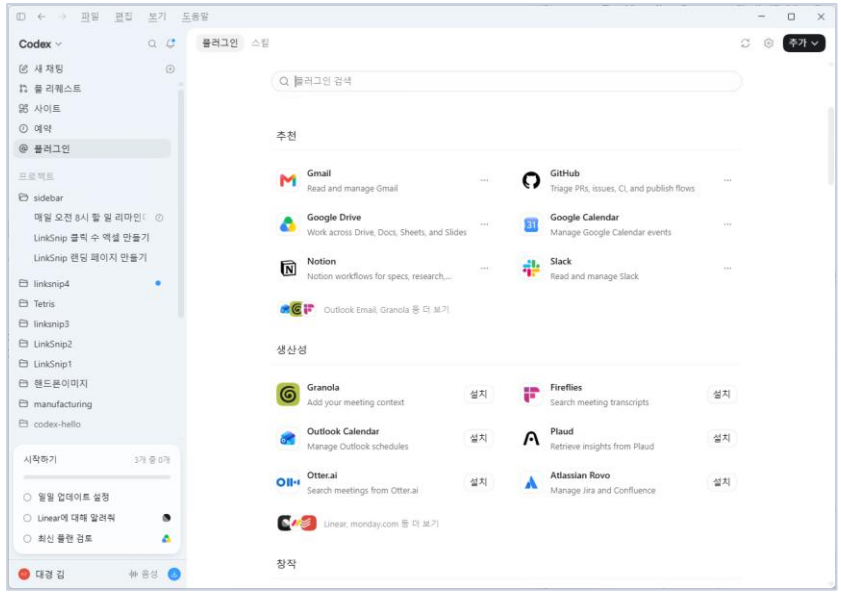
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




결과 — 클릭 수 내림차순으로 정렬 (가격 안내 3,536 → …)

# 추천 — Gmail · Notion 플러그인으로 메일을 문서화

Gmail 과 Notion 계정이 이미 있어야 한다 — 연결(로그인) 승인 화면이 나오면 허용한다




플러그인 추천 — Gmail · Google Drive · Notion · Slack …

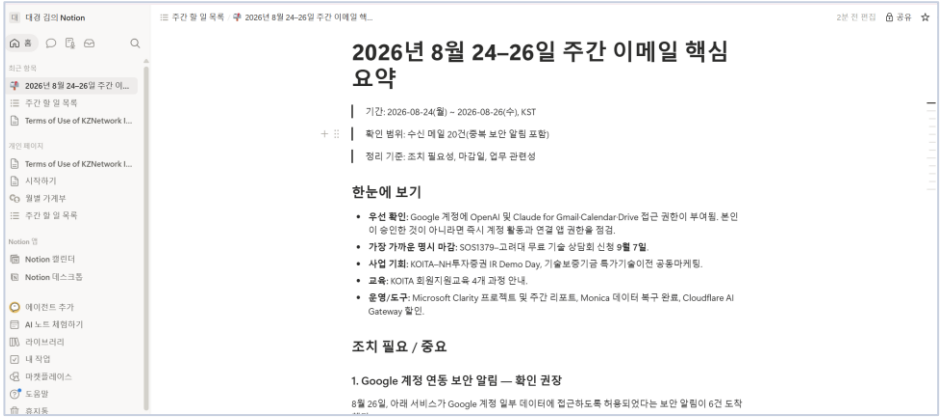
**프롬프트 — Codex App**

이번주에 온메일들 **@Gmail에서** 다 확인하고 핵심내용들 정리해서 **@notion에** 새페이지 만들어서 문서화해줘.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




Codex — 메일 20건 확인 → Notion '주간 할 일 목록' 아래 새 페이지 생성




Notion — '2026년 8월 24–26일 주간 이메일 핵심 요약'

지금 할 일 (선택) 플러그인에서 Gmail · Notion 설치 · 연결 ▸ 아래 프롬프트 ▸ Notion 에 새 페이지가 생겼는지 확인

05

CHAPTER 5 · 강의 + 실습

# 프롬프트 엔지니어링 — 받은 만큼 일한다

프롬프트는 Codex 에게 주는 '작업 지시문'이다. 핵심은 (1) 작업을 작고 명확하게 쪼개고, (2) '끝났는지 아는 법(검증)'을 함께 주는 것이다. 예제는 URL 단축기 linksnip 을 기준으로 한다.

5.1–5.5 개념 · 원칙 | 5.6 작게 쪼개기 | 5.7 기법 8가지 | 5.8 Goal | 5.9–5.10 패턴 · 연습

# 프롬프트 엔지니어링의 정의 — 한 줄 지시에서 실행 구조로

같은 기술을 사람마다 다르게 정의하지만, 공통점은 '자연어로 모델을 프로그래밍한다'는 것이다

## 프롬프트 엔지니어링

2022 ~ 2023

**한 번의 요청 문장을 잘 쓰는 기술**

예) "단계별로 생각해", 역할 부여

→ 5장

## 컨텍스트 엔지니어링

2025 ~

**모델이 보는 정보 환경 전체를 설계**

지시문 · 예시 · 검색 결과(RAG) · 도구 출력 · 대화 이력 · 메모리

→ 6장

## 하네스 엔지니어링

에이전트 시대

**세션을 넘어 계속 작동하는 실행 구조를 설계**

규칙 파일 · 설정 · 테스트 · 금지 규칙

→ 7장 · 8장

출처 · A. Karpathy, X(2023.1.24) · A. Karpathy,「Software Is Changing (Again)」YC AI Startup School(2025.6) · A. Ng & I. Fulford, DeepLearning.AI「ChatGPT Prompt Engineering for Developers」(2023)

# 나쁜 프롬프트 vs 좋은 프롬프트 — Codex 는 받은 만큼 일한다

같은 의도라도 표현에 따라 결과가 달라진다

✕ 나쁜 프롬프트

로그인 기능 만들어줘.

대상 파일 · 동작 규칙 · 검증 방법이 모두 빠졌다 → Codex 가 알아서 추측 → 결과가 매번 달라진다.

✓ 좋은 프롬프트

src/auth.ts 에 이메일+비밀번호 로그인 함수 loginWithEmail(email, password)를 추가해줘.
- 입력값이 비어 있으면 Error("INVALID_INPUT")를 던진다
- 비밀번호는 bcrypt로 해시 비교한다
- 성공 시 { userId, token } 을 반환한다

끝나면 tests/auth.test.ts 에 성공/실패 케이스 테스트를 추가하고 npm test 를 실행해서 모두 통과하는지 보여줘.

## 무엇이 달라졌나

- ① 대상 파일 / 함수 이름 지정
- ② 동작 규칙 구체화
- ③ 반환 형태 명시
- ④ 검증 방법(테스트 실행) 포함
- → Codex 가 '정답에 도달했는지' 스스로 확인할 길이 생겼다

## 좋은 프롬프트의 두 가지 핵심 원칙

1 작업을 작고 명확하게 쪼갠다 — 큰 작업은 '먼저 계획을 세우라'고 시킨 뒤 한 단계씩
2 '끝났는지 어떻게 아는지'를 함께 준다 — 테스트 · 실행 · 린트 등 스스로 검증할 방법

→ 이 둘을 매번 챙기는 체크리스트가 P · C · T · F (5.3)

# 재사용 템플릿 — P · C · T · F

네 글자만 외우면 거의 모든 작업에 적용되는 체크리스트 (순서가 아니라 점검 항목)

## P — Persona / Role

**역할 · 관점**

어떤 전문가의 관점으로 일하게 할지

예) "보안을 중시하는 TypeScript 개발자로서"

## C — Context

**배경 · 제약**

건드릴 파일 · 지켜야 할 규칙(스타일 · 금지사항 · 의존성) · 참고 자료

예) @src/auth.ts 수정 · 비밀번호 평문 저장 금지

## T — Task

**할 일 · 검증**

무엇을 하고, 성공을 어떻게 확인하는가 (테스트 / 실행 / 린트)

예) 로그인 구현 → 실패 시 에러 반환 → npm test 통과

## F — Format

**산출물 형태**

결과를 어떤 형태로 받을지 (변경 요약 · diff · 실행 결과)

예) 변경 요약과 diff 로 보여줘

**핵심** 네 조각을 이어 붙이면 곧 5.2 의 '좋은 프롬프트'다. 필요 없는 요소는 건너뛰어도 되지만, T 의 '검증 방법'은 절대 빼지 않는다.

# P · C · T · F 템플릿을 채운 예 — 결제 실패 재시도

**▶ 프롬프트 — P · C · T · F 범용 템플릿**

P: 결제 흐름을 잘 아는 신중한 프론트엔드 개발자로서 작업해줘.
C: 대상은 src/pages/Checkout.tsx 와 src/api/pay.ts.
  제약 — 기존 디자인 토큰만 사용, 새 라이브러리 추가 금지.
T: 결제 실패 시 사용자에게 재시도 버튼을 보여 준다.
  검증 — npm run build 와 npm test 가 통과해야 한다.
F: 변경한 파일 목록과 핵심 변경점 3줄 요약으로 보여줘.

**빈 템플릿**

```
# P (역할) : 어떤 전문가로서 일할지 한 줄
# C (맥락) : 대상 파일/모듈 + 지켜야 할 규칙
# T (작업) : 할 일 + 검증 방법
# F (형식) : 결과물 형태
```

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 변경한 파일
  · src/pages/Checkout.tsx
  · src/api/pay.ts
  핵심 변경점
  1. pay() 실패 시 PaymentError 를 던진다
  2. 실패 상태에서 '다시 시도' 버튼 표시
  3. 재시도는 같은 주문 ID 로 1회 호출
  > npm run build  ✓
  > npm test       12 passed
```

# 논문으로 검증된 26가지 원칙 ① — 구조 · 명확성 · 구체성

Bsharat 외(MBZUAI, 2023) — GPT-4 기준 응답 품질 평균 57.7%↑ · 정확도 36.4%↑ (ATLAS 벤치마크) · ★ 효과가 특히 컸던 원칙

## ① 프롬프트 구조와 명확성

| # | 원칙 | 예시 |
|---|---|---|
| 2 | 청중을 명시한다 ★ | "독자는 이 분야 전문가다" |
| 4 | "하지 마" 대신 "하라"로 긍정 지시 | "짧게 써"(○) / "길게 쓰지 마"(△) |
| 5 | 쉬운 설명을 요청한다 ★ | "11살에게 설명하듯", "입문자에게" |
| 7 | 예시를 준다 (Few-shot) | 입력 → 출력 예시 2~3개 |
| 8 | ###지시### ###예시### ###질문### 으로 구역을 나눈다 | 지시 · 예시 · 질문 · 자료를 분리 |
| 12 | "단계별로 생각해"를 쓴다 | 수학 · 논리 · 디버깅 |
| 17 | 구분자를 쓴다 | """ … """, <자료>…</자료> |
| 20 | 답변의 첫머리를 프롬프트 끝에 써 준다 | "…요약:" 으로 끝내기 |

## ② 구체성과 정보

| # | 원칙 | 예시 |
|---|---|---|
| 13 | 편향을 피하라고 명시한다 | "편향 없이 답해" |
| 15 | 가르치고 시험 보게 한다 ★ | "가르치고 끝에 테스트를 내줘" |
| 21 | 상세한 글을 요청한다 | "필요한 정보를 모두 넣어 자세히" |
| 24 | 주어진 첫 문장에 이어 쓰게 한다 | "이 문장으로 시작해 완성해" |
| 25 | 지켜야 할 요구사항을 명시한다 ★ | 키워드 · 규칙 · 힌트 · 형식 |
| 26 | 샘플과 같은 문체로 쓰게 한다 ★ | "같은 언어 · 문체로 써줘" |

**★ 대형 모델에서 효과가 가장 컸던 원칙 — 2 · 5 · 14 · 15 · 16 · 25 · 26. 특히 14번(모델이 먼저 질문)은 적용한 모든 질문에서 개선됐다.**

출처 · Bsharat, S. M., Myrzakhan, A., & Shen, Z. (2023). Principled Instructions Are All You Need for Questioning LLaMA-1/2, GPT-3.5/4. MBZUAI, arXiv:2312.16171

# 논문으로 검증된 26가지 원칙 ② — 상호작용 · 언어 스타일 · 코딩

★ 효과가 특히 컸던 원칙 · △ 효과 불안정 (5.5 참고)

### ③ 사용자 상호작용

| # | 원칙 | 예시 |
|---|---|---|
| 14 | 모델이 먼저 질문하게 한다 ★ | "충분한 정보가 모일 때까지 질문해" |
| 11 | 자연스러운 사람 말투를 요청한다 | "자연스럽고 사람처럼 답해" |
| 22 | 문체는 유지하고 문법 · 어휘만 고친다 | "격식체는 유지, 문법만 교정" |

### ⑤ 복잡한 작업과 코딩

| # | 원칙 | 예시 |
|---|---|---|
| 19 | 단계별 사고(CoT)와 예시를 결합한다 | 추론 과정이 담긴 예시 |
| 23 | 여러 파일 코드는 생성 스크립트로 받는다 | Codex 는 파일을 직접 편집한다 |

### ④ 내용과 언어 스타일

| # | 원칙 | 예시 |
|---|---|---|
| 1 | 간결한 답을 원하면 인사말을 생략한다 | 바로 본론 |
| 3 | 복잡한 작업은 대화로 쪼갠다 | 조사 → 개요 → 초안 → 수정 |
| 6 | "팁 $xxx 를 주겠다" △ | 효과 불안정 — 5.5 |
| 9 | "Your task is", "You MUST" △ | "너의 임무는 …이다" |
| 10 | "틀리면 불이익을 받는다" △ | 효과 불안정 — 5.5 |
| 16 | 역할을 부여한다 ★ | "너는 시니어 보안 리뷰어다" |
| 18 | 핵심 단어 · 구절을 반복한다 | 중요한 조건을 두 번 언급 |

출처 · Bsharat 외(2023) arXiv:2312.16171

강의 · 개념 이해 5장 · 5.5 | 2026년 기준

# 2026년 기준으로 본 26가지 원칙 — 지금도 유효한 것과 약해진 것

논문은 2023년 말 GPT-3.5/4 · LLaMA 실험 결과다 — 최신 모델에서는 원칙마다 유효성이 다르다

### ✓ 지금도 확실히 유효한 원칙

- 2 청중 명시 · 3 작업 분해 · 7 · 19 예시와 CoT
- 8 · 17 구조화 · 구분자 · 14 모델이 먼저 질문
- 16 역할 부여 · 25 요구사항 명시 · 26 문체 샘플
- → 현재 OpenAI · Anthropic 공식 가이드와 일치

### △ 효과가 약하거나 불안정한 원칙

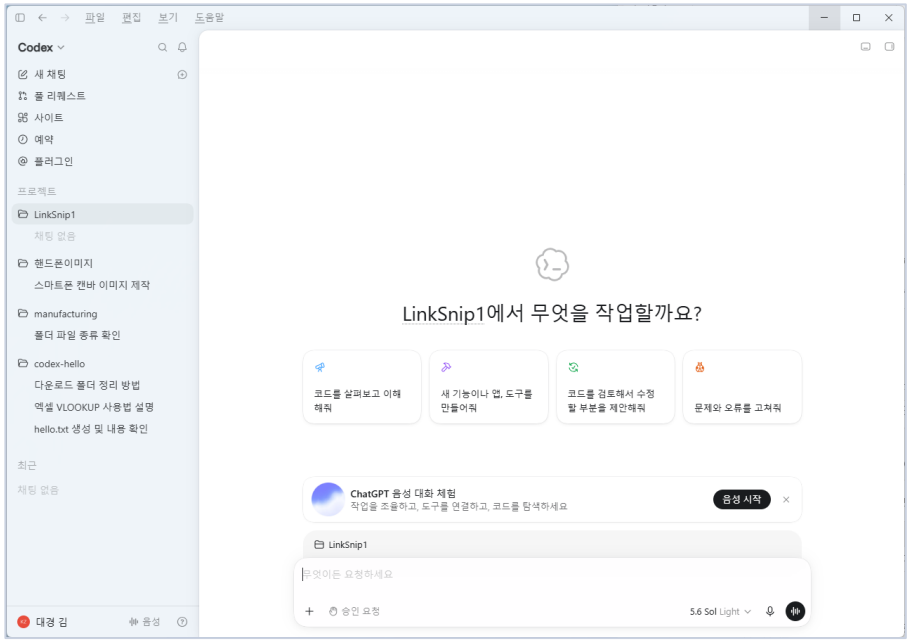
- 6 · 10 팁 주기 · 불이익 경고 — 후속 연구에서 효과가 일관되지 않았다
- 9 "You MUST" 같은 강한 표현 — 최신 모델에서 과잉 반응 → 이유를 설명하는 차분한 지시를 권장
- 12 "단계별로 생각해" — 추론 모델은 이미 내부에서 단계적으로 사고한다

**Codex 에서 쓰는 법 — 5.3 P · C · T · F 에 원칙을 대응시키기**

| P 역할 · 관점 | C 배경 · 제약 | T 할 일 · 검증 | F 산출물 형태 |
|---|---|---|---|
| 16 역할 부여 | 2 청중 · 25 요구사항 · 26 문체 | 3 작업 분해 · 14 먼저 질문 · 19 CoT | 8 · 17 구조 · 20 출력 유도 |

출처 · Bsharat 외(2023) · OpenAI「Prompt engineering」가이드(platform.openai.com) · Anthropic「Prompt engineering overview」(docs.claude.com)

# 실습 — 큰 작업은 '먼저 계획부터' (linksnip1)

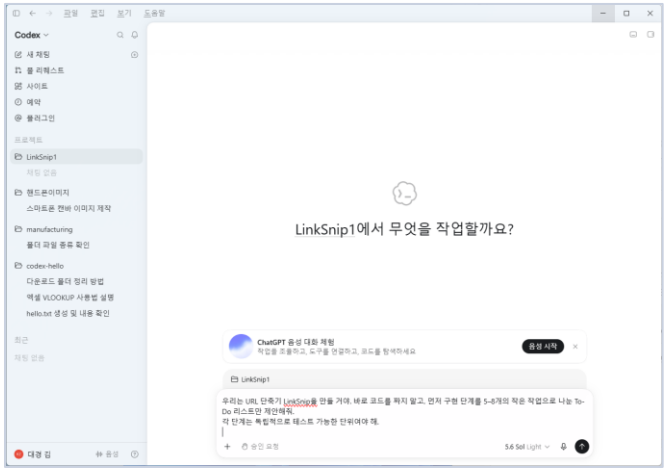



① linksnip1 프로젝트를 연 상태

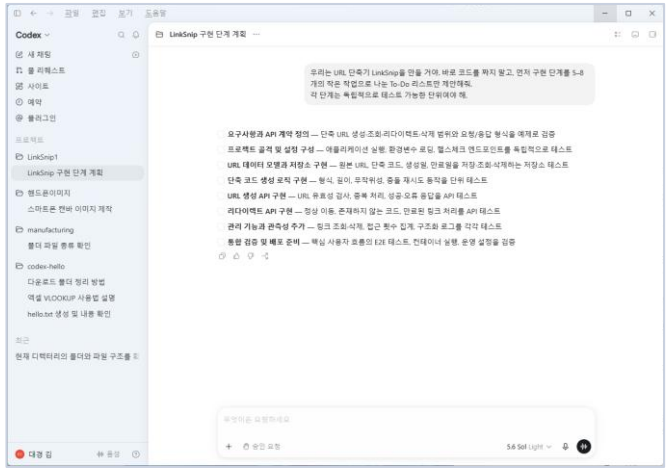
**▶ 프롬프트 — 계획 먼저**

우리는 URL 단축기 linksnip을 만들 거야. 바로 코드를 짜지 말고, 먼저 구현 단계를 5–8개의 작은 작업으로 나눈 To-Do 리스트만 제안해줘.
각 단계는 독립적으로 테스트 가능한 단위여야 해.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② 작성 칸에 입력




③ 결과 — 8단계 To-Do (API 계약 정의 → 배포 검증)

# 실습 — 계획 중 첫 단계만 골라 다시 시킨다

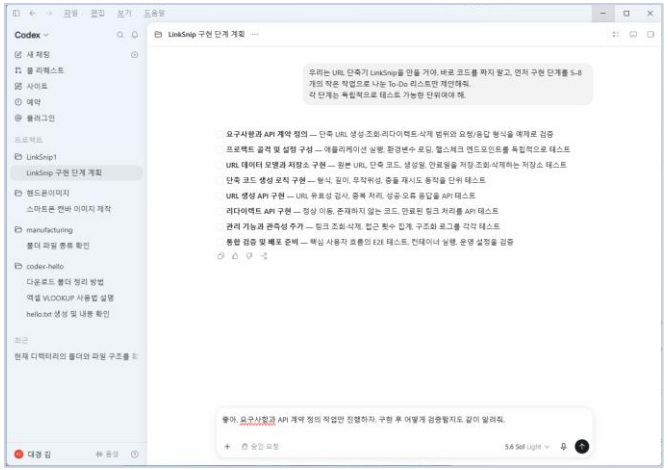
**▶ 프롬프트 — 한 단계씩**

좋아. 1번 작업만 진행하자. 구현 후 어떻게 검증할지도 같이 알려줘.

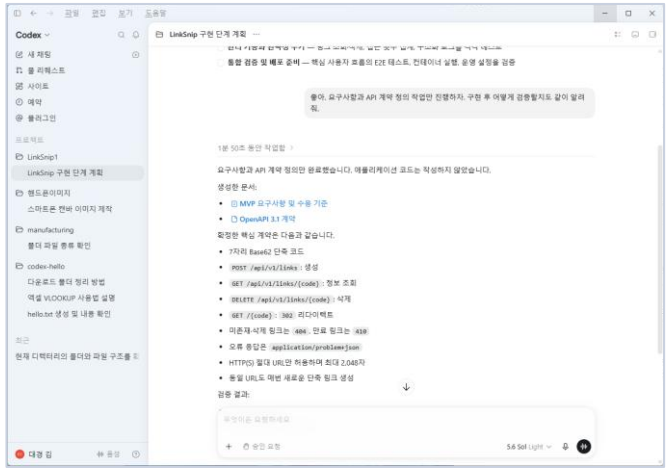
## 왜 한 단계씩?

- 한 번에 다 시키면 검토할 변경이 너무 커진다
- 단계마다 '검증 방법'을 받아 두면 다음 단계의 기준이 된다
- 원하는 방향이 아니면 그 단계에서 바로 고친다 (반복 개선)

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



① 후속 지시 입력



② 결과 — 요구사항 · API 계약 정의와 검증 결과

# 유형별 프롬프트 기법 8가지 — 하나씩 보내 보고 비교한다

P · C · T · F 가 '뼈대'라면, 아래는 상황에 맞게 골라 쓰는 '기법'이다 — 예제는 linksnip 기준

### 1 역할 부여
Persona

전문가 관점을 지정해 판단 기준을 좁힌다

### 2 예시 제공
Few-shot

입력 → 출력 쌍 1~2개로 형식 · 규칙 고정

### 3 계획 먼저
Chain-of-Thought

바로 코드 말고 계획부터 — 큰 작업의 실수를 줄인다

### 4 제약 · 금지
Constraints

하지 말아야 할 것을 적어 원치 않는 변경을 막는다

### 5 가드레일
Guardrails

모르면 '확인 필요' — 추측 금지로 환각을 줄인다

### 6 검증 내장
Test-first

통과해야 할 테스트 · 명령을 넣어 스스로 고치게

### 7 출력 형식
Output Format

보고 형태를 지정해 검토 속도를 높인다

### 8 셀프 리뷰
Self-critique

스스로 결과를 비판해 놓친 위험을 잡는다

**TIP** 기법은 하나만 쓰지 않는다 — 실제 프롬프트는 '역할(1) + 제약(4) + 검증(6) + 형식(7)'처럼 겹쳐 쓴다. P · C · T · F 가 그 조합의 최소 골격이다.

# ① 역할 부여 · ② 예시 제공(Few-shot)

## ① 역할 부여 (Persona)

▶ 프롬프트 — Codex App

보안을 중시하는 시니어 백엔드 개발자로서 검토해줘.
src/lib/shorten.ts 의 makeCode 를 보고, 코드 충돌 · 예측 가능성 관점의 문제를
심각도(높음 · 중간 · 낮음)와 함께 지적해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 검토 결과 (src/lib/shorten.ts)
  높음  Math.random() 기반 코드 — 충돌 가능
  높음  6자리 base36 → 추측 · 열거 공격 쉬움
  중간  충돌 시 재시도 로직 없음
  낮음  코드 길이 상수화 필요
```

▶ 실습 같은 코드를 '초보자에게 설명하는 선생님' 역할로 다시 요청 → 답변의 초점과 말투 비교

## ② 예시 제공 (Few-shot)

▶ 프롬프트 — Codex App

makeCode 의 동작을 아래 예시처럼 맞춰줘.
입력: "https://a.com" → 출력: 6자리 코드(예: k3d9xq)
입력: "" → 예외: Error("EMPTY_URL")
이 규칙에 맞는 구현과 테스트를 작성해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ makeCode 를 예시 규칙에 맞췄는다.
  · 정상 URL → [a-z0-9]{6}
  · 빈 문자열 → Error("EMPTY_URL")
  tests/shorten.test.ts
  ✓ returns 6-char code
  ✓ throws EMPTY_URL on ""
```

▶ 실습 예시를 1개만 줬을 때와 3개(정상 · 빈값 · 중복 URL)를 줬을 때 결과의 정확도 비교

# ③ 계획 먼저(Chain-of-Thought) · ④ 제약 · 금지 명시

## ③ 계획 먼저 (Chain-of-Thought)

▶ 프롬프트 — Codex App

linksnip 에 '최근 단축한 링크 5개 보기' 기능을 추가할 거야.
코드를 짜기 전에, 구현을 4~6단계로 나눈 계획과 각 단계의 검증 방법을 먼저 표로 제안해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ 단계 작업 검증

| | | |
|---|---|---|
| 1 | 최근 목록 저장 구조 정의 | 단위 테스트 |
| 2 | 저장 시 최신 5개 유지 | 경계값 테스트 |
| 3 | 조회 API GET /recent | API 테스트 |
| 4 | 화면에 목록 표시 | 브라우저 확인 |
| 5 | 빈 목록 안내 문구 | 스냅숏 확인 |

▶ 실습 계획을 받은 뒤 '2번 단계만 구현해줘'로 이어서 요청 — 한 번에 한 단계씩

## ④ 제약 · 금지 명시 (Constraints)

▶ 프롬프트 — Codex App

src/lib/shorten.ts 만 수정해줘.
제약 — 새 npm 패키지 추가 금지, 공개 함수 시그니처 변경 금지, 다른 파일은 건드리지 말 것.
중복 코드만 정리해 가독성을 높여줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ src/lib/shorten.ts 만 수정했다. (+12 -19)
· 검증 로직을 assertUrl() 로 추출
· 공개 함수 makeCode 시그니처 그대로
· 새 패키지 없음, 다른 파일 변경 없음

▶ 실습 제약을 뺀 버전과 넣은 버전을 각각 실행 → 변경된 파일 수 비교

# ⑤ 가드레일 — 추측 금지 · ⑥ 검증 내장(Test-first)

## ⑤ 가드레일 — 추측 금지 (Guardrails)

**▶ 프롬프트 — Codex App**

이 저장소의 배포 방식을 설명해줘.
단, 확인되지 않은 내용은 추측하지 말고 '확인 필요'라고 표시해줘.
근거가 된 파일 경로도 함께 알려줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ 배포 방식
- 빌드: npm run build → dist/ (근거: package.json)
- 호스팅: 확인 필요 — 배포 설정 파일을 찾지 못함
- CI: 확인 필요 — .github/workflows 없음

▶ 실습 존재하지 않는 파일에 대해 일부러 물어보고, '없음 / 확인 필요'로 답하는지 확인

## ⑥ 검증 내장 (Test-first)

**▶ 프롬프트 — Codex App**

makeCode 에 '같은 URL 은 항상 같은 코드' 규칙을 보장해줘.
tests/shorten.test.ts 에 이를 확인하는 테스트를 추가하고,
npm test 를 실행해 전부 통과하는 로그를 마지막에 보여줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

✦ 테스트 추가 후 실행
FAIL same URL returns same code
→ makeCode 를 해시 기반으로 변경
> npm test
✓ tests/shorten.test.ts (3 tests)
Test Files 1 passed · Tests 3 passed

▶ 실습 일부러 규칙을 깬 코드로 시작해, 테스트가 빨강 → 초록으로 바뀌는 과정 관찰

# ⑦ 출력 형식 지정 · ⑧ 셀프 리뷰 · 비평

## ⑦ 출력 형식 지정 (Output Format)

**▶ 프롬프트 — Codex App**

위 변경을 마친 뒤, 결과를 다음 형식으로만 보고해줘.
1) 변경한 파일 목록  2) 핵심 변경점 3줄  3) 실행한 명령과 통과 여부.
코드 전체를 다시 붙여넣지는 마.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 1) 변경 파일: src/lib/shorten.ts, tests/shorten.test.ts
  2) · 해시 기반 코드 생성
     · 빈 URL 예외 유지
     · 같은 URL → 같은 코드 테스트 추가
  3) npm test — 통과 (3 passed)
```

▶ 실습  같은 작업을 'diff 형태로', '표 형태로' 각각 요청 — 어느 쪽이 검토하기 편한지 비교

## ⑧ 셀프 리뷰 · 비평 (Self-critique)

**▶ 프롬프트 — Codex App**

방금 만든 변경분을 성능 · 보안 · 경계값 관점에서 셀프 리뷰하고,
발견한 위험을 '항목 / 심각도(높음 · 중간 · 낮음) / 개선안' 표로 정리해줘.
그다음 심각도 높음 항목부터 고쳐줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 항목                  심각도  개선안
  해시 충돌 미처리        높음   충돌 시 접미사 추가
  URL 길이 제한 없음      중간   2,048자 제한
  대소문자 정규화 없음    낮음   소문자 통일
  → '높음' 1건 수정 완료, npm test 통과
```

▶ 실습  셀프 리뷰 표에서 '중간' 이상 항목만 골라 수정하도록 이어서 요청

# Goal — 여러 턴에 걸쳐 하나의 목표를 끝까지 추적한다

Codex 의 Goal 은 '한 번의 응답'이 아니라, 검증 가능한 목표를 계속 추적하고 수행하게 하는 장기 작업 기능이다

| 구분 | 뜻 |
| --- | --- |
| Goal | 장기적으로 유지되는 최종 목적 |
| Plan | Goal 을 달성하기 위한 현재 작업 단계 |
| 일반 프롬프트 | 한 턴에서 처리할 요청 |
| Automation(예약) | 특정 시각이나 주기에 실행할 작업 (4.3) |

### 쓰는 법

- App · CLI: /goal 로 목표를 설정 · 수정 · 일시 정지 · 재개 · 확인 · 해제
- 목표 문장에는 '완료 조건'을 번호로 적는다
- config.toml 의 [features] goals = true (부록 B)

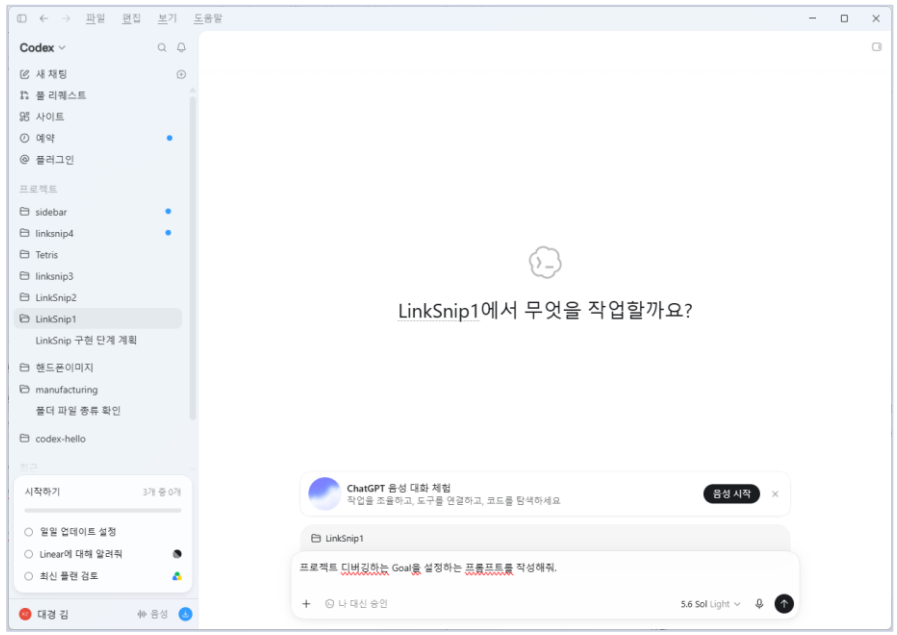
**핵심** 좋은 Goal = 측정 가능한 완료 조건 + 진행 원칙 + 막혔을 때 보고 방식. 다음 실습에서는 Codex 에게 'Goal 을 설정하는 프롬프트'부터 쓰게 한다.

**참고** Goal 과 예약은 다르다 — Goal 은 '무엇을 끝낼지', 예약은 '언제 실행할지'를 정한다. 둘을 함께 쓸 수도 있다.

# Goal 실습 ① — Goal 을 설정하는 프롬프트를 Codex 에게 쓰게 한다

**▶ 프롬프트 — Codex App**

프로젝트 디버깅하는 Goal을 설정하는 프롬프트를 작성해줘.




작성 칸에 입력

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답 — 작성된 Goal 프롬프트

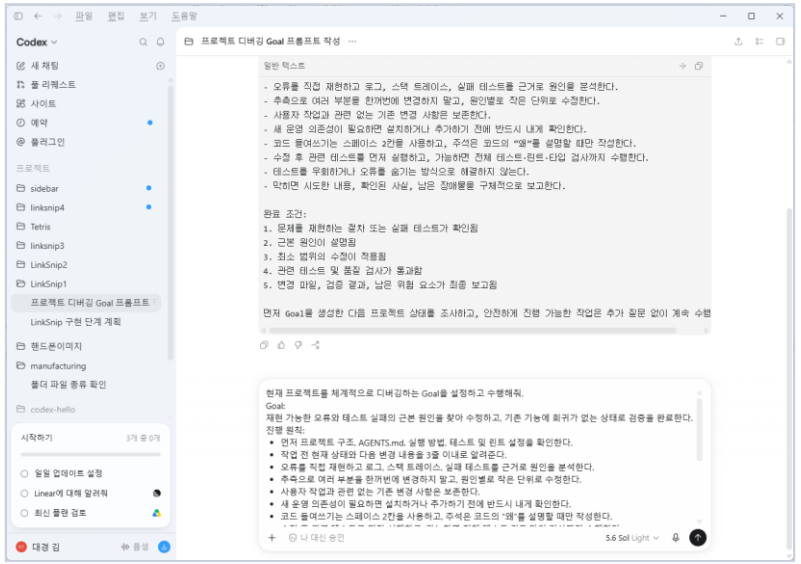
```
현재 프로젝트를 체계적으로 디버깅하는 Goal을 설정하고 수행해줘.
Goal: 재현 가능한 오류와 테스트 실패의 근본 원인을 찾아 수정하고,
      기존 기능에 회귀가 없는 상태로 검증을 완료한다.
진행 원칙:
- 먼저 프로젝트 구조, AGENTS.md, 실행 방법, 테스트 및 린트 설정을 확인한다.
- 작업 전 현재 상태와 다음 변경 내용을 3줄 이내로 알려준다.
- 오류를 직접 재현하고 로그 · 스택 트레이스 · 실패 테스트를 근거로 분석한다.
- 추측으로 한꺼번에 변경하지 말고, 원인별로 작은 단위로 수정한다.
- 사용자 작업과 관련 없는 기존 변경 사항은 보존한다.
- 새 운영 의존성은 추가 전에 반드시 내게 확인한다.
- 들여쓰기는 스페이스 2칸, 주석은 '왜'를 설명할 때만.
- 수정 후 관련 테스트 → 전체 테스트 · 린트 · 타입 검사까지 수행한다.
- 테스트를 우회하거나 오류를 숨기는 방식으로 해결하지 않는다.
- 막히면 시도한 내용 · 확인된 사실 · 남은 장애물을 보고한다.
완료 조건: 1. 재현 절차 또는 실패 테스트 확인  2. 근본 원인 설명
  3. 최소 범위 수정  4. 테스트 · 품질 검사 통과  5. 변경 · 검증 · 남은 위험 보고
먼저 Goal을 생성한 다음 프로젝트 상태를 조사하고, 안전하게 진행 가능한 작업은 추가 질문 없이 계속 수행해줘.
```

**지금 할 일** linksnip1 에서 아래 프롬프트 ▸ 돌아온 결과(오른쪽)를 다음 슬라이드에서 그대로 다시 붙여 넣는다

# Goal 실습 ② — 받은 Goal 프롬프트를 실행하고 진행을 지켜본다

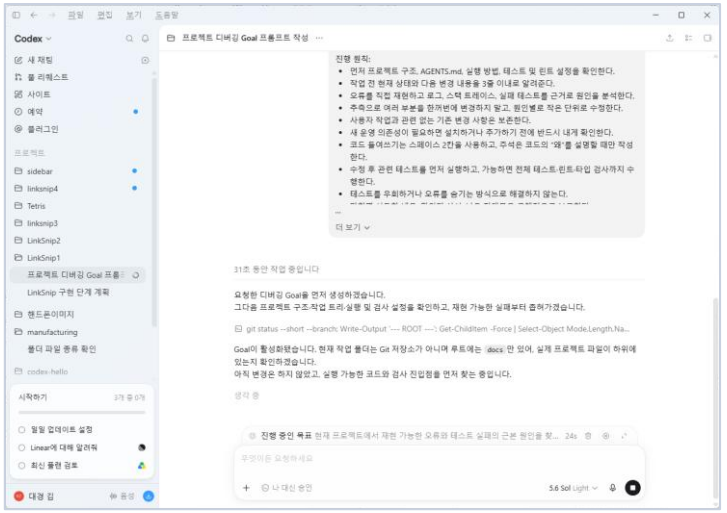
**▶ 프롬프트 — 앞 결과 그대로**

현재 프로젝트를 체계적으로 디버깅하는 Goal을 설정하고 수행해줘.
Goal: …
진행 원칙: …
완료 조건: …
(앞 슬라이드 결과 전체를 붙여 넣는다)

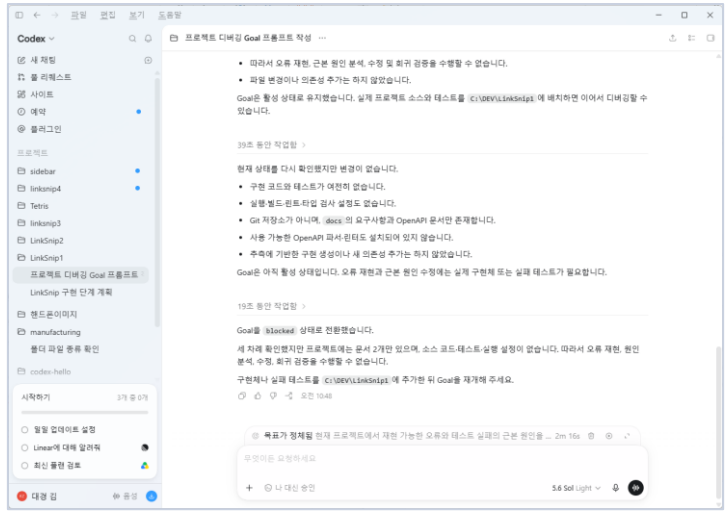


① 붙여 넣은 Goal 프롬프트

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



② Goal 생성 · 상태 조사 시작



③ 보고 — 확인된 사실 · 남은 장애물 (blocked 시 이유)

# 자주 쓰는 프롬프트 패턴 모음

상황별로 바로 꺼내 쓰는 한 줄 — 모두 '검증'을 포함한다

| 상황 | 프롬프트 예시 |
| --- | --- |
| **코드 이해** | transform 모듈이 어떻게 동작하고 어떤 모듈이 이를 사용하는지 설명해줘. |
| **버그 수정** | 다음 에러를 재현하는 방법은 ...야. 원인을 찾아 고치고, 회귀 테스트를 추가해줘. |
| **리팩터링** | src/utils.ts 의 동작은 그대로 두고, 중복을 제거해 가독성만 개선해줘. 테스트가 계속 통과해야 해. |
| **기능 추가** | --json 출력 옵션을 추가해줘. 사용법을 README 에도 한 줄 추가하고 예시를 보여줘. |
| **리뷰 요청** | 방금 변경분을 보안 · 성능 관점에서 셀프 리뷰하고 위험 요소를 표로 정리해줘. |

**TIP** '검증 가능하게'가 가장 중요한 한 가지 — 재현 절차, 통과해야 할 테스트, 실행해 볼 명령을 프롬프트에 넣는다. Codex 가 결과를 스스로 돌려 보며 고치기 때문에 정확도가 올라간다.

# 연습 — 나쁜 한 줄을 좋은 프롬프트로 고쳐 쓴다

세 상황에 P · C · T · F 와 핵심 기법을 적용한다 — 실제 실행은 9장 · 10장 Tetris 에서 한다

## ① 계획부터 — PRD 로 To-Do 뽑기

**✕ 나쁜 예**

테트리스 만들어줘

**✓ 좋은 예**

`@PRD.md` 를 읽고 — 수정하지 말고(읽기 전용) — Tetris MVP 완성을 위한 To-Do 를 체크리스트로 먼저 만들어줘.

→ 9.2

## ② 구현 + 검증 — 한 기능을 검증까지

**✕ 나쁜 예**

점수 기능 만들어줘

**✓ 좋은 예**

꼼꼼한 프론트엔드 엔지니어로서 `@script.js` 에 줄이 삭제될 때 점수가 오르는 로직을 구현해줘. 한 줄=100점, 여러 줄 동시 삭제 시 보너스. 완료 후 브라우저에서 점수가 정상적으로 오르는지 확인하는 방법을 알려줘.

→ 10.7

## ③ 디버깅 — 증상을 구체적으로

**✕ 나쁜 예**

게임이 안 돼요, 고쳐줘

**✓ 좋은 예**

블록이 좌우 벽을 통과해요. 벽 경계에서 멈추도록 충돌 검사를 고치고, 고친 뒤 브라우저에서 좌우 이동이 정상인지 확인하는 방법을 알려줘.

→ 10.8

06

CHAPTER 6 · 강의 + 실습

# 컨텍스트 엔지니어링 — 무엇을 알고 일하나

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 알고 일하나'다. Codex 가 참고하는 정보(파일 · 이미지 · 규칙 · 이전 대화)를 '필요한 만큼만, 정확히' 보여 주는 기술이다.

6.1–6.4 정의 · 종류 · 전략 | 6.5 @파일 콕 집기 | 6.6–6.7 컴팩션 · 3원칙

# 컨텍스트의 정의 — 모델이 한 번에 보는 정보 전체

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 보고 일하나'다

## Anthropic · 기술적 정의

「Effective context engineering for AI agents」 2025.9

- 컨텍스트 = LLM 이 응답을 생성할 때 포함되는 토큰의 집합
- 컨텍스트 엔지니어링 = 추론 중 최적의 토큰 집합을 선별 · 유지하는 전략의 총체
- 핵심: 결과 확률을 최대화하는 가장 작은 고신호 토큰 집합

## 안드레이 카파시 · 작업 메모리

X 게시물 2025.6

- "다음 단계에 딱 맞는 정보로 컨텍스트 창을 채우는 섬세한 예술이자 과학"
- 비유: 모델 = CPU, 컨텍스트 창 = RAM
- → 제한된 메모리에 무엇을 올릴지 정하는 일

## 토비 뤼트케 · 필요한 맥락 전부

Shopify CEO · X 게시물 2025.6

- "LLM 이 과제를 그럴듯하게 풀 수 있도록 필요한 맥락을 모두 제공하는 기술"
- 카파시 발언의 계기가 된 표현

출처 · Anthropic「Effective context engineering for AI agents」(2025.9.29) · A. Karpathy, X(2025.6.25) · T. Lütke, X(2025.6) · DeepLearning.AI「Context Hub」(github.com/andrewyng/context-hub, 2026.3)

# LLM 은 컴퓨터, 컨텍스트 윈도우는 작업 기억(RAM)

프롬프트가 '사용자가 지금 하는 한마디'라면, 컨텍스트는 '모델이 보고 있는 그 밖의 모든 것' — 규칙 · 예시 · 이전 대화 · 도구 결과

| 모델 = CPU | 컨텍스트 윈도우 = RAM | 무엇을 얼마나 넣을지 관리 = 컨텍스트 엔지니어링 |
|---|---|---|

### 정보가 부족하면

- 엉뚱한 답
- 모르는 부분을 추측해서 채운다

### 정보가 너무 많으면

- 비싸고 느리다
- 오히려 정확도가 떨어진다 — 입력이 길수록 성능이 흐트러지는 '컨텍스트 로트'

### 정확히, 관련 있는 것만

- 지금 작업에 필요한 파일만 @로
- 반복 규칙은 AGENTS.md 로
- 길어지면 요약 · 새 스레드

**주의** 흔한 오해 — '많이 넣을수록 똑똑해진다'는 틀린 말이다. 카파시는 LLM 을 새로운 종류의 운영체제에 비유한다: 한 번에 담을 수 있는 정보량이 정해져 있다.

# 컨텍스트의 종류 — 무엇을, 어떻게 주나

| 종류 | 설명 | 주는 방법 |
|---|---|---|
| **파일 내용** | 관련 소스 / 문서 파일 | @파일명 으로 참조하거나 경로 명시 |
| **이미지** | 디자인 시안 · 에러 스크린숏 | 이미지 파일 경로 / 붙여넣기 |
| **대화 기록** | 같은 스레드의 이전 지시 · 결과 | 스레드를 이어서 진행 |
| **AGENTS.md** | 프로젝트 상시 규칙 | 파일로 저장 (작업 전 자동 로드 · 7.1) |
| **도구 출력** | 테스트 · 빌드 로그 | Codex 가 실행해 자동 수집 |

**참고** App · IDE 는 컨텍스트를 자동으로 돕는다 — App 은 열어 둔 작업 폴더의 파일을, VS Code 확장은 '열린 파일 목록'과 '선택한 코드 영역'을 자동으로 넣는다. 그래도 @로 콕 집어 주면 결과가 더 좋다.

# 컨텍스트를 다루는 네 가지 전략 — 작성 · 선택 · 압축 · 격리

한정된 작업 기억을 네 가지 방식으로 '관리'한다 — 이 과정에서 그대로 실습한다

## 작성

Write

중요한 규칙 · 결정을 밖에 적어 두기

→ AGENTS.md 에 규칙 저장 (7장)

## 선택

Select

지금 작업에 필요한 것만 골라 넣기

→ @로 파일 콕 집어 주기 (6.5)

## 압축

Compress

길어진 대화를 요약해 자리 확보

→ 컴팩션 · 스레드 나누기 (6.6)

## 격리

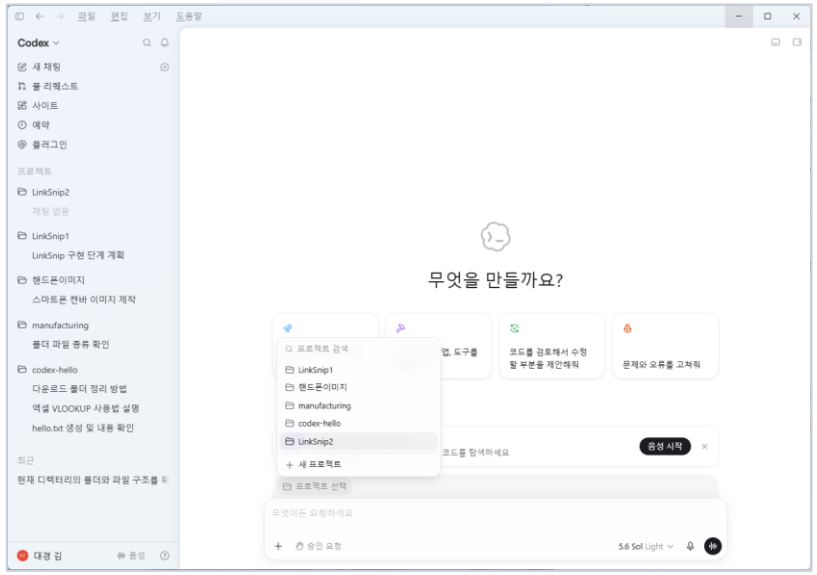
Isolate

작업별로 맥락을 분리하기

→ 한 작업 = 한 스레드 (3.3)

**핵심** 핵심 비유 — LLM 은 컴퓨터, 컨텍스트 윈도우는 작업 기억(RAM). '많이 넣을수록 똑똑'은 오해 — 정확히, 관련 있는 것만.

# 선택 전략 실습 ① — 예제 파일부터 만든다 (linksnip2)



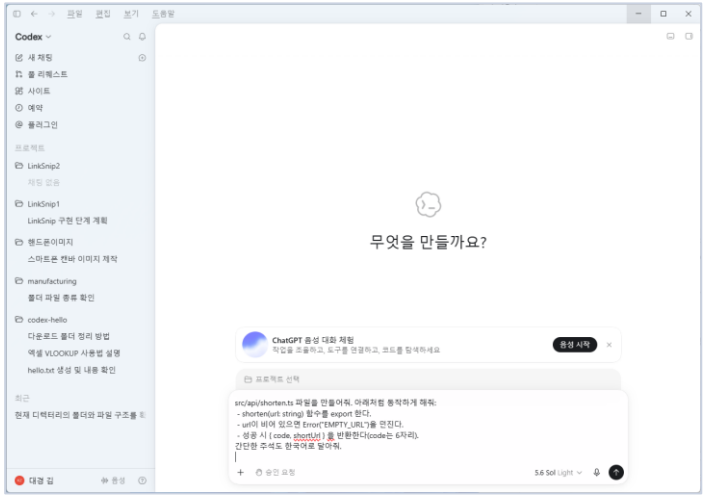

① 새 프로젝트 만들기 — linksnip2

**▶ 프롬프트 — 준비**

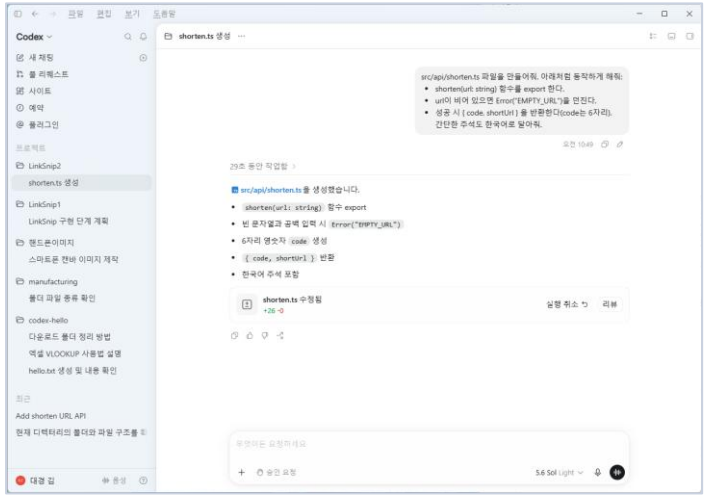
src/api/shorten.ts 파일을 만들어줘. 아래처럼 동작하게 해줘:
- shorten(url: string) 함수를 export 한다.
- url이 비어 있으면 Error("EMPTY_URL")을 던진다.
- 성공 시 { code, shortUrl } 을 반환한다(code는 6자리).

간단한 주석도 한국어로 달아줘.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② 작성 칸에 입력




③ 결과 — src/api/shorten.ts 생성 (+26)

# 선택 전략 실습 ② — 만들어진 파일 확인




tree /f — src/api/shorten.ts

**파일: src/api/shorten.ts (예시 — 개념 확인용)**

```
// src/api/shorten.ts — URL을 6자리 코드로 줄이는 최소 구현
export interface ShortenResult {
  code: string;
  shortUrl: string;
}

export function shorten(url: string): ShortenResult {
  if (!url) throw new Error("EMPTY_URL");   // 빈 입력 방어
  const code = Math.random().toString(36).slice(2, 8); // 6자리
  return { code, shortUrl: `https://snip.example/${code}` };
}
```

**참고** Codex 가 만든 코드는 이 예시와 모양이 조금 다를 수 있다 — 함수 이름 · 예외 · 반환 형태 세 가지가 맞으면 된다.

**TIP** 다음 단계 — 이 파일을 @로 콕 집어 '응답 형식 바꾸기'를 시킨다. 작성 칸에서 @ 를 입력하면 작업 폴더 파일을 빠르게 검색해 넣을 수 있다.

# 선택 전략 실습 ③ — @파일로 수정을 시킨다

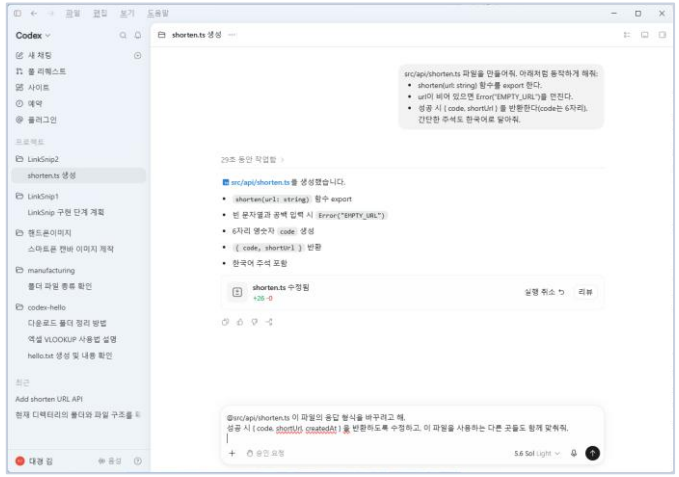
**▶ 프롬프트 — @파일 참조**

`@src/api/shorten.ts` 이 파일의 응답 형식을 바꾸려고 해.
성공 시 { code, shortUrl, createdAt } 을 반환하도록 수정하고, 이 파일을 사용하는 다른 곳들도 함께 맞춰줘.

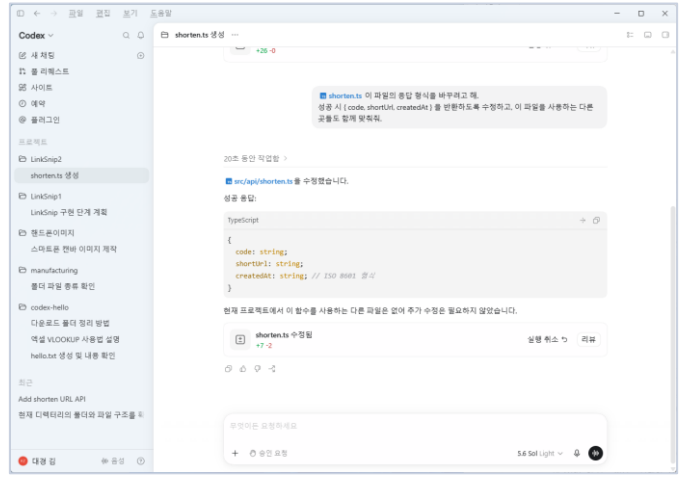
## 포인트

- @ 로 대상 파일을 정확히 지정 — '이 파일'이 어느 파일인지 추측할 필요가 없다
- '사용하는 다른 곳들도 함께' — 영향 범위까지 맡긴다
- 결과의 '다른 파일 변경 없음' 보고도 확인한다

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



① @src/api/shorten.ts 로 입력



② 결과 — createdAt: string (ISO 8601) 추가

지금 할 일 작성 칸에 @ 입력 → src/api/shorten.ts 선택 ▸ 아래 프롬프트 전송 ▸ createdAt 이 추가됐는지 확인

# 컨텍스트 윈도우와 컴팩션(compaction) — 압축 전략

한 스레드의 모든 정보는 컨텍스트 윈도우(작업 기억 한도) 안에 들어가야 한다 — 한도에 가까워지면 Codex 가 자동으로 압축한다

**오래 이어진 작업 기록**

```
1. 요구사항 분석
2. 프로젝트 구조 확인
3. DB 설계
4. API 작성
5. 테스트 실행
6. 오류 수정
7. 리팩터링
8. 배포 설정
...
```

→

**압축된 핵심 상태**

```
[Compacted context]
Project: linksnip
Goal: URL shortening service
Completed:
- Next.js project created
- PostgreSQL configured
- URL API implemented
- Tests passing
Important decisions:
- Use PostgreSQL · Prisma
- Short code length = 7
Current task:
- Implement deployment
```

### 컴팩션이란

- 오래된 컨텍스트를 요약 · 압축해
- 중요한 정보는 최대한 유지하고
- 다음 단계로 넘길 양을 줄이는 과정
- CLI 에서는 /compact 로 직접 요약할 수도 있다

### 그래도 도움이 되는 습관

- 긴 작업은 여러 개의 작은 작업으로 나눠 새 스레드에서 진행한다
- 관련 없는 파일을 잔뜩 붙여 넣지 않는다 (필요한 파일만)
- 핵심 규칙은 대화가 아니라 AGENTS.md 에 적어 둔다 (7장)

# 좋은 컨텍스트를 만드는 3원칙 — 적게, 정확히

## 1 관련성

지금 작업에 필요한 파일 / 정보만 보여준다.

예

**@src/lib/shorten.ts**
**@tests/shorten.test.ts** 이 두 파일만 보고 분석해줘.

## 2 구체성

'이 함수'가 아니라 파일 · 함수 · 줄 범위를 지정한다.

예

src/api/shorten.ts 의 shorten() 12–18행만 고쳐줘.

## 3 지속성

매번 반복되는 규칙은 대화 대신 AGENTS.md 에 적는다.

예

AGENTS.md: 변경 후 반드시 npm test 를 통과시킬 것.

TIP '많이 넣을수록 똑똑해진다'는 오해다. 불필요한 컨텍스트는 비용 · 속도 · 정확도를 모두 해친다 — 핵심만 주고, 반복 규칙은 AGENTS.md 로 옮긴다.

07

CHAPTER 7 · 강의 + 실습

# Codex 핵심 기술 — 지시 · 권한 체계

AGENTS.md 는 프로젝트의 상시 규칙(컨텍스트), config.toml 은 기본 동작 설정, Sandbox / Approvals 는 무엇을 할 수 있고 언제 사람에게 물어볼지를 정한다. Skill 은 반복 작업 지침을 묶는다. 이것이 하네스(8장)의 토대다.

7.1 AGENTS.md | 7.2–7.4 AGENTS.md 실습 | 7.5 config.toml | 7.6 Sandbox · Approvals | 7.7–7.8 Skill

# AGENTS.md — Codex 가 작업 전에 먼저 읽는 규칙서

Codex 는 작업을 시작할 때 AGENTS.md 파일들을 먼저 읽어 일관된 기준으로 일한다 — 파일은 계층적으로 합쳐진다

### 1 전역 (Global)

`~/.codex/AGENTS.md`

내 모든 프로젝트에 공통 적용되는 개인 기본 규칙

### 2 프로젝트

`저장소 루트/AGENTS.md`

이 프로젝트의 셋업 · 검증 규칙

### 3 하위 폴더

`services/x/AGENTS.md`
`(또는 AGENTS.override.md)`

특정 폴더에만 다르게 적용할 규칙

**핵심** 우선순위: 전역 → 프로젝트 루트 → 현재 폴더 순으로 합쳐지며, 현재 폴더에 가까울수록 우선한다. 같은 폴더에 AGENTS.override.md 가 있으면 그것이 AGENTS.md 를 대체한다 (합쳐 읽는 기본 한도 32KiB).

**참고** 이 파일이 곧 '컨텍스트'이자 '하네스'다 — '변경 시 반드시 lint · test 통과' 같은 규칙을 적어 두면, 매번 말하지 않아도 Codex 가 스스로 검증 루프를 돌린다 (8.6).

참고: CLI 의 /init 은 현재 프로젝트에 AGENTS.md 뼈대를 만들어 준다.

# 실습 — 전역 AGENTS.md 만들기 (내 개인 공통 규칙)

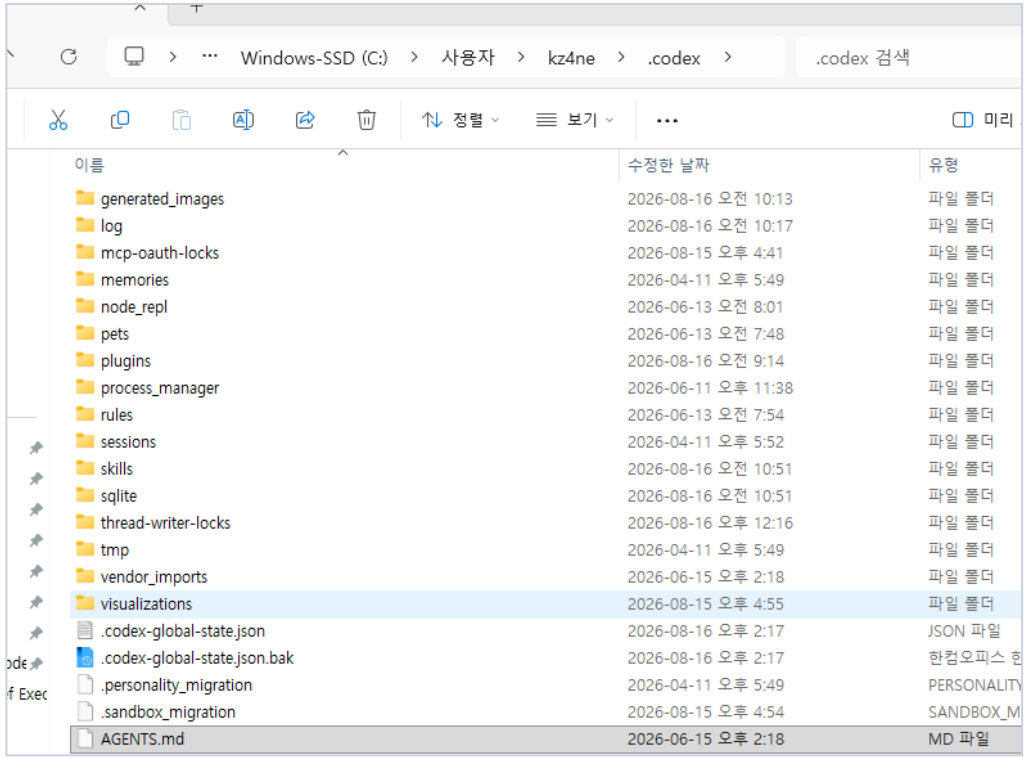
**▶ 프롬프트 — Codex App**

~/.codex/AGENTS.md 파일을 아래 내용으로 만들어줘.

```
# ~/.codex/AGENTS.md (내 개인 공통 규칙)
## 작업 방식
- 변경 전 한 일과 변경 요약을 3줄 이내로 알려 줄 것.
- 한 번에 너무 많이 바꾸지 말고 작은 단위로 진행할 것.
- 새 운영 의존성을 추가하기 전에는 먼저 물어볼 것.
## 코드 스타일
- 들여쓰기는 스페이스 2칸.
- 의미 없는 주석은 달지 말고, 왜(Why)를 설명하는 주석만 달 것.
```

편집기로 직접 만들어도 된다. Windows 에서 ~ 는 C:\Users\<사용자명> 이다 (예: C:\Users\kz4ne\.codex\AGENTS.md).

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




결과 — .codex 폴더 맨 아래 AGENTS.md

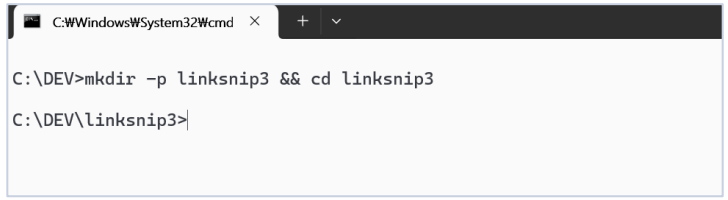
지금 할 일 Codex App 에 아래 프롬프트 ▸ 탐색기: C:\Users\<사용자>\.codex 에 AGENTS.md 가 생겼는지 확인

# 실습 — 프로젝트 AGENTS.md 만들기 (linksnip3)

폴더 생성은 터미널에서, 파일 작성은 Codex App 으로 맡긴다 — 이 파일은 8장에서 계속 보강한다

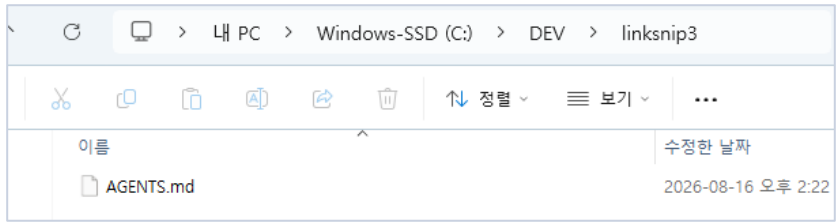
터미널

```
C:\DEV> mkdir linksnip3 && cd linksnip3
```




결과 — C:\DEV\linksnip3>

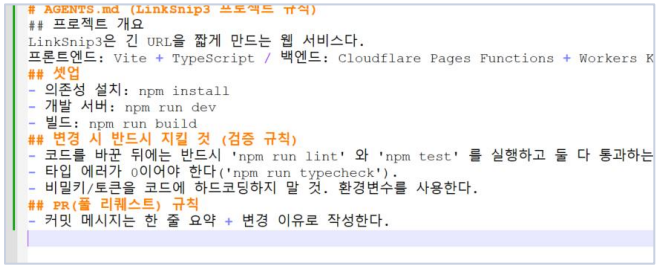
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




linksnip3 폴더에 AGENTS.md

**프롬프트 — Codex App**

AGENTS.md 파일을 아래 내용으로 만들어줘.
# AGENTS.md (linksnip3 프로젝트 규칙)
## 프로젝트 개요
linksnip3은 긴 URL을 짧게 만드는 웹 서비스다.
프론트엔드: Vite + TypeScript / 백엔드: Cloudflare Pages Functions + Workers KV.
## 셋업
- 의존성 설치: npm install - 개발 서버: npm run dev - 빌드: npm run build
## 변경 시 반드시 지킬 것 (검증 규칙)
- 코드를 바꾼 뒤에는 반드시 'npm run lint' 와 'npm test' 를 실행하고 둘 다 통과하는 것을 확인한 뒤 마무리할 것.
- 타입 에러가 0이어야 한다('npm run typecheck').
- 비밀키/토큰을 코드에 하드코딩하지 말 것. 환경변수를 사용한다.
## PR(풀 리퀘스트) 규칙
- 커밋 메시지는 한 줄 요약 + 변경 이유로 작성한다.

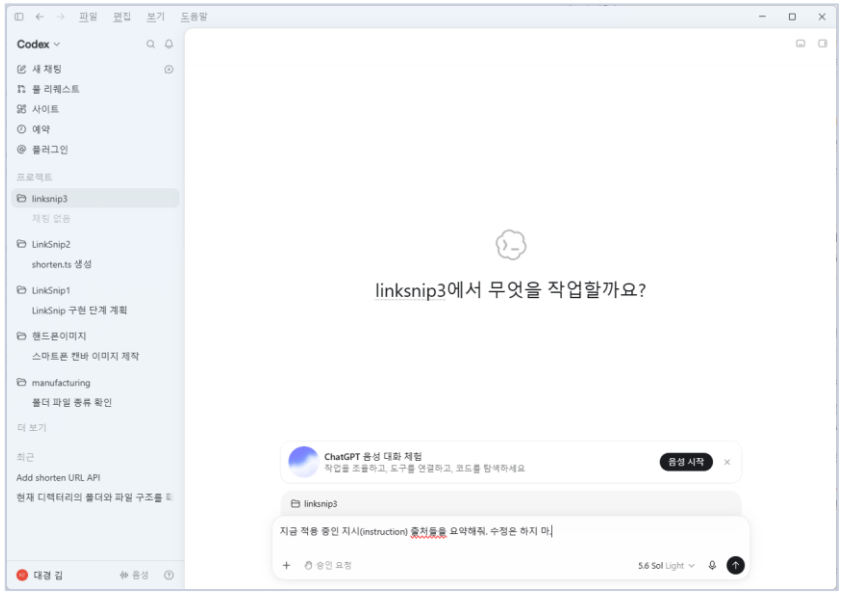



AGENTS.md 내용 — 개요 · 셋업 · 검증 규칙 · PR 규칙

# 실습 — AGENTS.md 가 잘 로드되는지 확인한다

**프롬프트 — Codex App**

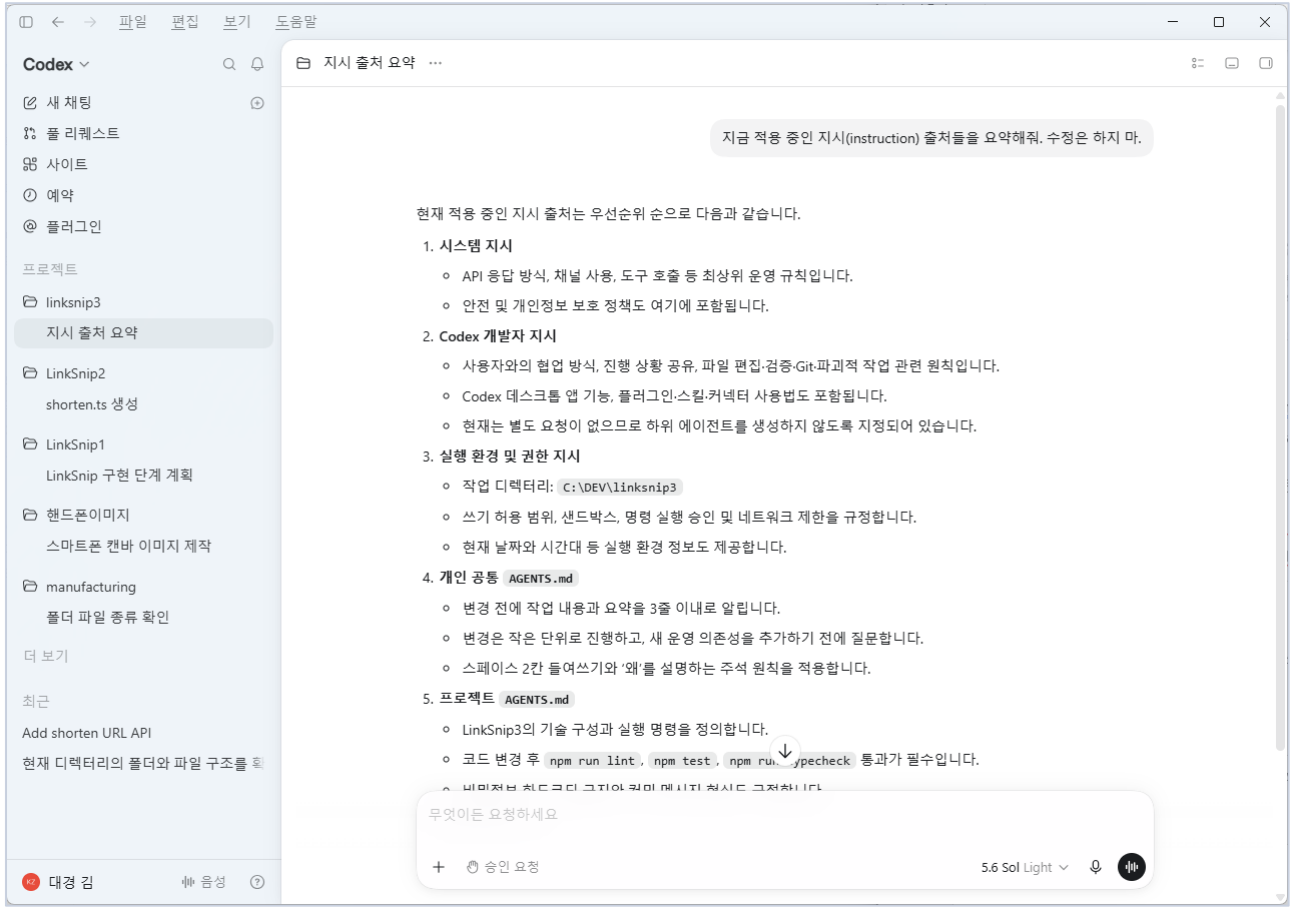
지금 적용 중인 지시(instruction) 출처들을 요약해 줘. 수정은 하지 마.




작성 칸에 입력

```
CLI: codex --ask-for-approval never "지금 적용 중인 지시 출처들을 요약해줘."
```

**실행 결과 (화면)** 기대 결과: 전역 → 프로젝트 루트 AGENTS.md 를 순서대로 인용




결과 — 시스템 · Codex 개발자 · 실행 환경 · 개인 공통(AGENTS.md) · 프로젝트(AGENTS.md)

**지금 할 일** linksnip3 를 연 상태에서 아래 프롬프트 ▸ 전역 · 프로젝트 AGENTS.md 가 순서대로 인용되는지 확인

# config.toml — Codex 기본 동작 설정

모델 · 승인 정책 · 샌드박스 등 기본값을 저장한다 — App · CLI · IDE 확장이 같은 설정을 공유한다

**파일 구조**

```
config.toml
│
├─ 전역 기본 설정        ← 여기
│   ├─ model
│   ├─ approval_policy
│   └─ sandbox_mode
│
├─ [sandbox_workspace_write]
│   └─ network_access
├─ notify
├─ [projects....]
├─ [windows]
├─ [plugins....]
├─ [features]
└─ [mcp_servers....]
```

| 범위 | 위치 |
|---|---|
| **사용자 전체** | ~/.codex/config.toml<br>(예: C:\Users\kz4ne\.codex\config.toml) |
| **특정 프로젝트** | <프로젝트>/.codex/config.toml<br>(예: C:\DEV\manufacturing\.codex\config.toml) |

**참고** 프로젝트 범위의 .codex/config.toml 은 사용자 설정을 덮어쓸 수 있다 — 팀이 함께 쓰는 프로젝트 규칙은 여기에 둔다.

### 세 가지 핵심 키

- model — 사용할 기본 모델 (예: gpt-5.6-sol)
- approval_policy — 언제 물어볼지: untrusted · on-request · never
- sandbox_mode — 접근 범위: read-only · workspace-write · danger-full-access

# config.toml 기본 설정 예시 — 주석으로 의미를 함께 적는다

```
# ~/.codex/config.toml – Codex 기본 설정 예시

# 사용할 기본 모델 (모델 선택 메뉴에서 확인)
model = "gpt-5.6-sol"

# 승인 정책: 언제 물어볼지
#   untrusted  = 신뢰된 명령 외에는 자주 승인 요청
#   on-request = 샌드박스 안은 자동, 벗어날 때 승인
#   never      = 승인 요청 없음 (격리 환경에서만)
approval_policy = "on-request"

# 샌드박스: 파일 · 명령 접근 범위
#   read-only          = 읽기 전용 (분석 · 계획)
#   workspace-write    = 작업 폴더 안 읽기/쓰기/실행 (권장)
#   danger-full-access = 제한 없음 (특별한 경우만)
sandbox_mode = "workspace-write"

[sandbox_workspace_write]
# false = 외부 네트워크 차단(기본) / true = npm install 등 허용
network_access = false
```

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




편집기에서 연 config.toml




작성 칸 — 승인 요청 · 5.6 Sol Light

**TIP 모델 이름은 환경에 맞게 — 본인 계정의 모델 선택 메뉴에서 쓸 수 있는 최신 gpt-5.x 계열로 지정한다 (2026년 8월 기준 gpt-5.6-sol 권장).**

# Sandbox & Approvals — 안전하게 실행하기

샌드박스 = '무엇을 할 수 있나(범위)' · 승인 정책 = '언제 물어보나(시점)' — App 은 '승인 모드' 버튼 한 번으로 전환한다

## 읽기 전용

`sandbox = read-only`

수정 없이 분석 · 계획만

가장 안전

## Auto (권장)

`workspace-write + on-request`

작업 폴더 안은 자동, 폴더 밖 · 네트워크는 승인

균형 · 권장

## Full access

`danger-full-access + never`

제한 없음 — 격리된 VM / CI 에서만

위험 · 격리 환경

참고 네트워크는 기본 차단(보안)이다. 패키지 설치처럼 필요할 때만 한 번 허용한다 — App 은 승인 창의 '이번에만 허용', CLI 는 아래처럼 .

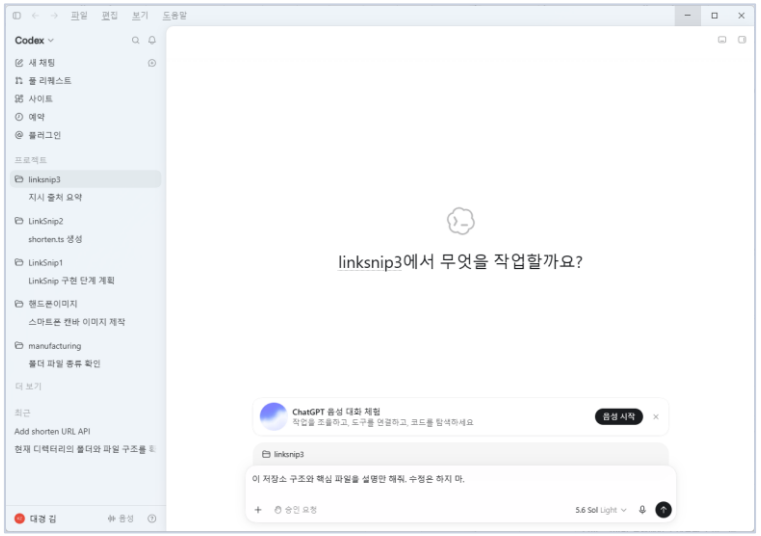
터미널 – 이번 실행에서만 네트워크 허용

```
$ codex -c 'sandbox_workspace_write.network_access=true' "필요한 npm 패키지를 설치해줘."
```

# 실습 — 권한을 바꿔 가며 실행 ① 읽기 전용으로 분석만

**▶ 작업 1 — 읽기 전용**

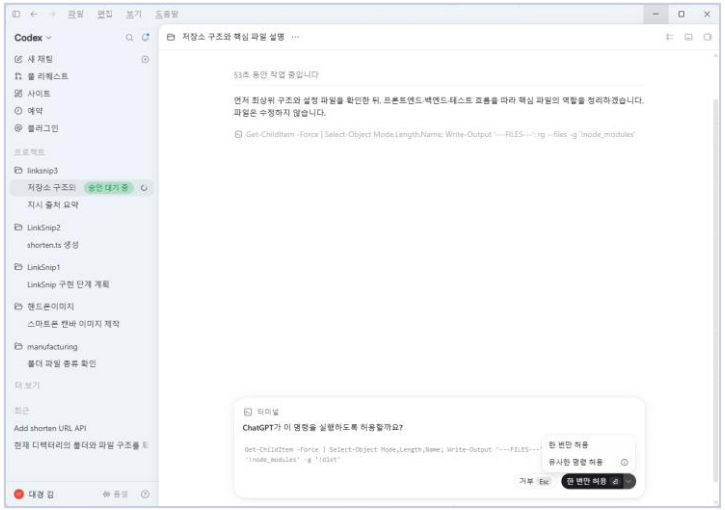
이 저장소 구조와 핵심 파일을 설명만 해줘. 수정은 하지 마.



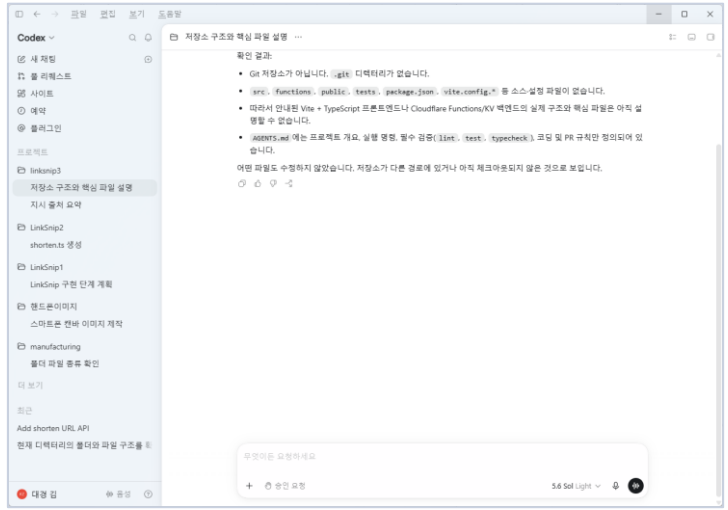

① linksnip3 에서 입력

CLI: codex --sandbox read-only "..." · 세션 도중에는 /permissions 로 즉시 전환

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② 명령 실행 승인 요청 — 읽기 명령만




③ 결과 — 구조 설명 · '어떤 파일도 수정하지 않았다'

# 실습 — 권한을 바꿔 가며 실행 ② Auto 로 실제 작업

**▶ 작업 2 — Auto**

README에 한 줄 설명을 추가해줘.

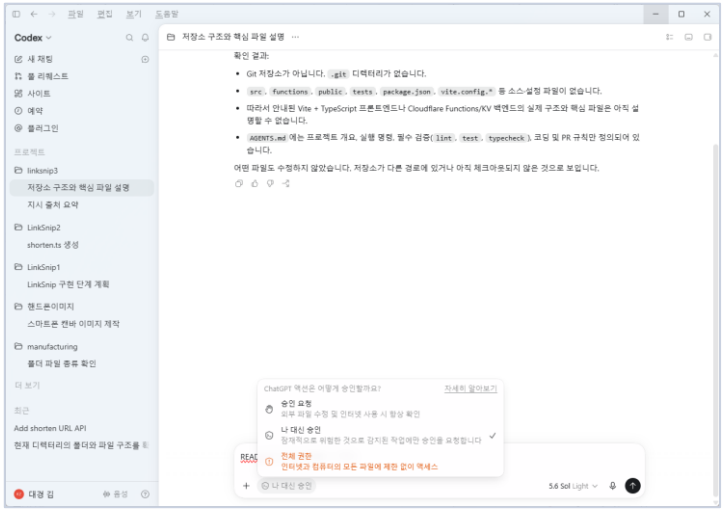
같은 일을 CLI 로

```
$ codex --sandbox workspace-write \
    --ask-for-approval on-request "…"
```

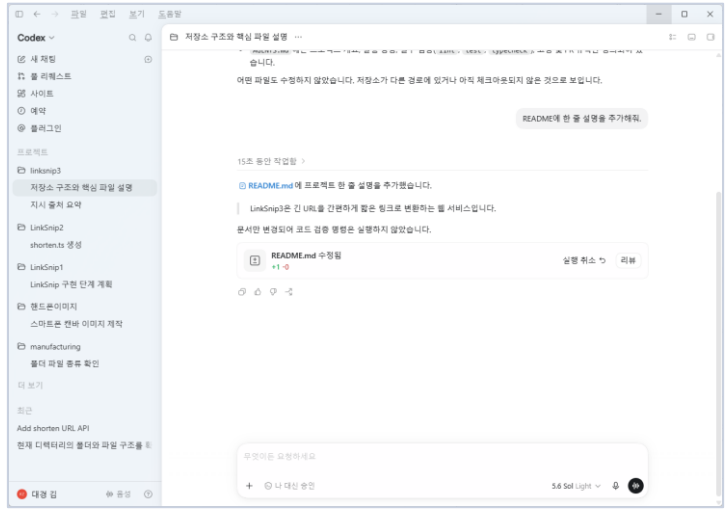
## 주의 — 이 옵션은 꼭 필요할 때만

- --dangerously-bypass-approvals-and-sandbox (별칭 --yolo) 는 모든 승인 · 샌드박스를 무력화한다
- 격리된 환경(CI 러너 / 일회용 VM)에서만 — 일반 PC에서는 절대 쓰지 않는다

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



① 승인 모드 선택 — 승인 요청 / 나 대신 승인 / 전체 권한



② 결과 — README.md 에 한 줄 추가 (+1)

# Skill — 반복 작업 지침을 폴더 하나로 묶는다

일반 프롬프트가 '이번 대화에서 무엇을 할지'라면, Skill 은 언제 · 어떤 절차와 기준으로 · 무엇을 조심하며 할지를 재사용하게 묶는다

### 가장 작은 구성

```
daily-summary/
└── SKILL.md
```

### 확장 구성 (선택)

```
daily-summary/
├── SKILL.md
├── scripts/      반복 실행할 도구
├── references/   필요할 때만 읽을 상세 지침
└── assets/       템플릿 · 이미지
```

### SKILL.md = 머리말 + 본문

```
---
name: daily-summary
description: 작업 메모를 간결한
  일일 업무 보고로 정리한다.
---
여기에 실제 작업 지침을 작성한다.
```

name · description 은 Codex 가 Skill 을 발견하고 고르는 데 쓴다. 이름은 소문자 · 숫자 · 하이픈, description 에는 '무엇을'과 '언제 쓰는지'를 분명히 적는다.

| 위치 | 경로 |
|---|---|
| 프로젝트 | <저장소>/.agents/skills/ |
| 사용자 전역 | ~/.agents/skills/ |

### 호출하는 법

- 작성 칸에 $ 를 입력해 고른다 (예: $daily-summary)
- CLI · IDE: /skills 로 찾아보기
- 작업이 description 과 맞으면 Codex 가 스스로 고르기도 한다

**TIP** 처음부터 복잡하게 만들지 않는다 — 실제로 쓰면서 반복적으로 틀리는 부분만 지침에 추가한다. 단순한 Skill 은 SKILL.md 하나로 충분하다.

# 실습 — daily-summary Skill 만들기 ① 폴더와 SKILL.md

폴더 구조

```
C:\DEV\Skill>tree /f
C:.
└─.agents
    └─skills
        └─daily-summary
                SKILL.md
```

**목표 출력 형식**

```
완료:
- 로그인 오류 수정
- 단위 테스트 추가

다음 작업:
- 모바일 화면 확인

이슈:
- 테스트 계정 필요
```

**파일: .agents/skills/daily-summary/SKILL.md**

```
---
name: daily-summary
description: 산발적인 작업 메모를 완료,
  다음 작업, 이슈로 구분한 간결한 일일
  업무 보고로 정리한다. 사용자가 업무 메모나
  데일리 보고 정리를 요청할 때 사용한다.
---

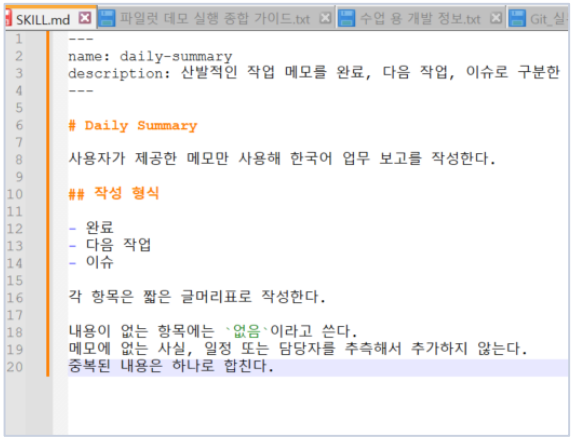
# Daily Summary

사용자가 제공한 메모만 사용해 한국어 업무
보고를 작성한다.

## 작성 형식
- 완료
- 다음 작업
- 이슈

각 항목은 짧은 글머리표로 작성한다.
내용이 없는 항목에는 `없음`이라고 쓴다.
메모에 없는 사실, 일정 또는 담당자를
추측해서 추가하지 않는다.
중복된 내용은 하나로 합친다.
```

**실행 결과 (화면)**




편집기에서 연 SKILL.md

이 예제에는 스크립트나 참고자료가 필요 없다 — SKILL.md 하나로 끝난다.

# 실습 — daily-summary Skill ② 호출하고 확인하기

**▶ 프롬프트 — Codex App**

**$daily-summary를** 사용해서 아래 메모를 정리해줘.

오늘 로그인 오류를 수정했고 관련 단위 테스트도 추가했다.
모바일 화면은 내일 확인해야 한다.
테스트 계정이 아직 없다.

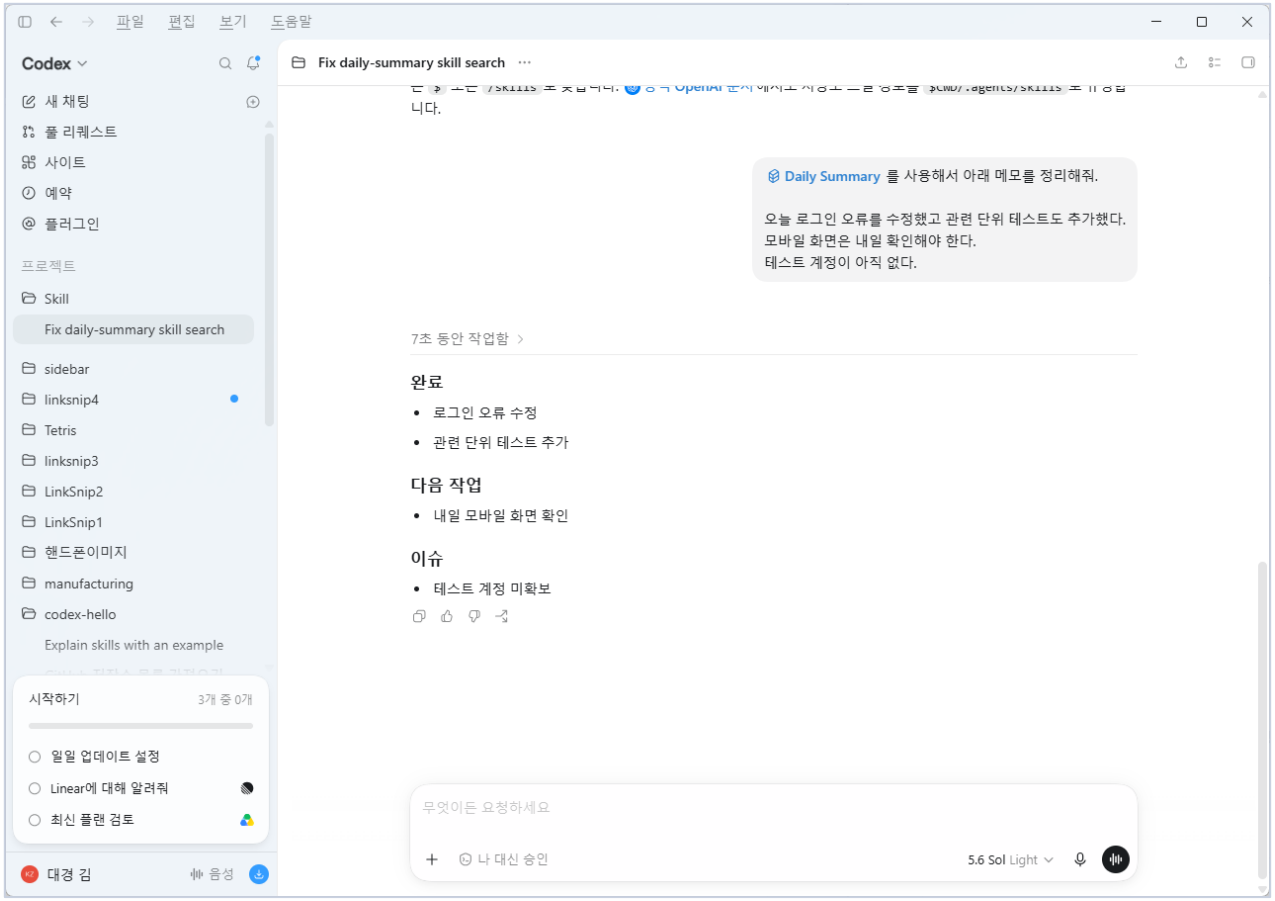



작성 칸 — $daily-summary 가 Skill 칩으로 표시된다

## 잘 작동하는지 확인

- ✅ 입력에 없던 사실을 만들지 않았는가?
- ✅ 세 항목으로 정확히 분류했는가?
- ✅ 내용이 없는 항목을 '없음'으로 표시했는가?

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




결과 — 완료 · 다음 작업 · 이슈로 정리된 일일 보고

# 한 걸음 더 — 대화 세션 내보내기 Skill ① 구조와 SKILL.md

스크립트까지 포함한 Skill — 진행 중인 Codex 작업(Thread)을 읽기 좋은 Markdown 으로 내보낸다 (회의록 · 인수인계 · 버그 재현 기록)

**① 폴더 구조**

```
.agents/
  skills/
    thread-export/
        SKILL.md
        scripts/
            export_thread.py
```

daily-summary 와 달리 scripts 폴더가 추가된다.

무엇을(what) · 언제(when) 쓰는지를 description 에 분명히 적고, 실행 방법과 출력 형식을 본문에 못 박는다.

**② 파일: .agents/skills/thread-export/SKILL.md**

```
---
name: thread-export
description: 현재 Codex 작업(Thread)의 대화 세션을 사람이 읽기 좋은 Markdown 파일로 내보낸다.
  사용자가 "이 대화 내보내줘", "세션 기록 저장", "스레드 export"를 요청할 때 사용.
---
# Thread Export
현재 진행 중인 Codex 세션(Thread)의 대화 기록을 Markdown 파일로 저장한다.

## 동작 방식
1. Codex 세션 로그 폴더(기본: ~/.codex/sessions)에서 가장 최근 세션 파일을 찾는다.
2. scripts/export_thread.py 를 실행해 세션을 Markdown 으로 변환한다.
3. 결과를 프로젝트의 exports/ 폴더에 thread-YYYYMMDD-HHMMSS.md 로 저장한다.

## 실행
python .agents/skills/thread-export/scripts/export_thread.py --out exports/

## 출력 형식
- 문서 맨 위에 세션 파일명, 내보낸 시각, 메시지 수를 요약한다.
- 각 발화는 "## 사용자" / "## Codex" 제목 아래 순서대로 기록한다.
- 코드 블록과 명령 실행 결과는 원문 그대로 보존한다.

## 주의
- 세션 파일을 찾지 못하면 "세션 파일을 찾을 수 없음"이라고 알리고 멈춘다.
- 내보낸 파일에는 토큰 · 비밀키가 포함될 수 있으니 공유 전에 반드시 확인한다.
```

지금 할 일 프로젝트에 .agents/skills/thread-export/ 와 scripts/ 폴더 만들기 ▸ SKILL.md 작성 (전역으로 쓰려면 ~/.agents/skills/)

# 대화 세션 내보내기 Skill ② 스크립트 export_thread.py

세션 로그(JSONL)를 읽어 Markdown 으로 바꾼다 — 세션 파일 형식이 다를 수 있어 role / content 키를 방어적으로 처리한다

### ③ scripts/export_thread.py (1/2)

```
import argparse, glob, json, os, datetime

def find_latest(sessions_dir):
    files = glob.glob(os.path.join(sessions_dir, "**", "*.jsonl"), recursive=True)
    return max(files, key=os.path.getmtime) if files else None

def to_markdown(path):
    lines, count = [], 0
    with open(path, encoding="utf-8") as f:
        for raw in f:
            raw = raw.strip()
            if not raw:
                continue
            msg = json.loads(raw)
            msg = msg.get("payload", msg)          # rollout 형식이면 payload 사용
            if "role" not in msg:
                continue                           # 메시지만 기록
            role = msg["role"]
            text = msg.get("content") or msg.get("text") or ""
            if isinstance(text, list):
                text = "".join(p.get("text", "") for p in text)
            title = "사용자" if role == "user" else ("Codex" if role == "assistant" else role)
            lines += ["## " + title, "", text, ""]
            count += 1
    return lines, count
```

### ③ scripts/export_thread.py (2/2)

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--sessions-dir", default=os.path.expanduser("~/.codex/sessions"))
    ap.add_argument("--out", default="exports")
    args = ap.parse_args()
    latest = find_latest(args.sessions_dir)
    if latest is None:
        print("세션 파일을 찾을 수 없음:", args.sessions_dir); raise SystemExit(1)
    body, count = to_markdown(latest)
    os.makedirs(args.out, exist_ok=True)
    stamp = datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
    out_path = os.path.join(args.out, "thread-" + stamp + ".md")
    header = ["# Thread Export", "",
              "- 세션 파일: " + os.path.basename(latest),
              "- 내보낸 시각: " + stamp,
              "- 메시지 수: " + str(count), "", "---", ""]
    with open(out_path, "w", encoding="utf-8") as f:
        f.write("\n".join(header + body))
    print("내보내기 완료:", out_path)

if __name__ == "__main__":
    main()
```

지금 할 일 scripts/export_thread.py 에 아래 코드 저장 (App 에 '아래 코드로 파일을 만들어줘' 요청해도 된다)

# 대화 세션 내보내기 Skill ③ 실행하고 확인하기

**▶ 방법 1 — Skill 호출**

`$thread-export` 를 사용해서 지금까지의 대화를 내보내줘.

방법 2 — 스크립트 직접 실행

```
$ python .agents/skills/thread-export/scripts/export_thread.py --out exports/
```

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ thread-export 스킬로 내보냈는다.
> python .agents/skills/thread-export/scripts/export_thread.py --out exports/
내보내기 완료: exports/thread-20260817-141205.md

# Thread Export
- 세션 파일: rollout-2026-08-17T13-58-02.jsonl
- 메시지 수: 14
```

## 잘 작동하는지 확인

- ✅ 가장 최근 세션이 정확히 선택되었는가?
- ✅ 사용자 · Codex 발화가 순서대로, 빠짐없이 기록되었는가?
- ✅ 코드 블록과 명령 실행 결과가 원문 그대로 보존되었는가?
- ✅ 내보낸 파일에 토큰 · 비밀키가 노출되지 않았는가?

## 7장 확인

- ✅ ~/.codex/AGENTS.md 와 linksnip3/AGENTS.md 를 만들었다
- ✅ 로드 확인 요청이 두 파일을 인용했다
- ✅ config.toml 에 model · approval · sandbox 를 설정했다
- ✅ 읽기 전용 / Auto 를 각각 실행해 봤다 · Skill 을 호출해 봤다

**지금 할 일** $thread-export 로 호출 (또는 스크립트 직접 실행) ▸ exports/ 폴더의 md 파일을 열어 네 가지 확인

08

CHAPTER 8 · 강의 + 실습

# 하네스(Harness) 엔지니어링 — 검증하는 틀

하네스는 Codex 의 작업을 감싸는 '검증 · 안전 장치 묶음'이다. 코드베이스를 마음대로 망가뜨리지 못하게 막고, 만든 결과가 '진짜로 동작하는지'를 매번 자동으로 확인한다.

8.1–8.4 정의 · 4요소 | 8.5–8.7 뼈대 · 완료의 정의 · 회귀 | 8.8 codex exec | 8.9 루프 · 모델

# 하네스의 정의 — Agent = Model + Harness

모델이 지능을 제공한다면, 하네스는 그 지능을 실제로 일하는 시스템으로 바꾼다

**Agent**
**= Model**
**+ Harness**

"하네스는 모델 자체가 아닌 모든 코드, 설정, 실행 로직이다."
— 비벡 트리베디, LangChain (2026.3)

**비유** 모델 = 엔진 · 하네스 = 차체 · 핸들 · 브레이크

| 하네스에 들어가는 것 | → Codex 에서는 |
|---|---|
| 1 시스템 프롬프트 · 상시 규칙 | AGENTS.md (7.1) |
| 2 도구 · 스킬 · MCP 와 그 설명 | 플러그인 · Skill · MCP (4.4 · 7.7) |
| 3 파일시스템 · 샌드박스 · 브라우저 | Sandbox · 인앱 브라우저 (7.6) |
| 4 오케스트레이션 — 서브에이전트 · 모델 라우팅 | 스레드 · 모델 선택 (8.9) |
| 5 결정적 처리를 하는 훅 — 압축 · 린트 검사 | 빌드 · 테스트 · 컴팩션 (8.4 · 6.6) |

출처 · Vivek Trivedy,「The Anatomy of an Agent Harness」, LangChain Blog (2026.3.10)

# 하네스 엔지니어링 — 인물 · 기관별 정의

2025년 말 ~ 2026년 초에 정립된 용어 — 공통점은 '실수를 구조로 막는다'는 것이다

| 출처 | 정의 | 관점 |
|---|---|---|
| 미첼 하시모토<br>HashiCorp 공동창업자 · 「My AI Adoption Journey」(2026.2) | "에이전트가 실수하는 것을 발견할 때마다, 그 실수를 다시는 하지 않도록 해결책을 설계하는 데 시간을 쓰는 것."<br>주로 AGENTS.md 와 설정 파일을 고쳐서 실천한다. | 실수 → 구조적 재발 방지. 용어를 대중화 |
| OpenAI · 라이언 로포폴로<br>「Harness engineering: leveraging Codex in an agent-first world」(2026.2) | "사람은 방향을 잡고, 에이전트가 실행한다." (Humans steer. Agents execute.)<br>사람의 일은 코드 작성 → 시스템 설계 · 의도 명세 · 피드백 장치 구축으로 이동 | 5개월간 사람이 직접 짠 코드 없이 약 100만 줄 개발 |
| 비르기타 뵈켈러<br>Thoughtworks · martinfowler.com (2026.4) | "AI 에이전트에서 모델을 제외한 모든 것. 결과물의 신뢰성을 높이기 위한 주변 인프라." | 제어 이론 관점 — 가이드 · 센서 (8.3) |
| Anthropic<br>「Effective harnesses for long-running agents」(2025.11) | 여러 세션에 걸쳐 오래 일하는 에이전트가 진행 상황을 잃지 않게 하는 실행 구조<br>진행 기록 파일 · 기능 목록 · git 커밋 · 테스트 활용 | 긴 작업의 연속성 |

출처 · M. Hashimoto, mitchellh.com(2026.2) · R. Lopopolo, openai.com/index/harness-engineering(2026.2.11) · B. Böckeler, martinfowler.com(2026.4.2) · Anthropic Engineering(2025.11)

# 가이드와 센서 — 세 엔지니어링의 관계

하네스는 행동 전에 방향을 잡는 '가이드'와 행동 뒤에 교정하게 하는 '센서'로 구성된다 (뵈켈러)

| 구분 | 가이드 (사전 제어 · Feedforward) | 센서 (사후 제어 · Feedback) |
|---|---|---|
| **역할** | 행동하기 전에 방향을 잡는다 | 행동한 뒤에 관찰하고 스스로 교정하게 한다 |
| **결정적 수단 (연산형)** | 규칙 파일 · 스켈레톤 · 아키텍처 경계<br>→ AGENTS.md · config.toml | 테스트 · 린터 · 타입 검사<br>→ npm test · npm run build |
| **의미 기반 수단 (추론형)** | 스킬 · 역할 지시<br>→ 플러그인 · P · C · T · F 의 P | 리뷰 에이전트 · LLM-as-judge<br>→ Codex 코드 리뷰 · 품질관리 Skill (10.3) |

**세 엔지니어링의 관계**

| 수준 | 최적화 대상 | 한 줄 정의 | 이 과정 |
|---|---|---|---|
| **프롬프트 엔지니어링** | 한 번의 요청 | 지시문을 잘 쓰는 기술 | 5장 |
| **컨텍스트 엔지니어링** | 모델이 보는 것 전체 | 컨텍스트 창을 딱 맞는 정보로 채우는 기술 (카파시) | 6장 |
| **하네스 엔지니어링** | 에이전트가 일하는 시스템 전체 | 모델 외 모든 것을 설계해 실수를 구조적으로 막는 기술 (하시모토) | 7장 · 8장 |

# 하네스를 이루는 4요소 — 중심은 빌드와 테스트

사람이 '확인'하는 대신 빌드 · 테스트가 '확인'하도록 판정 기준을 명령으로 고정해 두는 것이 출발점이다

| 요소 | 역할 | 이 과정에서의 구현 |
|---|---|---|
| 빌드 (Build) | 코드가 실제로 컴파일 / 번들되는가 | npm run build / typecheck |
| 테스트 (Test) | 기능이 의도대로 동작하는가 | npm test (Vitest) |
| 배포 (Deploy) | 산출물을 실제로 배포할 수 있는가 | GitHub Pages (10장) |
| 로그 (Log) | 실행 결과 · 에러를 확인할 수 있는가 | 테스트 / 런타임 로그 확인 |

### 되먹임(feedback) 루프

① Codex 가 코드를 바꾸고
② 빌드 · 타입체크 · 테스트를 돌려
③ 실패하면 로그를 읽고 원인을 고쳐
④ 다시 통과할 때까지 반복

**핵심** 빌드 · 테스트 = 하네스의 두 기둥 — 빌드가 깨지면 실행할 산출물이 없고, 테스트가 없으면 '동작하는 것처럼 보이는' 코드와 '실제로 동작하는' 코드를 구분할 수 없다.

**참고** 하네스 = '검증 가능한 규칙' + '권한 경계' — 5장(검증 가능한 프롬프트)과 7장(AGENTS.md · Sandbox / Approvals)이 모여 하네스가 된다. 발목을 잡는 장치가 아니라 안심하고 속도를 내게 해 주는 안전망이다.

# 실습 — 하네스를 갖춘 프로젝트 뼈대 만들기 (linksnip3)

패키지 설치를 위해 네트워크 허용이 필요하다 — App 이라면 승인 창에서 허용한다

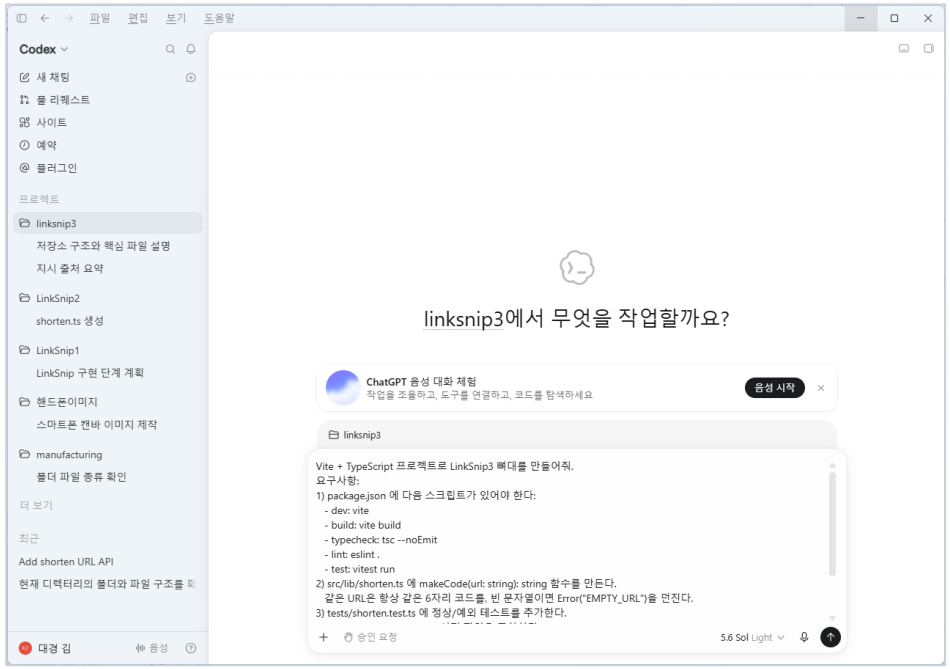
**▶ 프롬프트 — 하네스 갖춘 뼈대 생성**

Vite + TypeScript 프로젝트로 linksnip3 뼈대를 만들어줘.
요구사항:
1) package.json 에 다음 스크립트가 있어야 한다:
- dev: vite
- build: vite build
- typecheck: tsc --noEmit
- lint: eslint .
- test: vitest run

2) src/lib/shorten.ts 에 makeCode(url: string): string 함수를 만든다.
같은 URL은 항상 같은 6자리 코드를, 빈 문자열이면 Error("EMPTY_URL")을 던진다.
3) tests/shorten.test.ts 에 정상/예외 테스트를 추가한다.
4) ESLint, Vitest, TypeScript 설정 파일을 포함한다.
끝나면 npm install 후 npm run typecheck, npm run lint, npm test 를 차례로 실행해서 모두 통과하는 것을 보여줘.

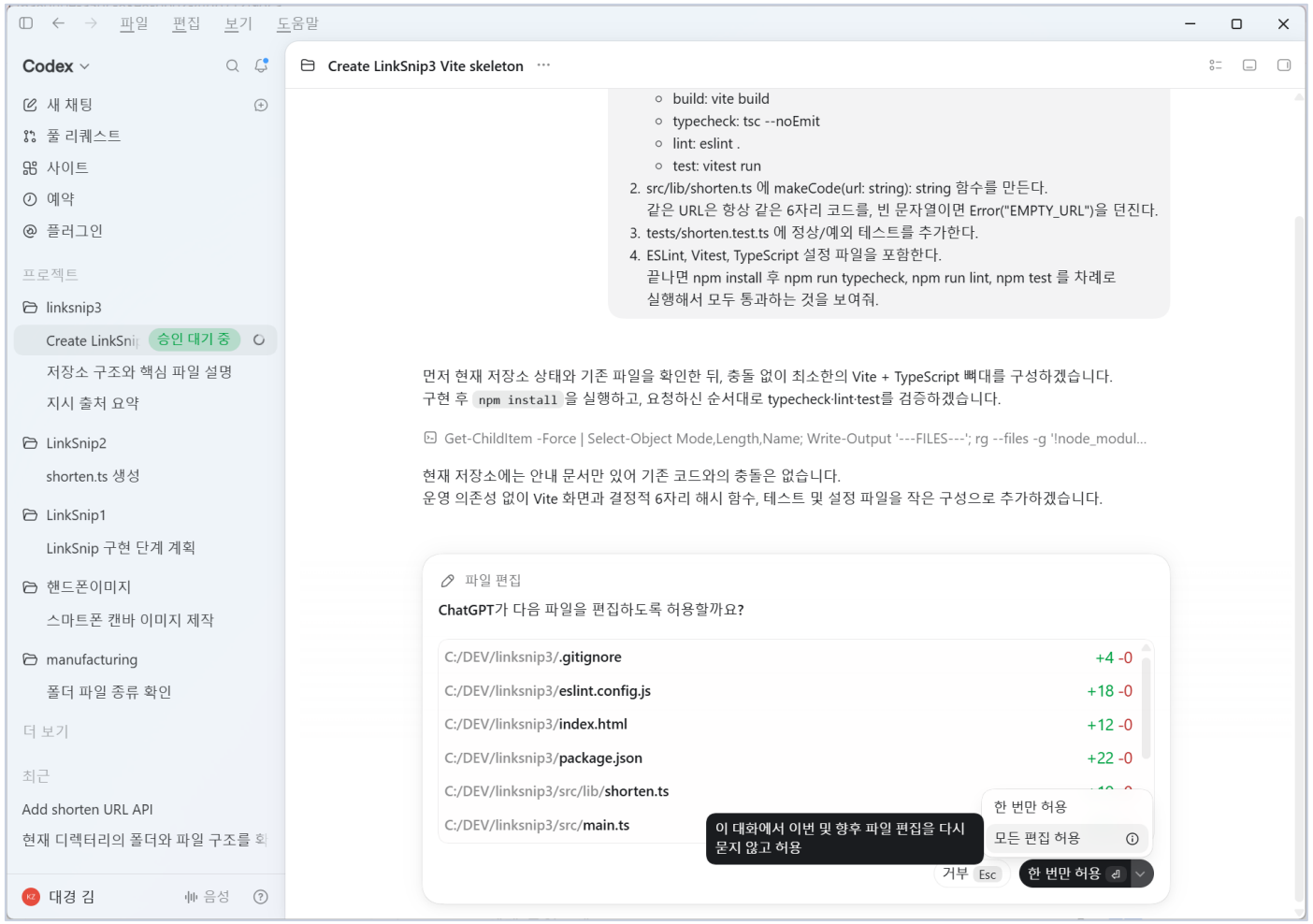
CLI 로 시작할 경우

```
$ cd linksnip3
$ codex -c 'sandbox_workspace_write.network_access=true' \
    --ask-for-approval on-request
```

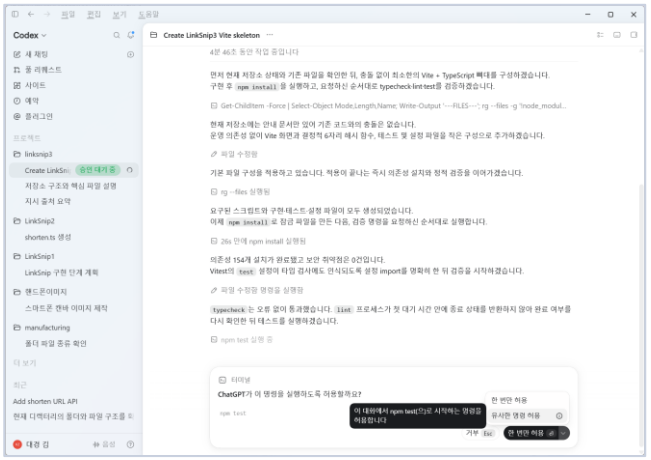



App 작성 칸에 붙여 넣은 모습

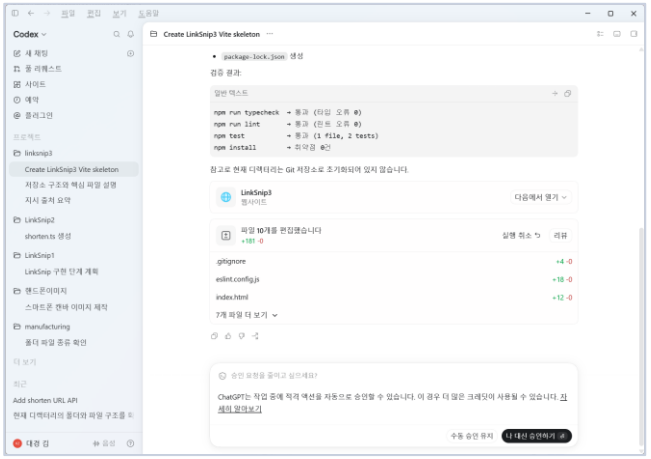
# 뼈대 생성 진행 — 파일 편집과 명령 실행 승인




① 파일 편집 승인 — .gitignore · eslint.config.js · index.html · package.json · shorten.ts …

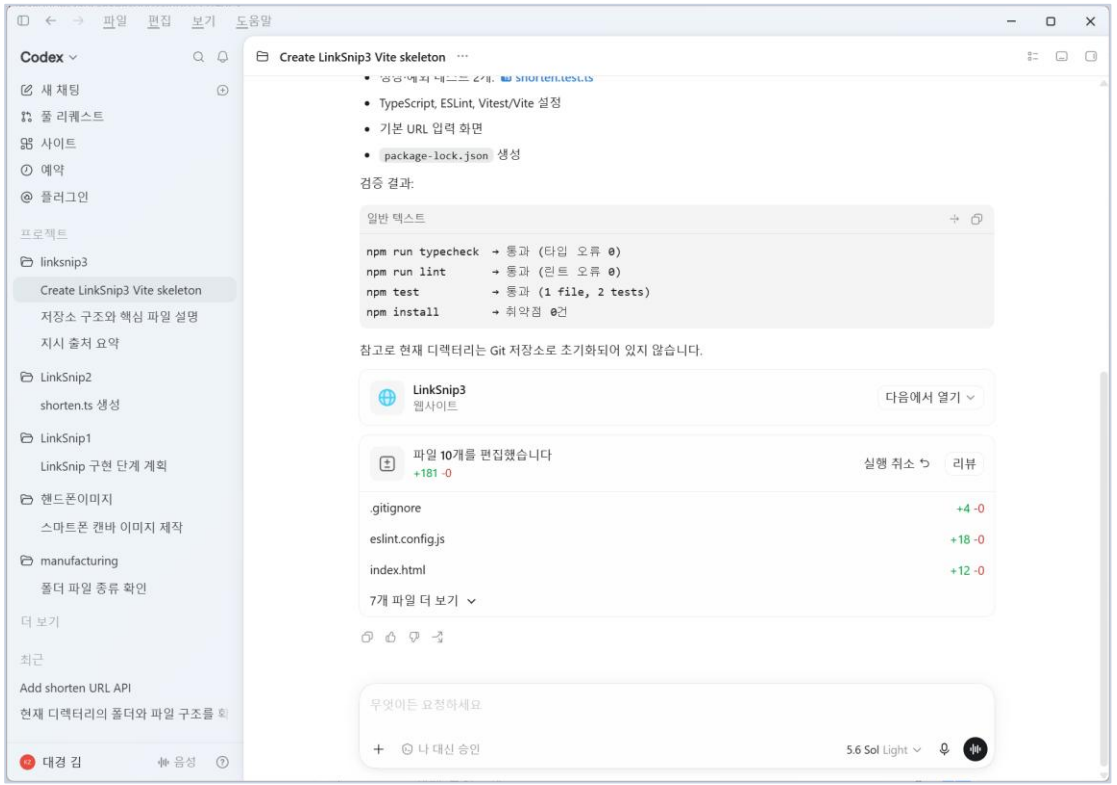


② npm test 실행 승인 요청

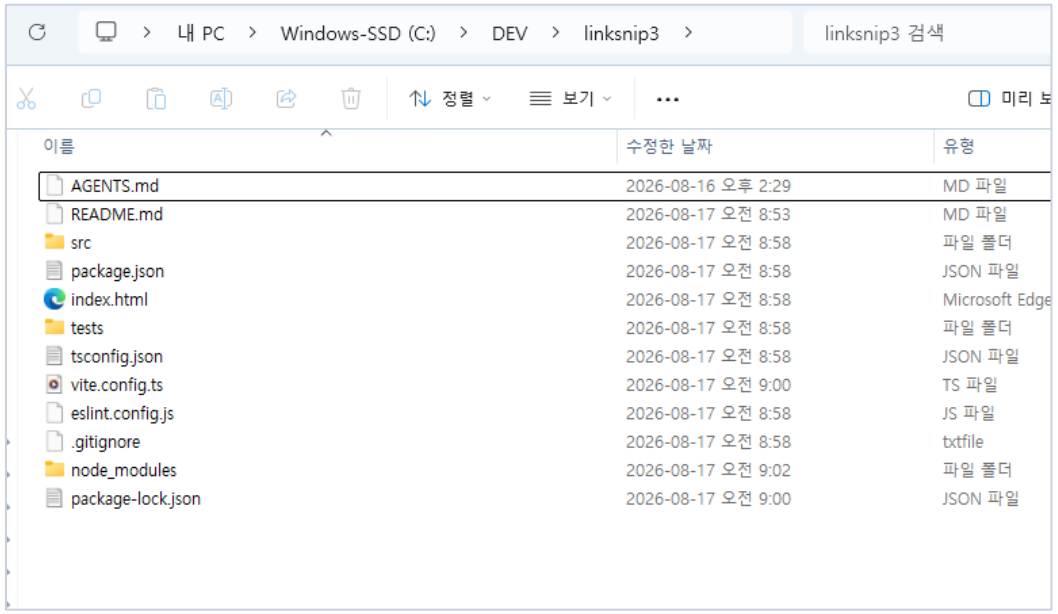


③ 검증 결과 요약

# 뼈대 생성 결과 — 세 명령이 모두 통과하면 하네스 완성




결과 — typecheck 통과 · lint 통과 · test 통과 (1 file, 2 tests)




linksnip3 — package.json · src · tests · tsconfig · vite.config · eslint.config

**결과 예시**

```
검증 결과
> npm run typecheck   → 통과 (타입 오류 0)
> npm run lint        → 통과 (린트 오류 0)
> npm test            → 통과 (1 file, 2 tests)
```

**핵심** PASS 세 개 = 하네스 완성. 이제부터는 Codex 가 바꾼 코드도 같은 세 명령으로 판정한다.

# 실습 — AGENTS.md 에 '완료의 정의' 못 박기

7.3 에서 만든 AGENTS.md 의 검증 규칙이 실제 명령과 일치하는지 확인하고, 한 가지 강한 규칙을 추가한다

**▶ 프롬프트 — Codex App**

linksnip3/AGENTS.md 맨 아래에 다음 절을 덧붙여줘.

## 작업 완료의 정의 (Definition of Done)
다음 세 명령이 모두 통과하기 전에는 작업을 끝났다고 하지 말 것:
1) npm run typecheck  (타입 에러 0)
2) npm run lint  (린트 에러 0)
3) npm test  (테스트 전부 통과)
실패하면 원인을 고치고 다시 실행한다. 통과 로그를 마지막에 요약해서 보여 준다.

**TIP**

'완료의 정의'가 하네스의 심장 — 무엇을 통과해야 끝인지 명문화하면, 따로 시키지 않아도 매번 빌드 · 린트 · 테스트를 돌리고 실패를 스스로 고친다. 이것이 반복 수정 (iterative repair) 루프다.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다

```
1  # AGENTS.md (LinkSnip3 프로젝트 규칙)
2  ## 프로젝트 개요
3  LinkSnip3은 긴 URL을 짧게 만드는 웹 서비스다.
4  프론트엔드: Vite + TypeScript / 백엔드: Cloudflare Pages Functions + Workers KV.
5  ## 셋업
6  - 의존성 설치: npm install
7  - 개발 서버: npm run dev
8  - 빌드: npm run build
9  ## 변경 시 반드시 지킬 것 (검증 규칙)
10 - 코드를 바꾼 뒤에는 반드시 'npm run lint' 와 'npm test' 를 실행하고 둘 다 통과하는 것을 확인한 뒤 마무리할 것.
11 - 타입 에러가 0이어야 한다('npm run typecheck').
12 - 비밀키/토큰을 코드에 하드코딩하지 말 것. 환경변수를 사용한다.
13 ## PR(풀 리퀘스트) 규칙
14 - 커밋 메시지는 한 줄 요약 + 변경 이유로 작성한다.
15 ## 작업 완료의 정의 (Definition of Done)
16 다음 세 명령이 모두 통과하기 전에는 작업을 끝났다고 하지 말 것:
17 1) npm run typecheck   (타입 에러 0)
18 2) npm run lint        (린트 에러 0)
19 3) npm test           (테스트 전부 통과)
20 실패하면 원인을 고치고 다시 실행한다. 통과 로그를 마지막에 요약해서 보여 준다.
21
```

결과 — AGENTS.md 15–20행에 '작업 완료의 정의' 추가

# 실습 — 일부러 깨고, 하네스가 해결하는 지 본다

테스트가 빨간색으로 실패했다가, 수정 후 다시 통과하는 흐름을 직접 보면 '하네스 = 안전망'이라는 감이 온다

**▶ 프롬프트 — 실패 → 복구 루프 체험**

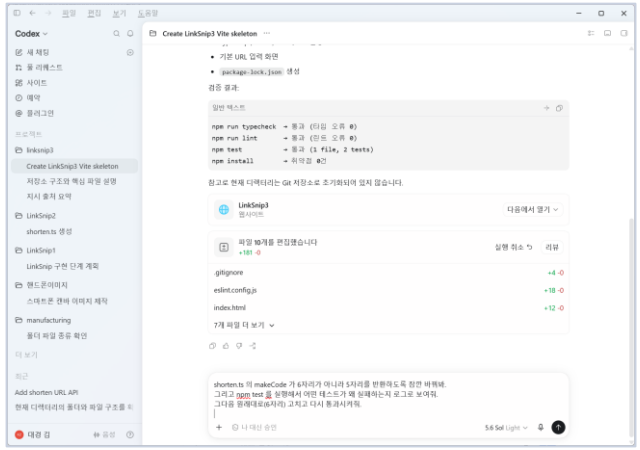
shorten.ts 의 makeCode 가 6자리가 아니라 5자리를 반환하도록 잠깐 바꿔봐.
그리고 npm test 를 실행해서 어떤 테스트가 왜 실패하는지 로그로 보여줘.
그다음 원래대로(6자리) 고치고 다시 통과시켜줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

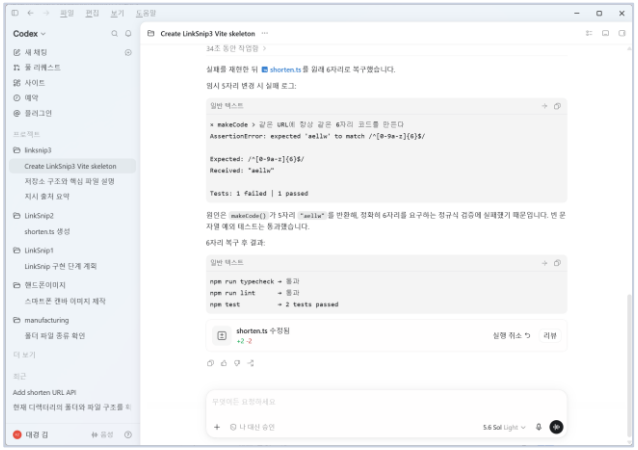
회귀 → 복구

```
> npm test
× makeCode > 같은 URL은 항상 같은 6자리 코드를 반환
AssertionError: expected 'aellu' to match /^[0-9a-z]{6}$/
Tests  1 failed | 1 passed
# 6자리로 복구
> npm run typecheck  → 통과
> npm run lint       → 통과
> npm test           → 2 tests passed
```

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



① 프롬프트 입력



② 결과 — 실패 로그 → 6자리로 복구 → 모두 통과

# 비대화형 하네스 — codex exec 로 CI · 스크립트 자동화

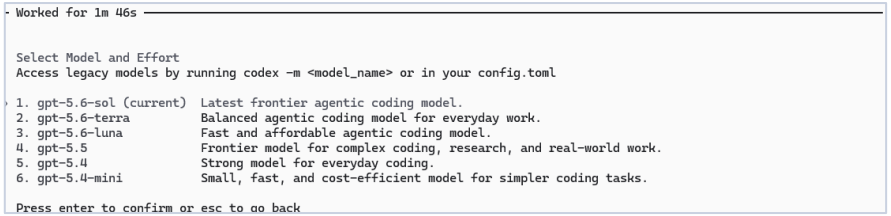
codex exec (짧게 codex e) 는 사람이 지켜보지 않아도 끝까지 도는 비대화형 실행이다

터미널 / CI 스크립트

```
# 비대화형으로 작업 + 기계가 읽을 결과 + 최종 요약 저장
$ codex exec -m gpt-5.6-sol --json --output-last-message result.txt \
    "테스트를 실행하고, 실패가 있으면 고친 뒤 다시 통과시켜줘."

# CI 전용(이미 외부 격리가 있는 경우에만): 승인/샌드박스 우회
# codex exec --dangerously-bypass-approvals-and-sandbox "npm ci && npm run build && npm test"
```

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



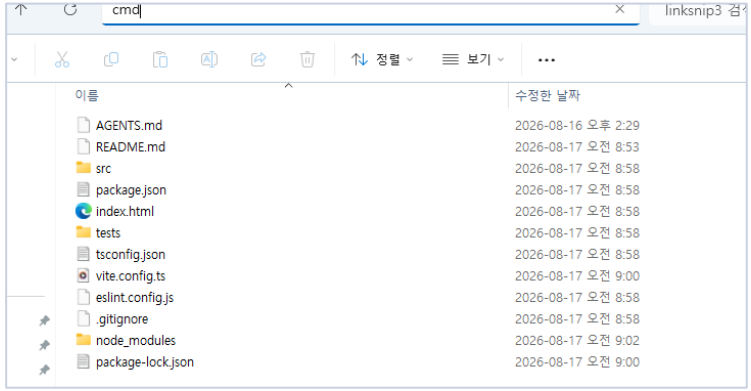

/model — 모델과 추론 강도 선택

**1 모델 확인**

Codex CLI 의 > 뒤에 /model 입력 → Enter → 목록에서 GPT-5.6 Sol 이 선택되어 있는지 확인

**2 나머지는 계속 Enter**

추론 강도 등 기본값으로 진행



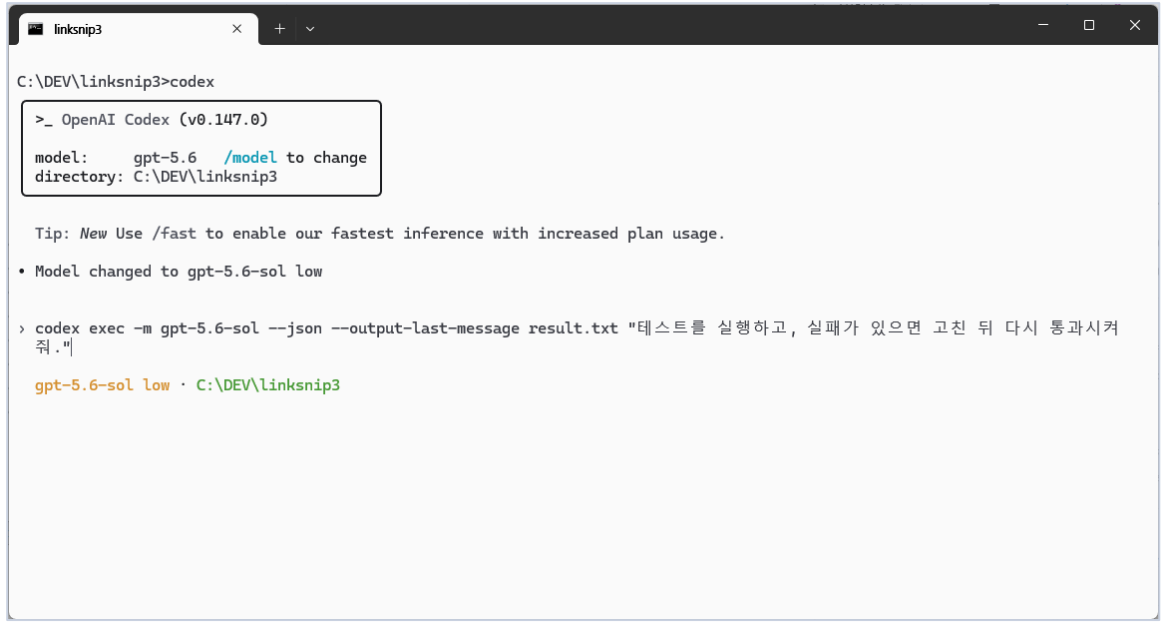

linksnip3 폴더 주소창에 cmd → 그 위치에서 터미널 열기

## 주의 — 우회 옵션은 '격리된 CI' 한정

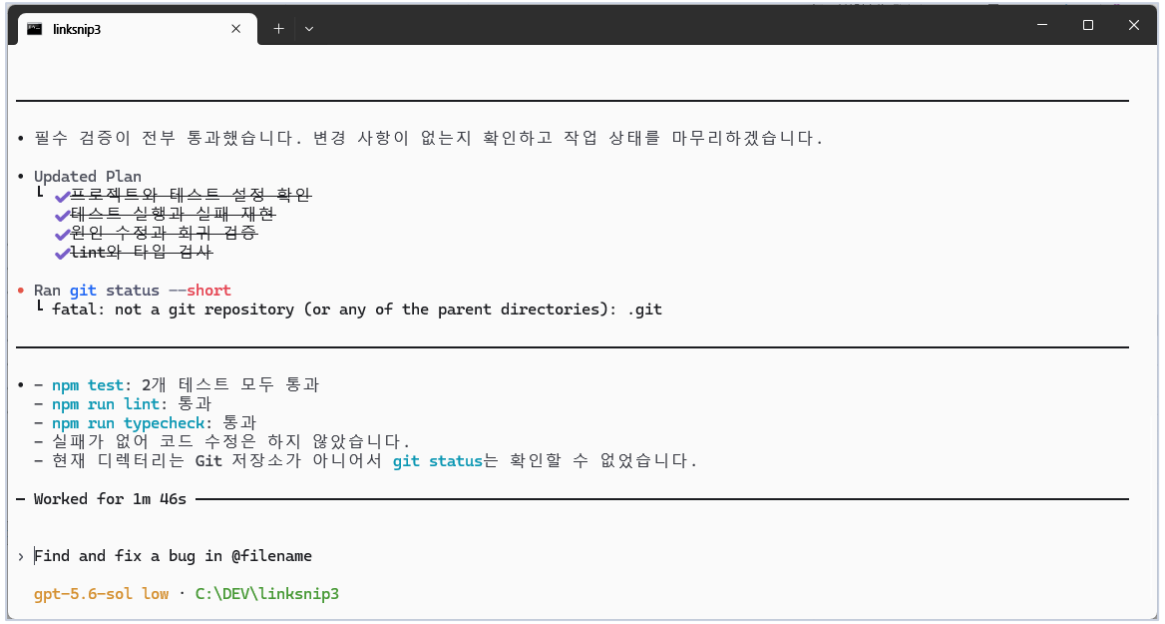
- --dangerously-bypass-approvals-and-sandbox 는 신뢰된 일회용 러너에서만 — 개발용 노트북에서는 쓰지 않는다

# codex exec 실행 결과 — 끝까지 혼자 돌고 요약을 남긴다

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




① codex 시작 → 모델 gpt-5.6-sol low 로 변경 → codex exec 입력




② 결과 — 계획 체크 · npm test 2개 통과 · lint · typecheck 통과

**결과 예시**

result.txt — 마지막 메시지

result.txt ▸ 필수 검증이 전부 통과했다. 변경 사항이 없는지 확인하고 작업 상태를 마무리했다. (npm test 2개 · lint · typecheck 통과 / git 저장소 아님)

# 루프 엔지니어링과 모델 엔지니어링 — 하네스 다음 단계

반복을 자동화하고(루프), 단계마다 모델을 배분한다(모델)

| 용어 | 정의 | 출처 |
|---|---|---|
| **루프 엔지니어링**<br>**Loop Engineering** | 사람이 턴마다 지시하지 않아도 에이전트를 반복 실행하고, 결과를 검증하고, 스스로 이어 가게 하는 자동화된 반복 구조의 설계 | 애디 오스마니 (2026.6) |
| **앤드류 응의 3중 루프** | ① 에이전트 코딩 루프 (작성 → 테스트 → 반복)<br>② 개발자 피드백 루프 (사람의 검토 · 방향 조정)<br>③ 외부 피드백 루프 (사용자 반응) | ADTmag (2026.7) |
| **모델 엔지니어링**<br>**(해석)** | 단계마다 어떤 모델을 어떤 추론 강도로, 어떤 속도 · 비용으로 쓸지 설계하는 일 — Agent = Model + Harness 의 'Model' 쪽 설계 | 정립된 용어는 아님 |

### ⚠ 하네스가 루프의 전제 조건

- 테스트 · 규칙 · 샌드박스 없이 루프를 돌리면 잘못된 코드가 그만큼 빠르게 쌓인다

### Codex App 에서는

- 예약 — 루프의 트리거 · 주기 (4.3)
- 스레드 분리 · 검증 지시 — 반복마다 격리
- 모델 · 추론 강도 선택 (8.8)

### 하네스 설계 체크리스트

- ✅ 빌드 / 타입체크 / 린트 / 테스트를 한 줄로 실행
- ✅ '완료의 정의'가 AGENTS.md 에
- ✅ 실패 시 스스로 고치게 했나
- ✅ 위험 작업은 샌드박스 / 승인으로

09

CHAPTER 9 · 실습

# PRD 작성 · To-Do 도출 — 제품을 글로 정의하기

코드를 짜기 전에 '무엇을 왜 만드는가'를 한 페이지로 정리한 문서가 PRD(Product Requirements Document, 제품 요구사항 정의서)다. PRD 가 좋으면 Codex 가 만들 결과도 좋아진다.

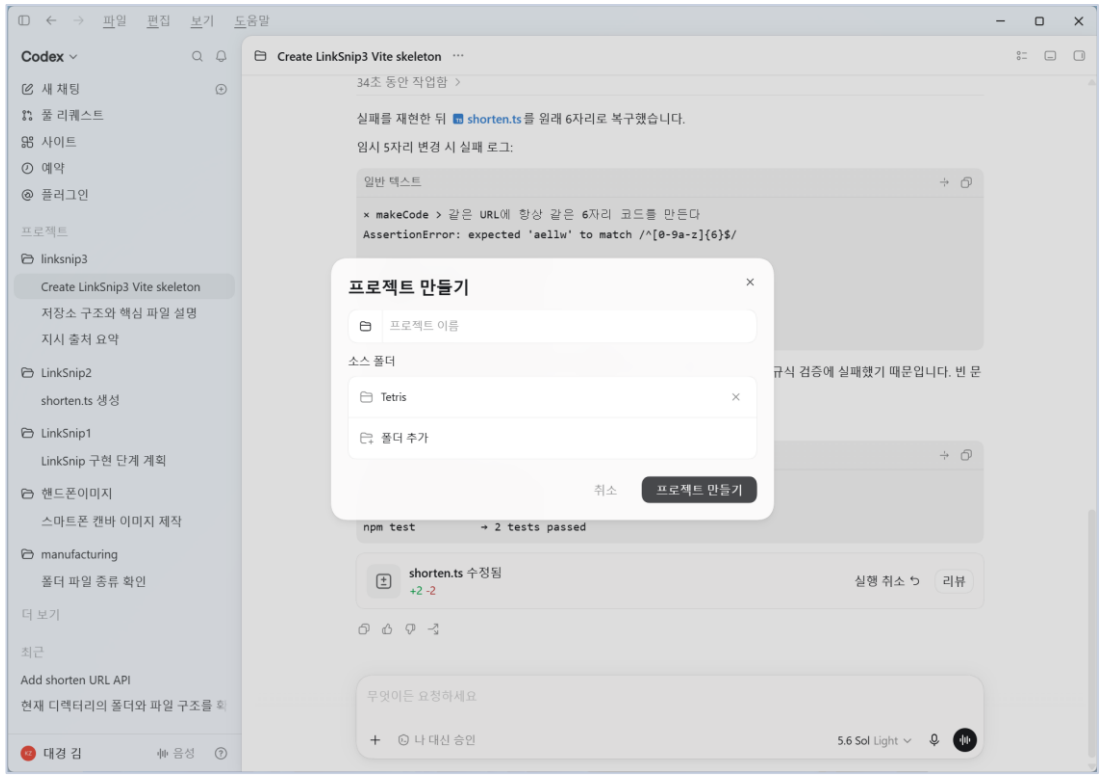
9.1 Tetris PRD

9.2 To-Do 도출

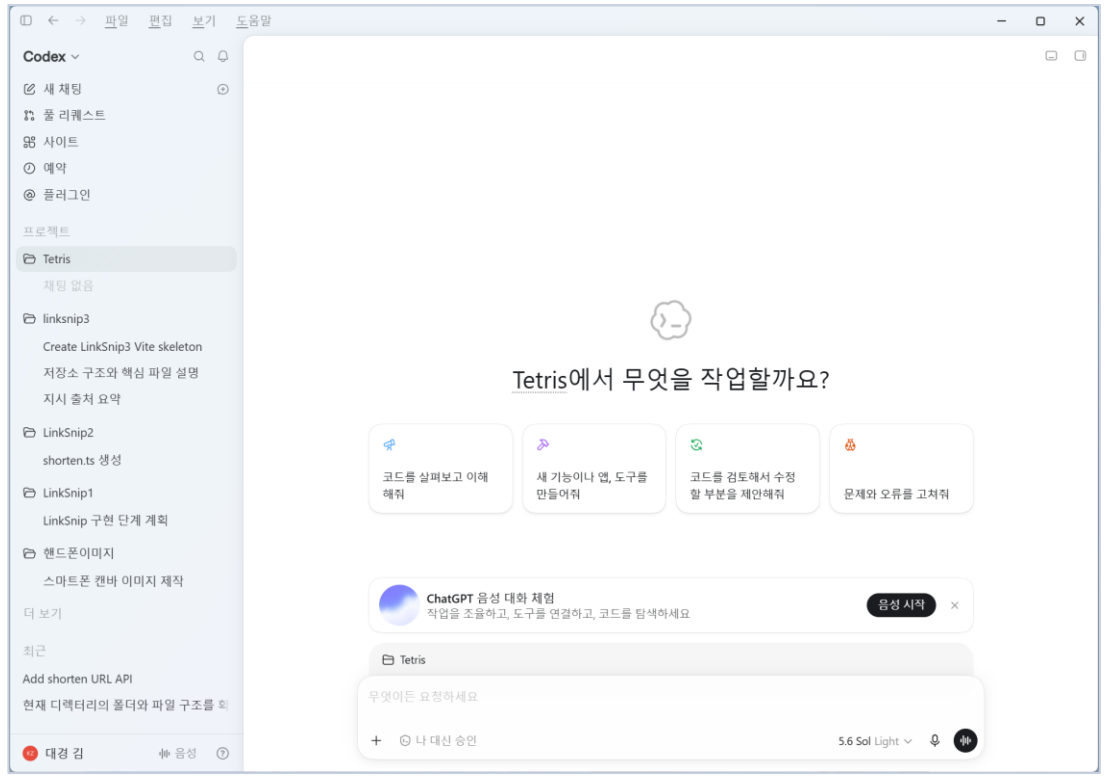
# 실습 준비 — Tetris 프로젝트 만들기

PRD 를 쓰고 → @ 로 참조해 To-Do 를 받는 흐름은 모든 프로젝트에 그대로 적용된다

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




① 프로젝트 만들기 — 소스 폴더 Tetris




② Tetris 프로젝트가 열린 상태

로드맵 9.1 PRD → 9.2 To-Do → 10.4–10.8 게임 만들기 → 10.9 실행 · 테스트 → 10.10 GitHub → 10.11 GitHub Pages → 10.12 README · 재배포

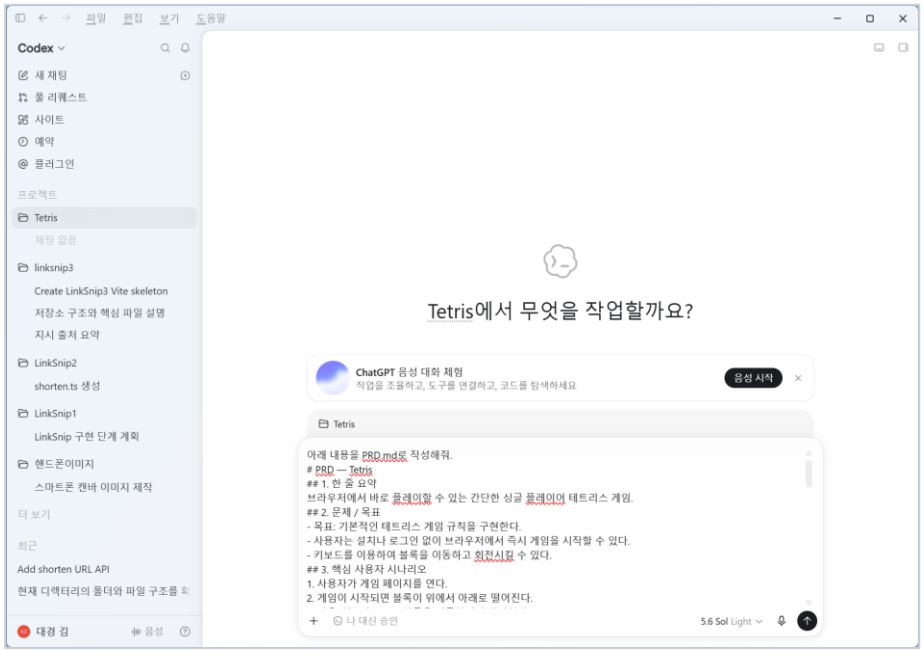
# Tetris PRD (완성 예시) — 그대로 붙여 넣어 PRD.md 로 저장한다

▶ 프롬프트 — Codex App

**아래 내용을 PRD.md로 작성해줘.**

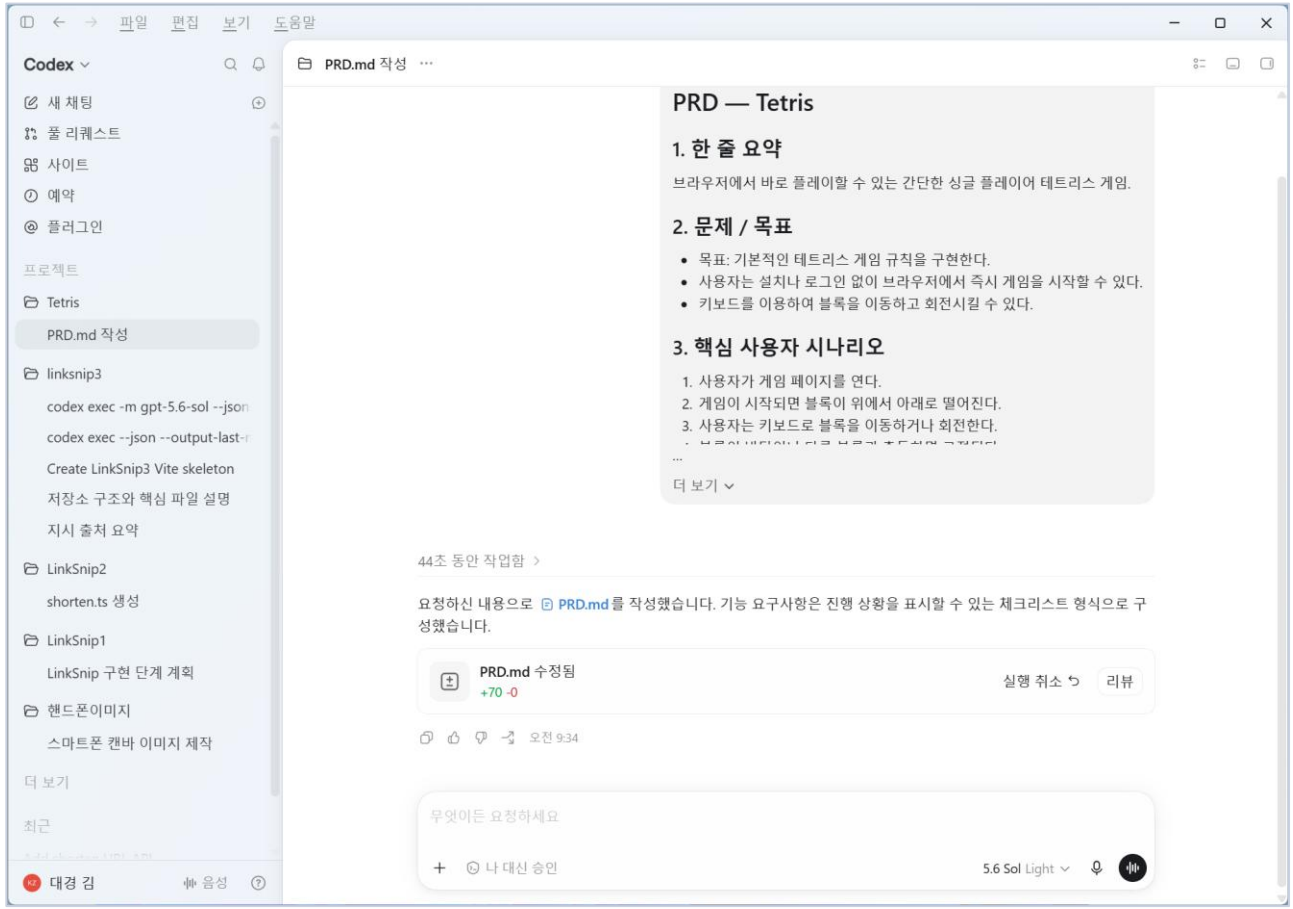
```
# PRD — Tetris
## 1. 한 줄 요약
브라우저에서 바로 플레이할 수 있는 간단한 싱글 플레이어 테트리스 게임.
## 2. 문제 / 목표
- 목표: 기본적인 테트리스 게임 규칙을 구현한다.
- 사용자는 설치나 로그인 없이 브라우저에서 즉시 게임을 시작할 수 있다.
- 키보드를 이용하여 블록을 이동하고 회전시킬 수 있다.
## 3. 핵심 사용자 시나리오
1. 사용자가 게임 페이지를 연다.
2. 게임이 시작되면 블록이 위에서 아래로 떨어진다.
3. 사용자는 키보드로 블록을 이동하거나 회전한다.
4. 블록이 바닥이나 다른 블록과 충돌하면 고정된다.
5. 한 줄이 완성되면 해당 줄이 제거된다.
6. 줄 제거에 따라 점수가 증가한다.
7. 블록이 게임 보드 상단까지 쌓이면 게임이 종료된다.
8. 사용자는 Restart 버튼으로 게임을 다시 시작할 수 있다.
## 4. 기능 요구사항 (MVP)
- [F1] 10 × 20 게임 보드 생성     - [F2] 7종 Tetromino 생성
- [F3] 블록 자동 하강            - [F4] 좌우 이동
- [F5] 블록 회전                 - [F6] Soft Drop
- [F7] 벽과 다른 블록에 대한 충돌 감지  - [F8] 블록 고정
- [F9] 완성된 라인 제거          - [F10] 점수 계산
- [F11] Game Over 판정           - [F12] Restart 기능
## 5. 비기능 요구사항
- 데스크톱 브라우저에서 정상 동작
- 입력에 즉각적으로 반응할 것
- 게임 로직과 UI를 가능한 한 분리할 것
- 핵심 게임 로직을 테스트할 수 있는 구조로 작성할 것
- 외부 유료 서비스에 의존하지 않을 것
## 6. 범위 제외
이번 MVP에서는 다음 기능을 구현하지 않는다.
- 사용자 로그인 · 온라인 멀티플레이 · 랭킹 서버
- 아이템 · Hold 기능 · Ghost Piece
- 모바일 터치 컨트롤 · 사운드 효과
위 기능은 이후 버전에서 추가할 수 있다.
## 7. 성공 기준
- 7종 Tetromino가 정상적으로 생성된다.
- 블록 이동과 회전이 정상 동작한다.
- 충돌 시 블록이 정확한 위치에 고정된다.
- 완성된 라인이 정상적으로 제거된다.
- 점수가 정상적으로 증가한다.
- Game Over 조건이 정상적으로 동작한다.
- Restart 후 게임 상태가 초기화된다.
```

# PRD 작성 결과 — PRD.md 가 만들어진다




① 작성 칸에 PRD 프롬프트 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② 결과 — PRD.md 작성 (+70) · 미리보기

## PRD 를 읽는 법

- 4절 [F1]~[F12] 가 곧 구현 항목
- 7절 성공 기준이 곧 10.9 테스트 시나리오

지금 할 일 결과의 PRD.md 카드 확인 ▸ 파일 트리에서 PRD.md 를 열어 1–7절이 모두 있는지 확인

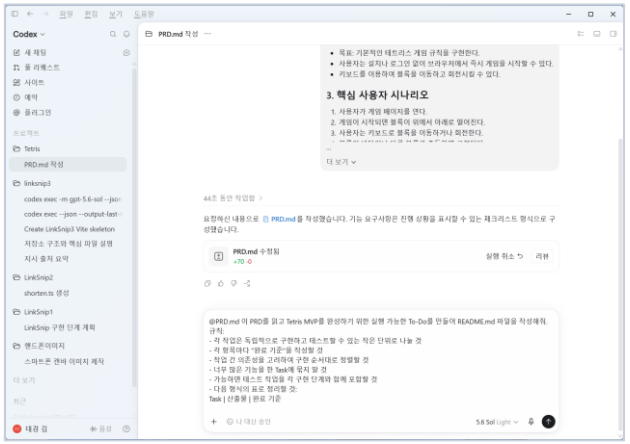
# 실습 — PRD 로부터 실행 가능한 To-Do 를 도출해 README.md 로

**▶ 프롬프트 — Codex App**

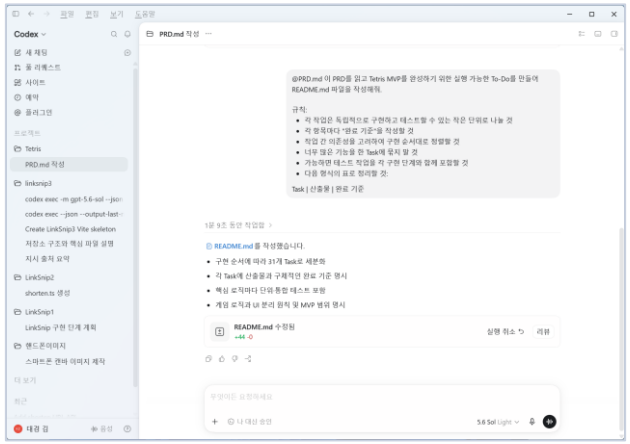
`@PRD.md` 이 PRD를 읽고 Tetris MVP를 완성하기 위한 실행 가능한 To-Do를 만들어 README.md 파일을 작성해줘.
규칙:
- 각 작업은 독립적으로 구현하고 테스트할 수 있는 작은 단위로 나눌 것
- 각 항목마다 "완료 기준"을 작성할 것
- 작업 간 의존성을 고려하여 구현 순서대로 정렬할 것
- 너무 많은 기능을 한 Task에 묶지 말 것
- 가능하면 테스트 작업을 각 구현 단계와 함께 포함할 것
- 다음 형식의 표로 정리할 것:
Task | 산출물 | 완료 기준

**TIP** 계획만 먼저 보고 싶으면 읽기 전용 모드에서 '파일은 만들지 말고 표로만 보여줘'를 붙인다. 마음에 들면 Auto 로 바꿔 README.md 로 저장한다.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



① 입력 (Auto 모드)



② 결과 — README.md 작성 (+44) · 31개 Task

10

CHAPTER 10 · 실습

# Tetris 만들고 GitHub Pages 에 배포하기

각 '프롬프트' 상자의 내용을 그대로 복사해 Codex 작성 칸에 붙여 넣는다. 한 번에 완벽하지 않아도 괜찮다 — 단계마다 품질 게이트(품질관리 Skill)로 스스로 점검하며 반복한다.

10.1–10.3 산출물 · 품질 게이트 | 10.4–10.9 게임 만들기 | 10.10–10.12 GitHub · 배포

# 최종 산출물과 로드맵 — 코딩 경험 0 에서도  완주

Tetris · 순수 HTML / CSS / JS · 외부 연동 없음 · GitHub Pages 배포

### 이 장을 끝내면 손에 남는 것

- Tetris 프로젝트 폴더 — index.html · style.css · script.js
- README.md — 실행 방법 · 조작법 · 기능 목록 · 배포 링크
- 품질 점검 기록 — code-review · qa-playtest · bug-hunt · release-check
- GitHub 저장소와 GitHub Pages 배포 URL




**핵심** 완료 조건 — 블록 낙하 · 조작 · 줄 삭제 · 점수 · 게임 오버가 모두 동작하고, GitHub Pages 주소로 접속된다.

**TIP** 문제가 생기면 증상을 구체적으로 적어 디버깅 프롬프트로 — 예) "블록이 벽을 통과해요. 벽 경계에서 멈추도록 충돌 검사를 고쳐줘."

# 품질 게이트가 포함된 실습 흐름

각 단계에는 품질 게이트가 하나씩 붙어 있다 — 확인 기준을 통과해야 다음 단계로 넘어가며, 품질관리 Skill 을 호출해 스스로 점검한다

| 단계 | 구현 작업 | 품질관리 Skill | 확인 기준 |
|---|---|---|---|
| 0 | 프로젝트 골격 생성 (10.4) | `$review-structure` | 파일 구조, 역할 분리, 실행 가능성 확인 |
| 1 | 보드와 블록 렌더링 (10.5) | `$code-review` | DOM 구조, 렌더링 함수, 가독성 확인 |
| 2 | 낙하 · 충돌 · 조작 구현 (10.6) | `$review-game-logic` | 충돌 판정, 경계값, 회전 오류 확인 |
| 3 | 라인 삭제 · 점수 · 게임오버 (10.7) | `$qa-playtest` | 사용자 시나리오별 기대 결과 확인 |
| 4 | 버그 수정과 리팩터링 (10.9) | `$bug-hunt` · `$refactor-safe` | 고위험 버그 제거, 기능 변화 없는 구조 개선 |
| 5 | README 와 배포 준비 (10.12) | `$release-check` | 파일 연결, 실행법, GitHub Pages 설정 확인 |

**참고** Skill 은 작성 칸에 $ 를 붙여 호출한다 (7.7). 아직 만들지 않았다면 다음 슬라이드(10.3)의 프롬프트로 한 번에 만든다.

# 실습 — 품질관리 Skill 7개를 Codex 로 한 번에 만들기

**▶ 프롬프트 — 품질관리 Skill 일괄 생성**

.agents/skills/ 아래에 아래 목록의 품질관리 skill 을 각각 SKILL.md 로 만들어줘.
공통 규칙:
- 폴더명과 name 은 이름에서 앞의 "$"를 뗀 소문자 · 하이픈 형태로 한다(예: **$code-review** → code-review).
- description 에는 "무엇을 점검하는지"와 "언제 호출하는지"를 한 문장으로 적는다.
- 본문에는 각 skill 의 확인 기준을 점검 체크리스트로 옮기고, 항목마다 통과/실패 판정 기준을 한 줄씩 붙인다.
- 원칙적으로 코드를 직접 고치지 말고, 발견한 문제를 "항목 / 심각도(높음 · 중간 · 낮음) / 개선안" 표로 보고한다.
- 예외: refactor-safe 는 기능과 공개 API 를 바꾸지 않는 선에서 리팩터링을 직접 수행하고, 변경 전후로 테스트가 모두 통과함을 보여 준다.
만들 skill:
- review-structure : 파일 구조, 역할 분리, 실행 가능성
- code-review : DOM 구조, 렌더링 함수, 가독성
- review-game-logic : 충돌 판정, 경계값, 회전 오류
- qa-playtest : 사용자 시나리오별 기대 결과(예: 4줄=800점, 게임오버 후 재시작)
- bug-hunt : 고위험 버그 탐지 및 우선순위화
- refactor-safe : 기능 변화 없는 구조 개선
- release-check : 파일 연결, 실행법, GitHub Pages 설정
끝나면 생성한 폴더 트리와 각 SKILL.md 의 name · description 을 표로 요약해서 보여줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ .agents/skills/ 에 7개 skill 을 만들었다.
  name                description(요약)
  review-structure    파일 구조 · 실행 가능성 점검
  code-review         DOM · 렌더링 · 가독성 점검
  review-game-logic   충돌 · 경계값 · 회전 점검
  qa-playtest         시나리오별 기대 결과 점검
  bug-hunt            고위험 버그 우선순위화
  refactor-safe       기능 불변 리팩터링 수행
  release-check       배포 전 최종 점검
```

**확인**

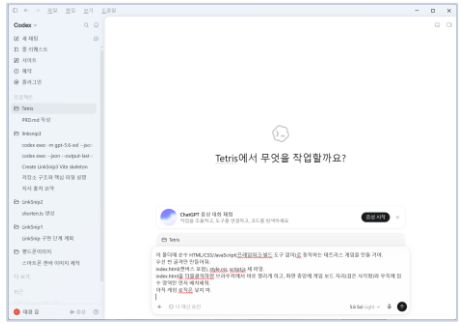
- ✅ 7개 폴더와 SKILL.md 가 생겼는가?
- ✅ description 에 '무엇을 · 언제'가 있는가?
- ✅ $code-review 호출 시 기준대로 보고하는가?

**지금 할 일** Tetris 프로젝트에서 아래 프롬프트 ▸ .agents/skills/ 아래 7개 폴더와 SKILL.md 확인 ▸ $code-review 를 한 번 호출해 보기

# 프롬프트 0 — 프로젝트 골격 (index.html · style.css · script.js)

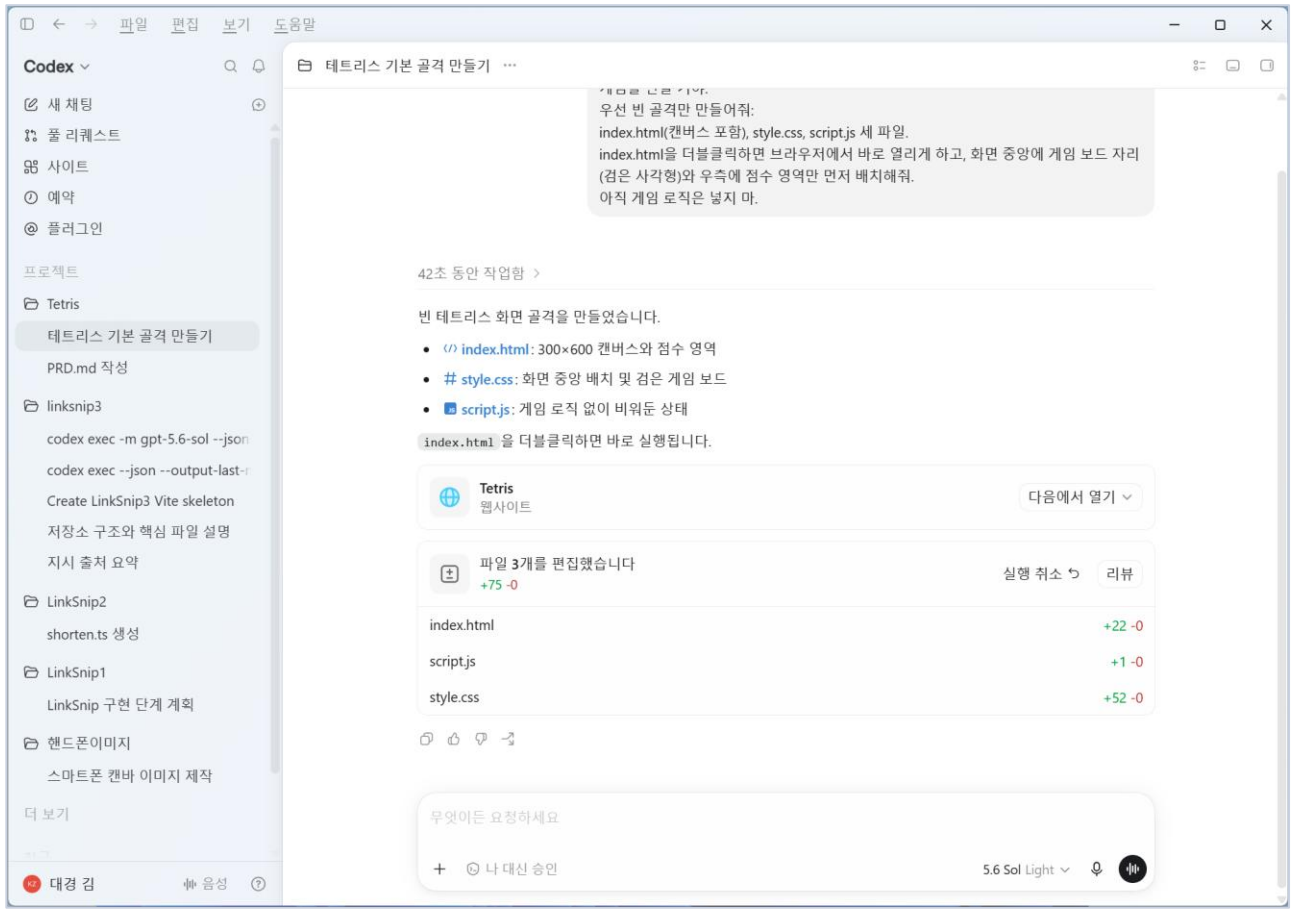
**▶ 프롬프트 0 — 프로젝트 골격**

이 폴더에 순수 HTML/CSS/JavaScript(프레임워크 · 빌드 도구 없이)로 동작하는 테트리스 게임을 만들 거야.
우선 빈 골격만 만들어줘:
index.html(캔버스 포함), style.css, script.js 세 파일.
index.html을 더블클릭하면 브라우저에서 바로 열리게 하고, 화면 중앙에 게임 보드 자리(검은 사각형)와 우측에 점수 영역만 먼저 배치해줘.
아직 게임 로직은 넣지 마.




작성 칸에 입력

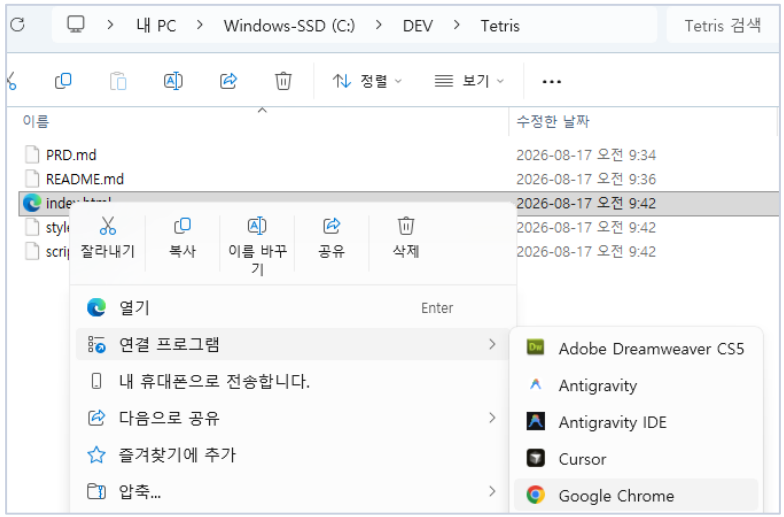
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



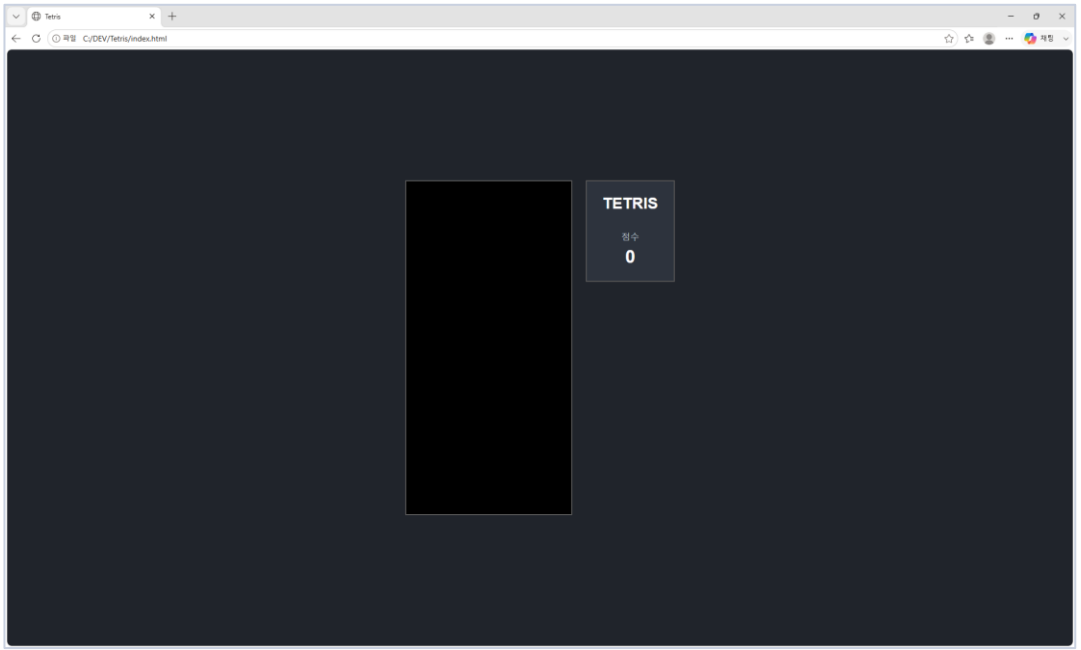

결과 — index.html 300×600 캔버스 · style.css · script.js (+75)

실습 · 직접 따라 하기 10장 · 10.4 | 프롬프트 0

# 골격 확인 — 브라우저로 열고 품질 게이트 0 을 통과한다




① index.html → 연결 프로그램 → Google Chrome




② 결과 — 검은 보드와 우측 'TETRIS / 점수 0'

**▶ 품질 게이트**

`$review-structure` 로 파일 구조 · 역할 분리 · 실행 가능성을 점검해줘.

**결과 예시** 지적된 부분을 고친 뒤 다음 단계로

```
품질 게이트 0 — 결과 예시

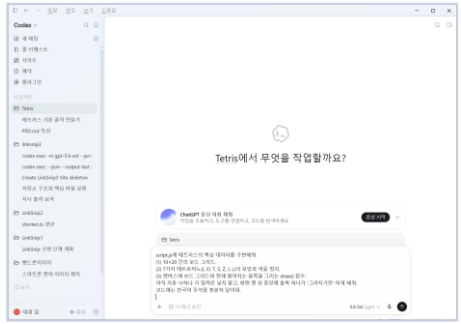
✦ $review-structure 점검 결과   항목              심각도   개선안
  · 파일 구조 (3개 분리)        통과              —
  · script.js 로드 위치         중간   </body> 직전 또는 defer 로 변경
```

지금 할 일 탐색기: index.html → 연결 프로그램 → Chrome ▸ 빈 보드 · 점수 영역 확인 ▸ $review-structure 실행 → 지적 사항 반영

# 프롬프트 1 — 보드 그리드와 테트로미노 정의

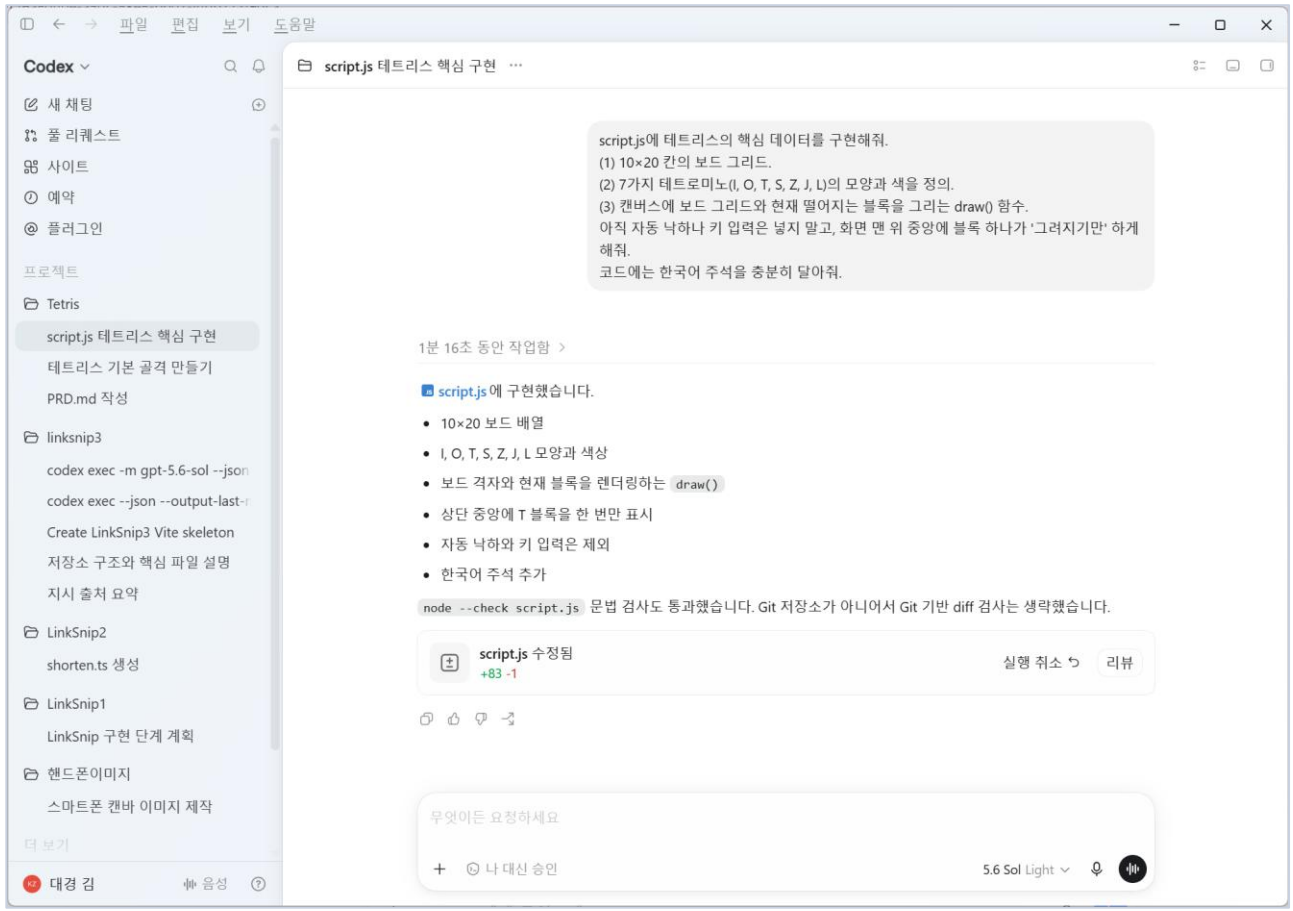
**▶ 프롬프트 1 — 보드 · 테트로미노**

script.js에 테트리스의 핵심 데이터를 구현해줘.
(1) 10×20 칸의 보드 그리드.
(2) 7가지 테트로미노(I, O, T, S, Z, J, L)의 모양과 색을 정의.
(3) 캔버스에 보드 그리드와 현재 떨어지는 블록을 그리는 draw() 함수.
아직 자동 낙하나 키 입력은 넣지 말고, 화면 맨 위 중앙에 블록 하나가 '그려지기만' 하게 해줘.
코드에는 한국어 주석을 충분히 달아줘.




작성 칸에 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다

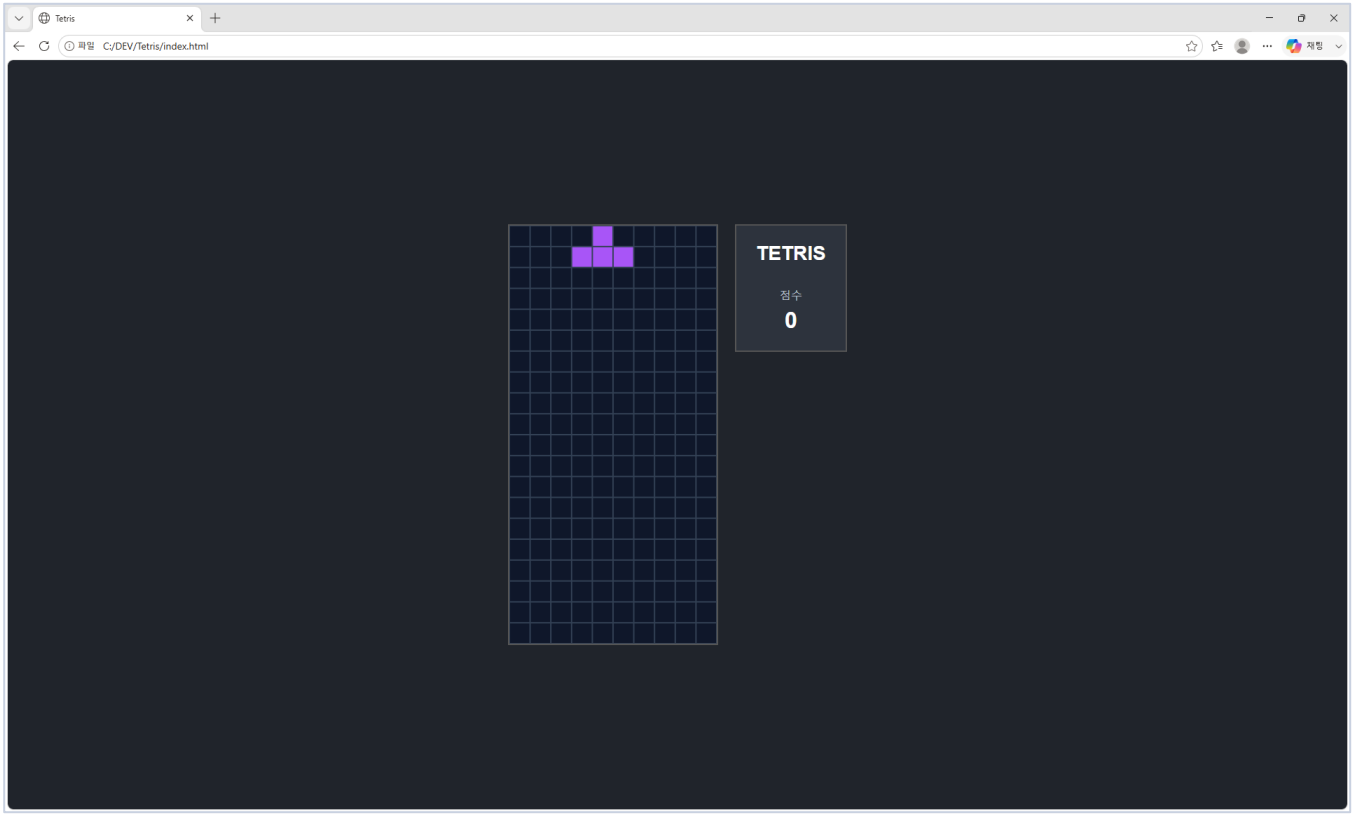



결과 — 10×20 배열 · I,O,T,S,Z,J,L · draw() · 한국어 주석

실습 · 직접 따라 하기 10장 · 10.5 | 프롬프트 1

# 보드 · 블록 확인 — 품질 게이트 1

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




결과 — 그리드 위쪽 중앙에 T 블록이 '그려지기만' 한다

**▶ 품질 게이트**

`$code-review` 로 DOM 구조 · 렌더링 함수 · 가독성을 점검해줘.

**결과 예시**

품질 게이트 1 — 결과 예시

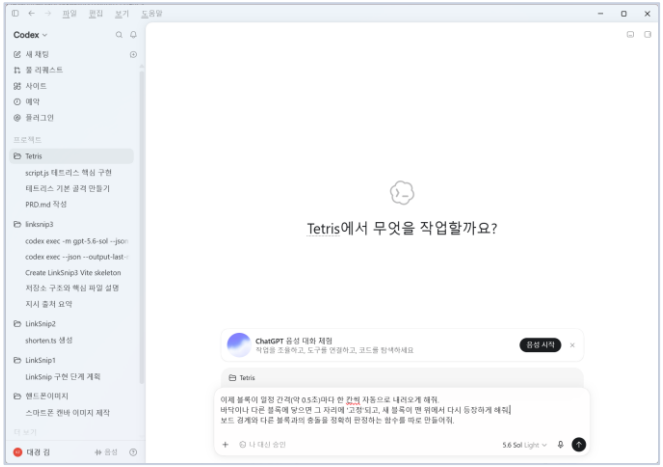
```
✦ $code-review 결과
  항목                    심각도   개선안
  draw() 가 보드 · 블록을
  한 함수에서 처리        중간     drawBoard /
                                   drawPiece 분리
  매직 넘버 30(px)        낮음     CELL 상수화
  주석 · 이름             통과     —
```

지금 할 일 브라우저 새로고침 → 보라색 T 블록 확인 ▸ $code-review 실행 ▸ 지적 사항을 반영해야 이 단계 통과

# 프롬프트 2 — 자동 낙하와 충돌 판정

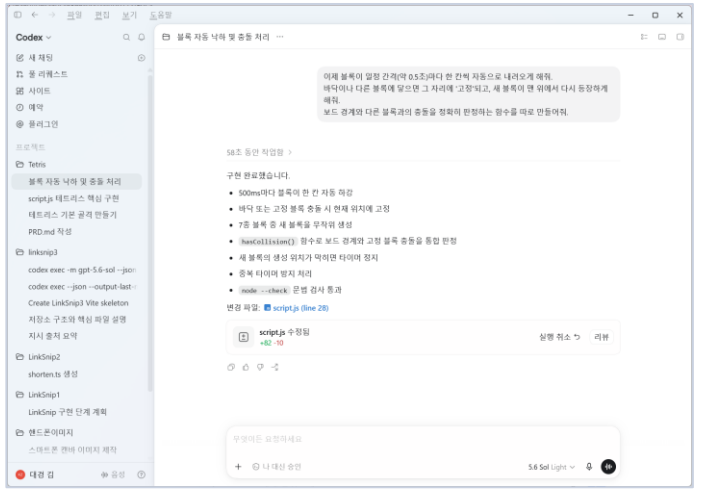
**▶ 프롬프트 2 — 자동 낙하 · 충돌**

이제 블록이 일정 간격(약 0.5초)마다 한 칸씩 자동으로 내려오게 해줘.
바닥이나 다른 블록에 닿으면 그 자리에 '고정'되고, 새 블록이 맨 위에서 다시 등장하게 해줘.
보드 경계와 다른 블록과의 충돌을 정확히 판정하는 함수를 따로 만들어줘.

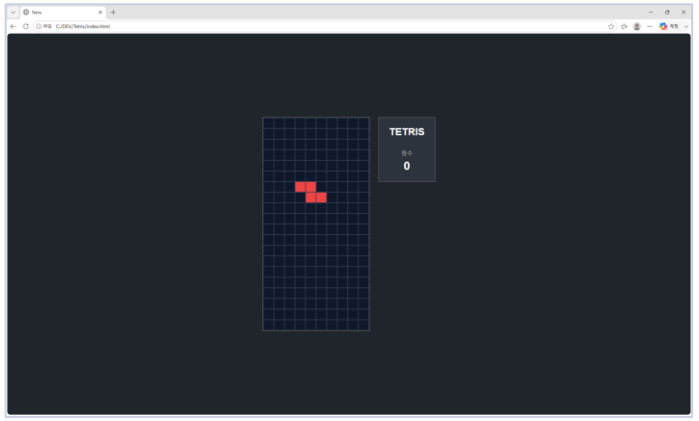



① 작성 칸에 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



② 결과 — 500ms 낙하 · hasCollision() · 새 블록

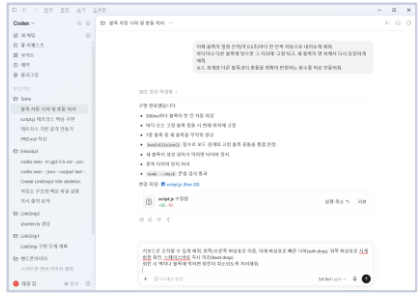



③ 브라우저 — 블록이 내려와 쌓인다

# 프롬프트 3 — 키보드 조작 · 품질 게이트 2

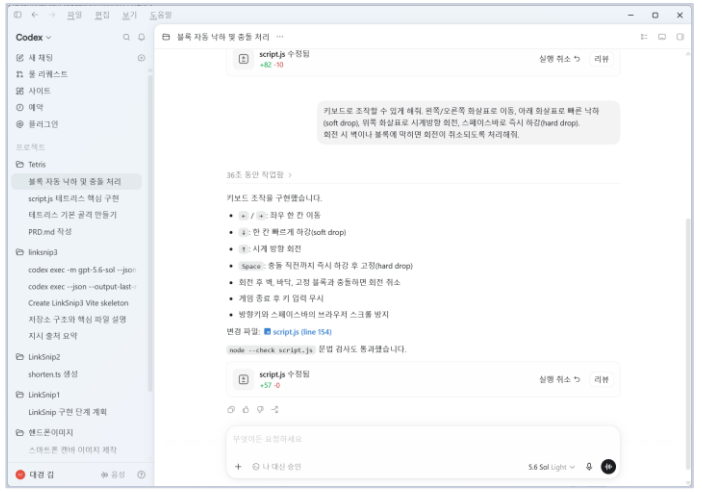
**▶ 프롬프트 3 — 키보드 조작**

키보드로 조작할 수 있게 해줘. 왼쪽/오른쪽 화살표로 이동, 아래 화살표로 빠른 낙하(soft drop), 위쪽 화살표로 시계 방향 회전, 스페이스바로 즉시 하강(hard drop).
회전 시 벽이나 블록에 막히면 회전이 취소되도록 처리해줘.



① 작성 칸에 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



② 결과 — ←/→ · ↓ · ↑ 회전 · Space 하드 드롭 · 회전 취소




③ 브라우저 — I 블록 회전 · 이동

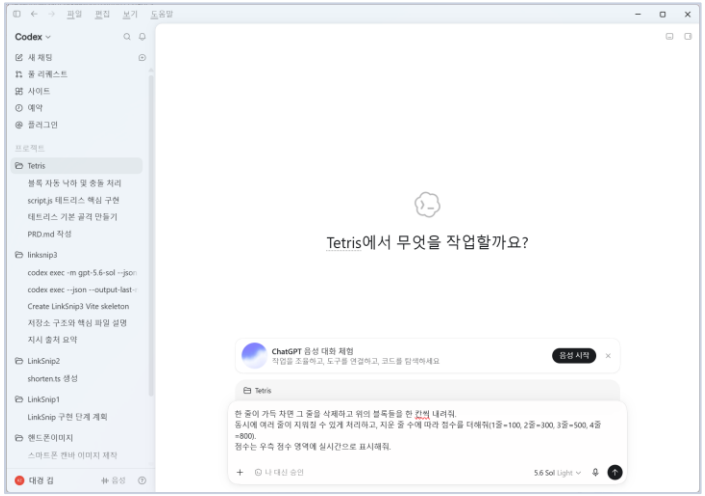
**▶ 품질 게이트**

`$review-game-logic` 로 충돌 판정 · 경계값 · 회전 오류를 점검해줘.

# 프롬프트 4 — 라인 클리어와 점수

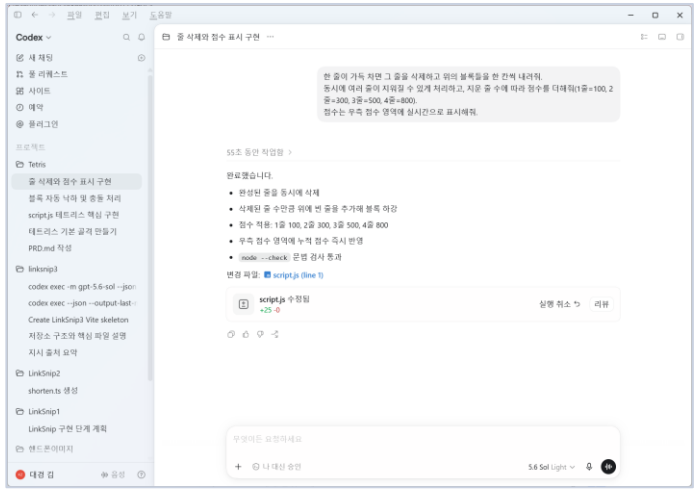
**▶ 프롬프트 4 — 라인 클리어 · 점수**

한 줄이 가득 차면 그 줄을 삭제하고 위의 블록들을 한 칸씩 내려줘.
동시에 여러 줄이 지워질 수 있게 처리하고, 지운 줄 수에 따라 점수를 더해줘(1줄=100, 2줄=300, 3줄=500, 4줄=800).
점수는 우측 점수 영역에 실시간으로 표시해줘.




① 작성 칸에 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② 결과 — 줄 삭제 · 1/2/3/4줄 100/300/500/800



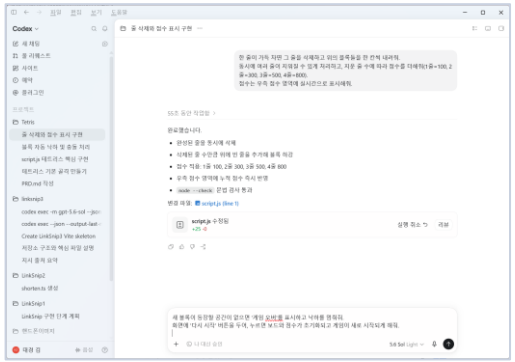

③ 브라우저 — 한 줄 삭제 후 점수 100

# 프롬프트 5 — 게임 오버와 다시 시작 · 품질 게이트 3

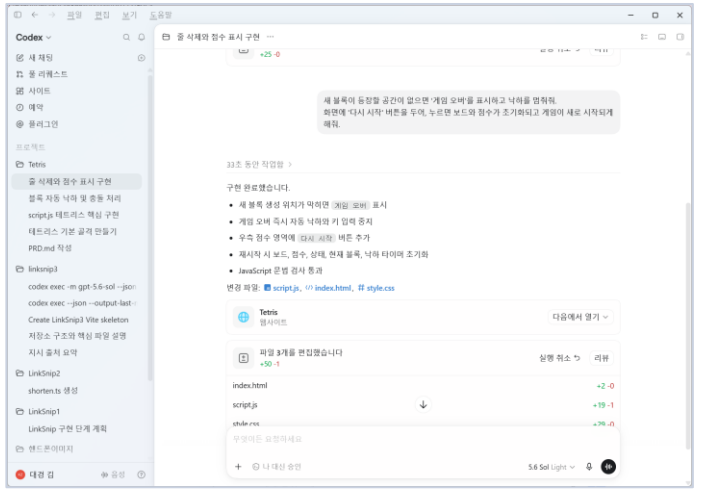
**▶ 프롬프트 5 — 게임 오버 · 재시작**

새 블록이 등장할 공간이 없으면 '게임 오버'를 표시하고 낙하를 멈춰줘.
화면에 '다시 시작' 버튼을 두어, 누르면 보드와 점수가 초기화되고 게임이 새로 시작되게 해줘.

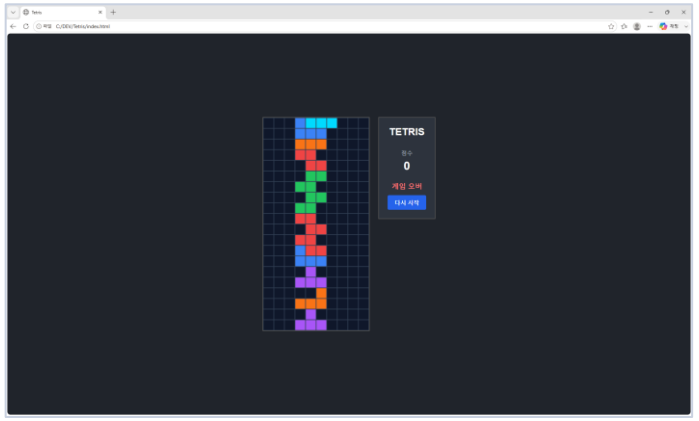
**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



① 작성 칸에 입력



② 결과 — 게임 오버 표시 · 다시 시작 버튼 · 초기화




③ 브라우저 — '게임 오버' · '다시 시작'

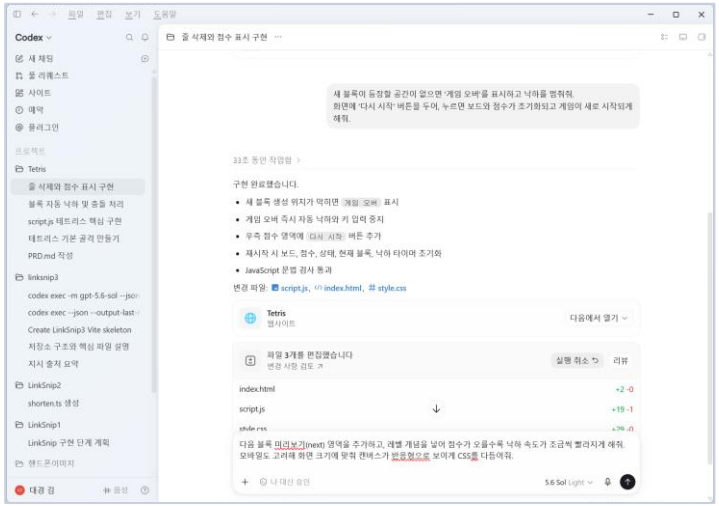
**▶ 품질 게이트**

`$qa-playtest` 로 시나리오별 기대 결과를 점검해줘 (4줄 동시 삭제 = 800점, 게임오버 후 재시작).

# 프롬프트 6 — 다듬기(선택): 다음 블록 · 레벨 · 반응형

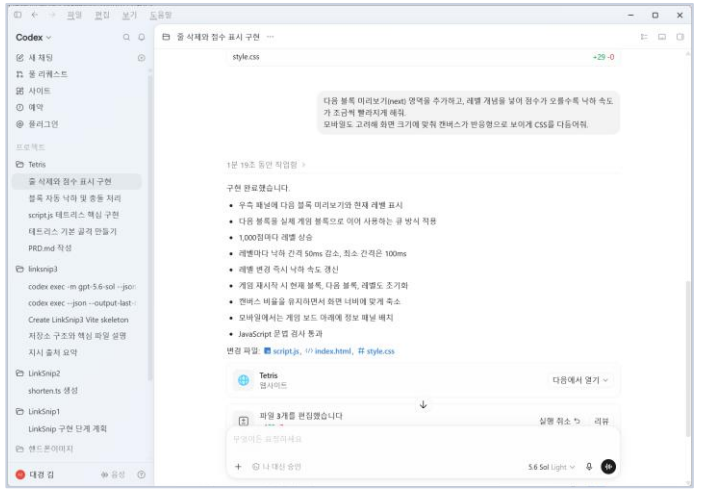
**▶ 프롬프트 6 — 다듬기 (선택)**

다음 블록 미리보기(next) 영역을 추가하고, 레벨 개념을 넣어 점수가 오를수록 낙하 속도가 조금씩 빨라지게 해줘.
모바일도 고려해 화면 크기에 맞춰 캔버스가 반응형으로 보이게 CSS를 다듬어줘.

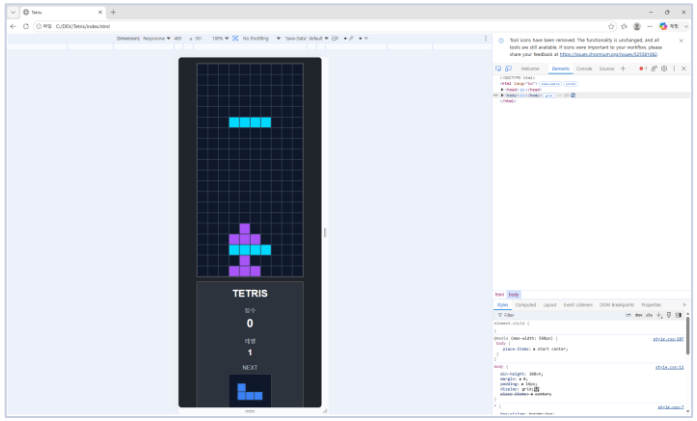


① 작성 칸에 입력

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다



② 결과 — NEXT · 레벨(1,000점마다) · 반응형




③ 브라우저(모바일 폭) — 점수 · 레벨 · NEXT

# 프롬프트 7 — 테스트 시나리오 점검 · 품질 게이트 4

가장 간단한 실행은 index.html 더블클릭 — 일부 브라우저 보안 정책 때문에 '로컬 서버'로 여는 편이 더 안정적이다

**▶ 프롬프트 7 — 테스트 시나리오 점검**

다음을 직접 점검하고, 안 되는 항목이 있으면 원인을 찾아 고쳐줘:
(1) 블록이 7종 모두 나오는가
(2) 좌우 이동 · 회전 · 하강이 모두 되는가
(3) 줄이 가득 차면 삭제되고 점수가 오르는가
(4) 4줄 동시 삭제 점수가 800인가
(5) 게임 오버 후 다시 시작이 정상 동작하는가.

**▶ 품질 게이트**

`$bug-hunt` 로 고위험 버그부터 찾아 제거해줘.

**▶ 품질 게이트**

`$refactor-safe` 로 기능 변화 없이 구조만 정리하고, 전후 점검 결과를 보여줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

Codex 응답

```
✦ 시나리오 점검 결과
  (1) 7종 생성 (7-bag 무작위)          통과
  (2) 이동 · 회전 · 하강                 통과
  (3) 줄 삭제 · 점수 증가                통과
  (4) 4줄 동시 삭제 = 800                실패 → 수정
      원인: 삭제 후 점수 계산 전에 줄 수 초기화
  (5) 게임 오버 후 다시 시작             통과

  수정 파일: script.js (+4 -3)
  > node --check script.js   문법 검사 통과
  재점검: 5/5 통과
```

# Git 으로 기록하고 GitHub 에 올리기 — 방법 A: GitHub CLI 로 한 번에

세 단어만 기억한다: 저장소(repo) = 버전 관리 단위 · 커밋(commit) = 변경 스냅숏 · 푸시(push) = GitHub 로 전송

## 사전 준비

- github.com 계정 만들고 로그인
- git --version 확인 (없으면 git-scm.com)
- GitHub CLI(권장): cli.github.com 에서 gh 설치 → gh auth login 으로 1회 인증

터미널 (Codex 가 실행하거나 직접 입력)

```
$ git init
$ git add -A
$ git commit -m "feat: 테트리스 게임 초기 버전"

# GitHub에 새 저장소를 만들고 그대로 업로드 (공개 저장소)
$ gh repo create tetris-codex --public --source=. --remote=origin --push
```

**▶ 프롬프트 8 — Git 초기화**

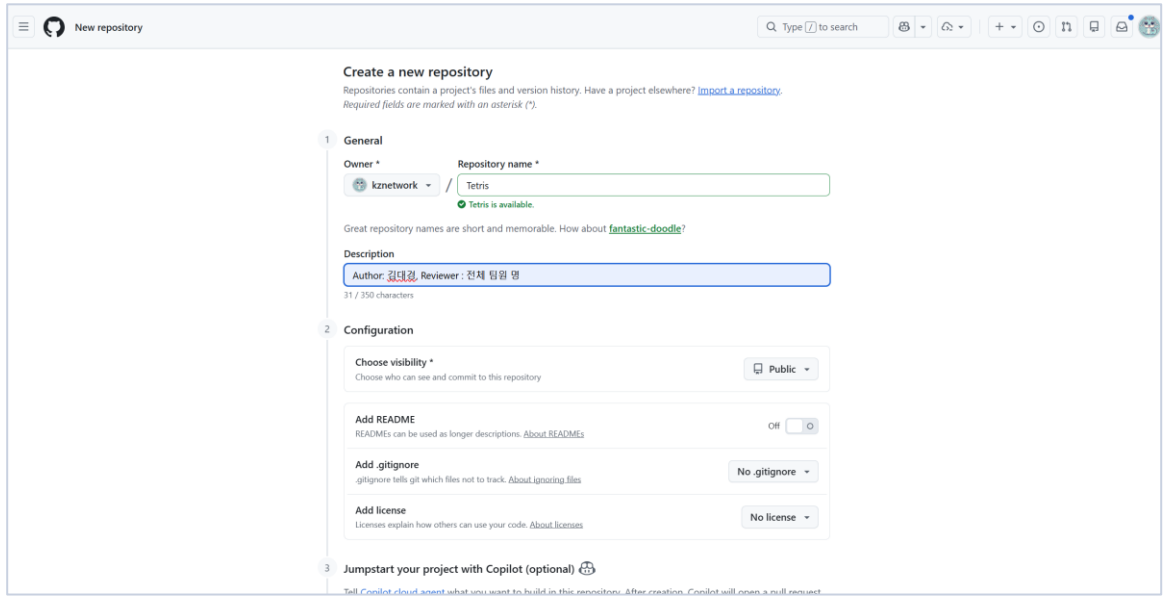
이 프로젝트를 Git 저장소로 초기화하고, OS · 에디터 임시 파일을 무시하는 .gitignore를 만들어줘. 그리고 첫 커밋을 'feat: 테트리스 게임 초기 버전' 메시지로 만들어줘. 터미널 명령을 실행하기 전에 무엇을 할지 먼저 한국어로 설명해줘.

**결과 예시** 응답 문구는 실행마다 다를 수 있다

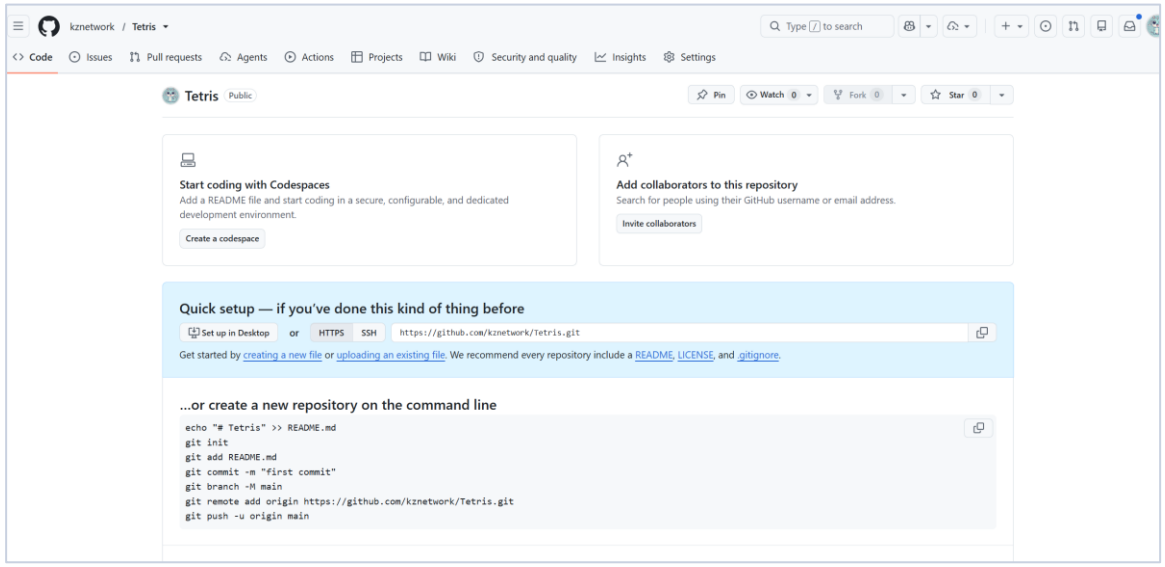
Codex 응답

```
✦ 실행 전 설명: git init 으로 저장소를 만들고, .gitignore 에 .DS_Store ·
  Thumbs.db · .vscode/ 를 넣은 뒤 첫 커밋을 만든다.
[main (root-commit) bf49373] feat: 테트리스 게임 초기 버전
 ✓ Created repository <사용자명>/tetris-codex on GitHub
 ✓ Pushed commits to https://github.com/<사용자명>/tetris-codex.git
```

# 방법 B — 웹에서 빈 저장소를 먼저 만들 때 (권장)




① New repository — Repository name · Public · README 끔



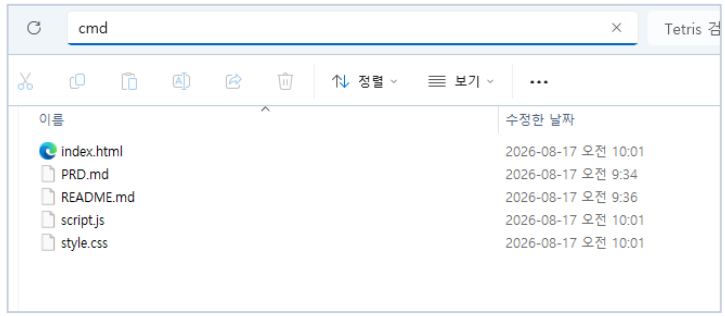

② 생성 직후 — Quick setup 의 원격 주소와 명령

터미널 예시 – 생성 화면의 원격 주소를 넣어 실행

```
$ git init
$ git add -A
$ git commit -m "feat: 테트리스 게임 초기 버전"
$ git branch -M main
$ git remote add origin https://github.com/<사용자명>/tetris-codex.git
$ git push -u origin main
```

지금 할 일 github.com → New repository → 이름 입력 → Create (README 추가 안 함) ▸ 표시되는 원격 주소 복사

# 방법 B 실행 결과 — 푸시하고 저장소에서 확인한다



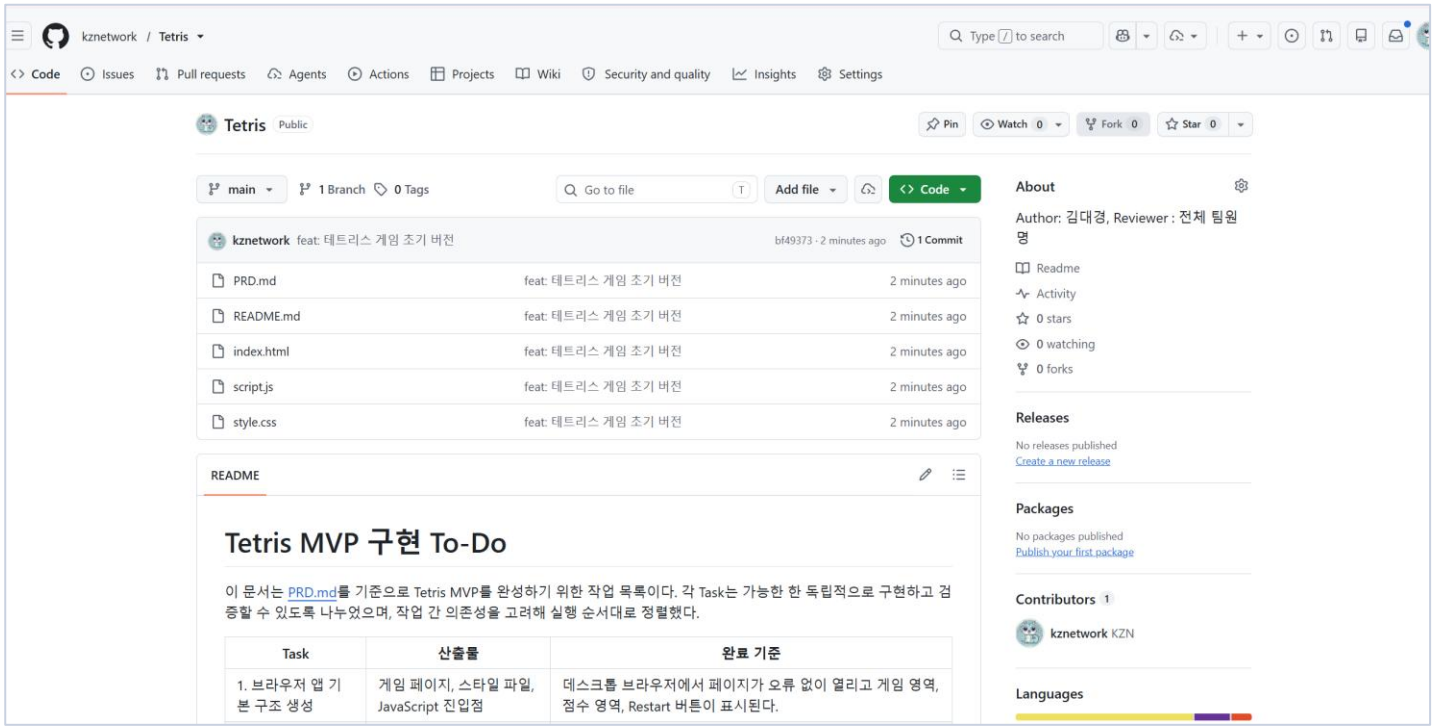

① Tetris 폴더 주소창에 cmd

**실행 결과** 워크북에서 실제로 실행한 출력

```
명령 프롬프트

C:\DEV\Tetris>git commit -m "feat: 테트리스 게임 초기 버전"
[main (root-commit) bf49373] feat: 테트리스 …
 5 files changed, 570 insertions(+)
C:\DEV\Tetris>git push --set-upstream origin main
Writing objects: 100% (7/7), 8.41 KiB, done.
To https://github.com/kznetwork/Tetris.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② GitHub 저장소 — PRD.md · README.md · index.html · script.js · style.css

**참고** 인증 팁 — 비밀번호 대신 '토큰'을 요구하면 GitHub Settings → Developer settings → Personal access tokens 에서 발급해 비밀번호 자리에 입력한다. gh 를 쓰면 자동 처리된다.

**지금 할 일** Tetris 폴더 주소창에 cmd → 명령 실행 ▸ 'branch main set up to track origin/main' 확인 ▸ GitHub 저장소 새로고침

# GitHub Pages 로 배포 — 누구나 접속하는 링크 만들기

정적 파일(HTML / CSS / JS)이므로 GitHub Pages 로 무료 배포할 수 있다

**1** **Settings → Pages**

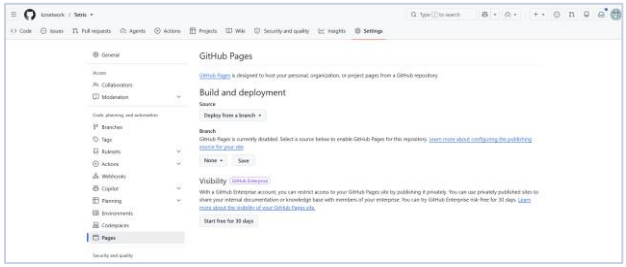
GitHub 저장소 페이지 상단 Settings → 왼쪽 Pages

**2** **Source · Branch 지정**

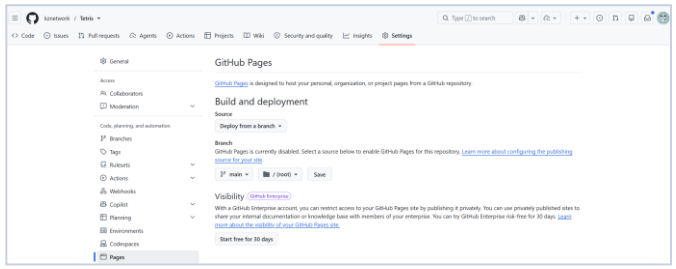
Source = 'Deploy from a branch', Branch = main / (root) → Save

**3** **주소로 접속**

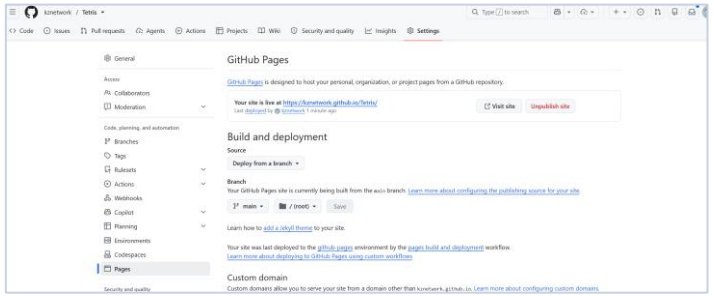
https://<사용자명>.github.io/<저장소>/ — 1~2분 걸릴 수 있다




① Settings → Pages — Branch None (아직 꺼져 있음)




② Branch: main / (root) 선택 → Save

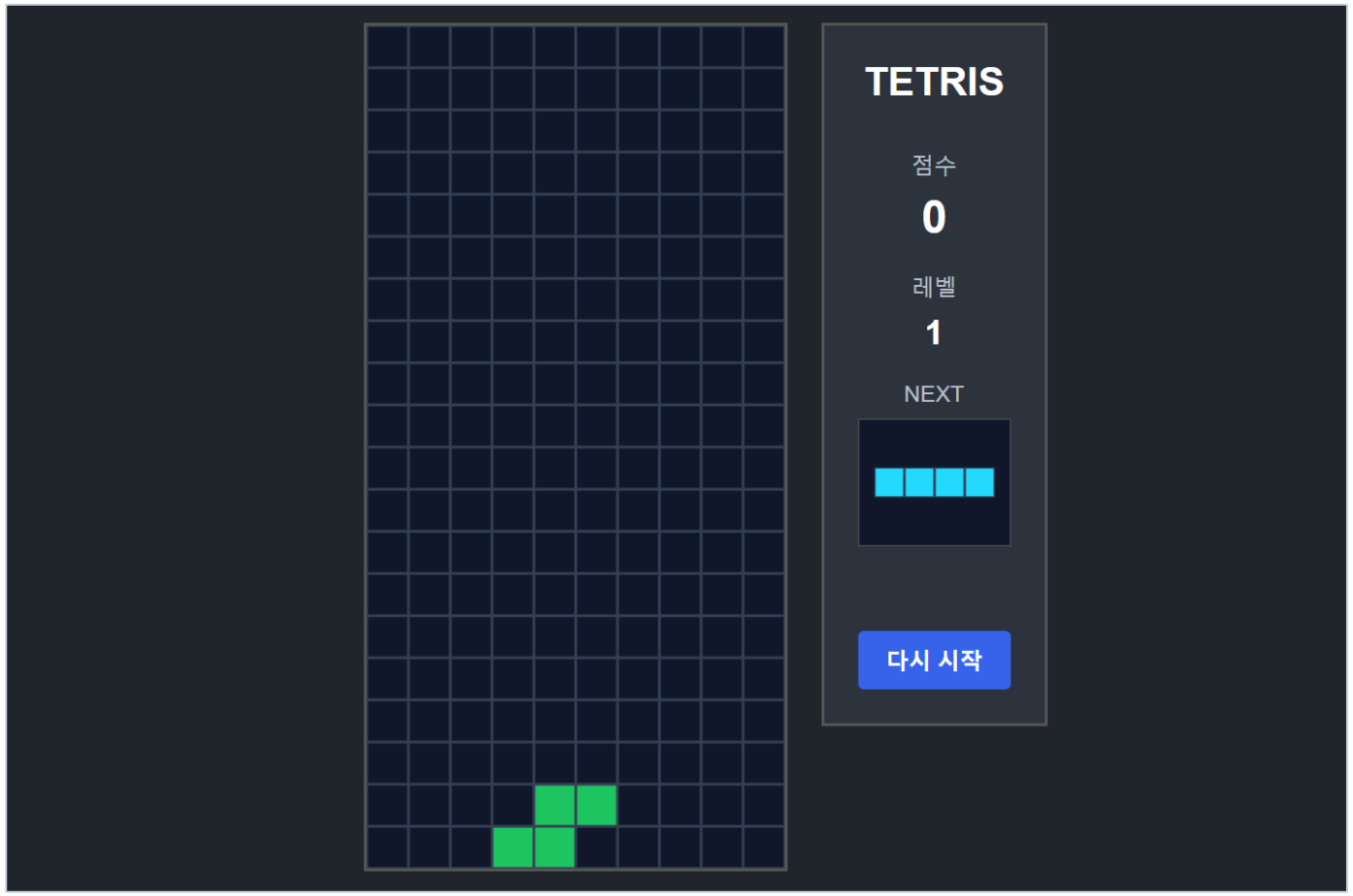



③ 'Your site is live at https://kznetwork.github.io/Tetris/'

**TIP** 저장소 이름이 tetris-codex 면 주소는 …github.io/tetris-codex/ 다. 404 가 나오면 잠시 뒤 새로고침한다.

# 배포 결과 — 인터넷 어디서나 내 게임이 열린다

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




https://kznetwork.github.io/Tetris/ — 점수 · 레벨 · NEXT · 다시 시작

**배포 주소 (예)**
**https://<사용자명>.github.io/<저장소>/**

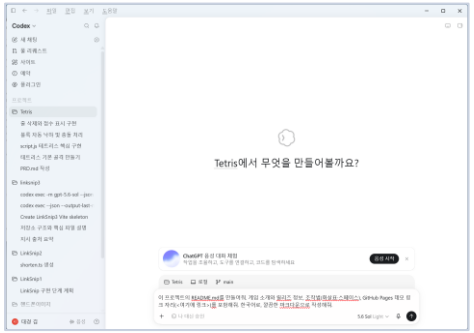
### 완료 조건

- ✅ index.html · style.css · script.js 골격을 만들었다
- ✅ 블록 낙하 · 조작 · 줄 삭제 · 점수 · 게임 오버가 모두 동작한다
- ✅ GitHub 에 올리고 GitHub Pages 주소로 접속된다

# 프롬프트 9 — README 업그레이드 · 품질 게이트 5

**▶ 프롬프트 9 — README 업그레이드**

이 프로젝트의 README.md를 만들어줘. 게임 소개와 릴리즈 정보, 조작법(화살표 · 스페이스), GitHub Pages 데모 링크 자리(<여기에 링크>)를 포함해줘. 한국어로, 깔끔한 마크다운으로 작성해줘.

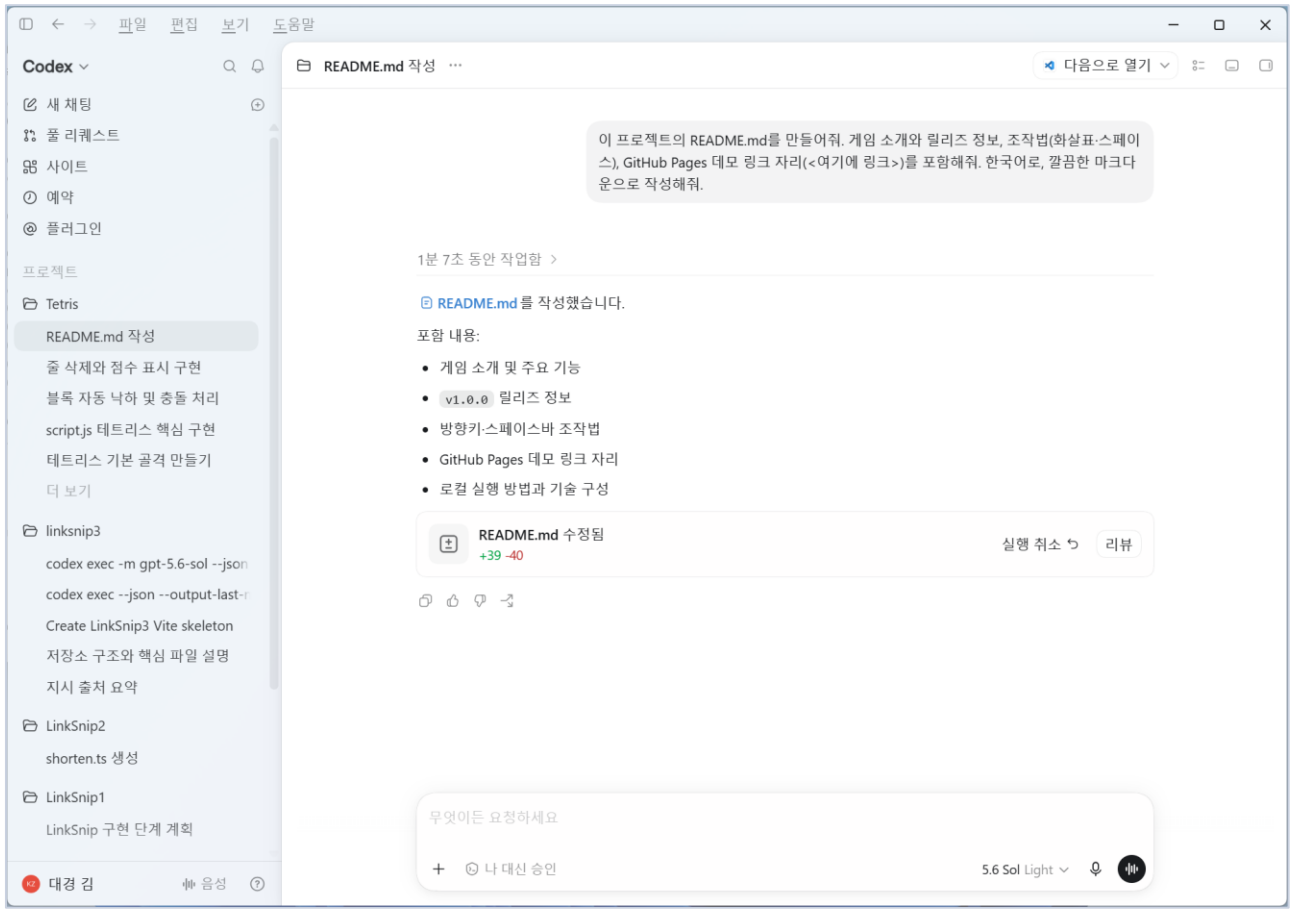



① 작성 칸에 입력

**▶ 품질 게이트**

`$release-check` 로 파일 연결 · 실행법 · GitHub Pages 설정을 최종 점검해줘.

**실행 결과 (화면)** 워크북 실습 화면 — 앱 버전에 따라 조금 다를 수 있다




② 결과 — README.md 작성 (게임 소개 · 릴리즈 정보 · 조작법 · 데모 링크 · 로컬 실행)

# 프롬프트 10 — 변경 후 재배포는 add → commit → push

GitHub Pages 는 main 브랜치가 바뀌면 자동으로 다시 배포된다

터미널

```
$ git add -A
$ git commit -m "feat: README.md 파일에 릴리즈 정보 추가"
$ git push
```

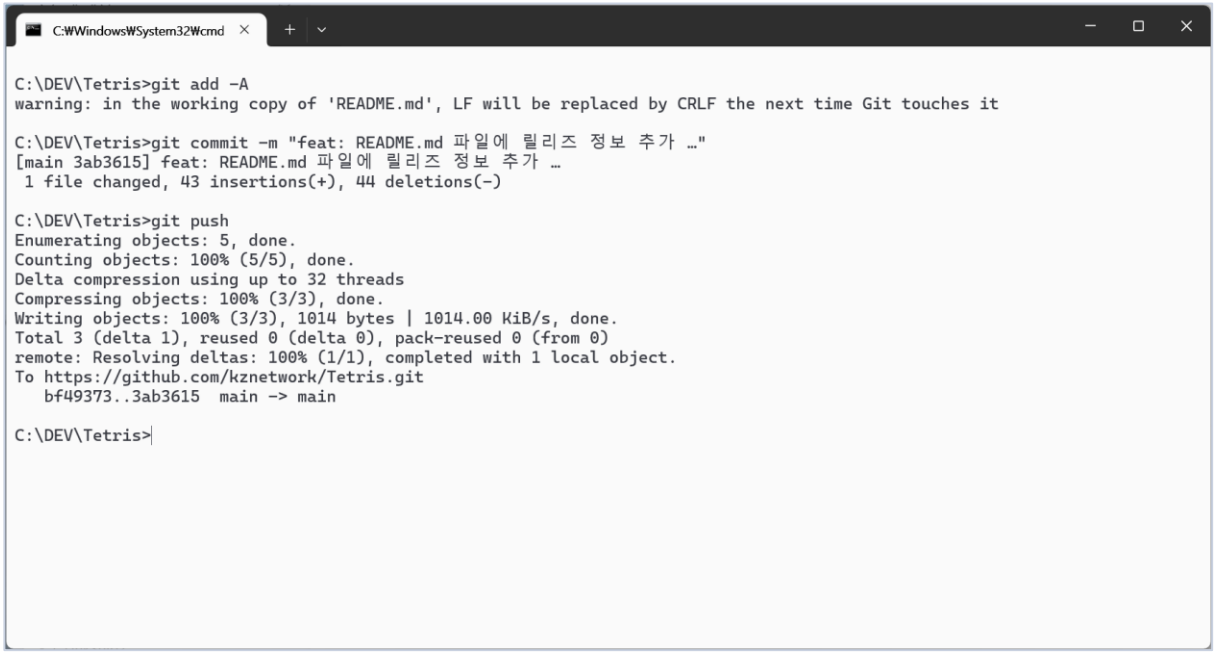
**프롬프트 10 — Codex 에 맡길 때**

변경 사항을 커밋하고 GitHub에 push해줘. 커밋 메시지는 'feat: README.md 파일에 릴리즈 정보 추가'로 하고, 실행 전에 무엇을 할지 먼저 설명해줘.

## VS Code 로 해도 된다

- 왼쪽 '소스 제어' 패널 → 변경 파일 확인 → 메시지 입력 → 커밋 → 동기화(push)

**실행 결과 (화면)** 워크북에서 실제로 실행한 출력




결과 — 1 file changed · main -> main 푸시 완료

지금 할 일 터미널에서 아래 세 명령 ▸ 1~2분 뒤 배포 주소 새로고침 ▸ README 가 저장소 첫 화면에 보이는지 확인

11

CHAPTER 11 · 강의

# 마무리 회고 — 개념은 함께, 실습은 각자

전체 흐름을 한눈에 보고, 핵심 개념을 '한 문장으로' 말해 본 뒤, 자주 막히는 문제를 정리한다.

11.1 한 문장 회고 | 11.2 핵심 한 장 | 11.3 트러블슈팅

# 한 문장으로 말해 보기 — 개념 회고와 최종 점검

### 프롬프트

작게 쪼개고 검증 방법을 함께 준다

### 컨텍스트

필요한 것만, 정확히 (많이 ≠ 똑똑)

### AGENTS.md

작업 전 먼저 읽는 상시 규칙서

### 하네스

결과가 진짜 동작하는지 검증하는 안전망

### 최종 점검 체크리스트

- ✅ Codex App 으로 폴더를 열고 작업을 시킬 수 있다
- ✅ P · C · T · F 로 검증이 포함된 프롬프트를 쓸 수 있다
- ✅ AGENTS.md · config.toml · 승인 모드를 설정할 수 있다
- ✅ typecheck · lint · test 하네스를 갖출 수 있다
- ✅ PRD → To-Do → 구현 → GitHub Pages 배포를 할 수 있다

**부록 — 언제든 꺼내 쓰는 참고**

| A 치트시트 | B config.toml | C AGENTS.md | D 용어집 | E 참고 자료 | F 마크다운 작성법 |
|---|---|---|---|---|---|
| 프롬프트 · 컨텍스트 · 하네스 | 전체 예시 (주석 포함) | 템플릿 (복사용) | 16개 용어 | 출처 모음 | README 작성 |

# 핵심 한 장 요약 — 이것만 기억하면 된다

1

## Codex 는 에이전트다

챗봇이 아니라 파일을 직접 고치고 테스트까지 돌린다

2

## 프롬프트

작게 쪼개고, 검증 방법을 함께 준다

3

## 컨텍스트

많이가 아니라 '정확히, 필요한 것만'

4

## 규칙 & 안전

AGENTS.md 에 규칙, Sandbox / Approvals 로 안전

5

## 하네스

빌드 · 테스트 · 배포 · 로그로 결과를 검증

6

## 실습은 직접

Tetris — 브라우저 게임을 만들어 배포

# 자주 막히는 문제 — 증상별 해결

| 증상 | 원인 · 해결 | 참고 |
|---|---|---|
| npm install 이 실패한다 | 네트워크 기본 차단 — 승인 창 '이번에만 허용' 또는 -c 'sandbox_workspace_write.network_access=true' | 7.6 |
| 사이트 · 예약 메뉴가 없다 | 앱 버전 · 요금제 차이 — 새 채팅에 @Sites 처럼 직접 호출, 없으면 읽기만 | 4.1 · 4.2 |
| 규칙을 매번 잊는다 | 규칙을 대화가 아니라 AGENTS.md 에 — '지시 출처 요약'으로 로드 확인 | 7.1 · 7.4 |
| 결과가 매번 다르다 | 대상 파일 · 규칙 · 검증이 빠졌다 — P · C · T · F 로 다시 쓰기 | 5.3 |
| 대화가 길어져 엉뚱해진다 | 새 스레드로 나누고, 필요한 파일만 @ 로 | 6.4 · 6.6 |
| $skill 이 안 보인다 | 폴더 위치(.agents/skills) · name 확인 → 새 채팅에서 다시 호출 | 7.7 |
| git push 에서 비밀번호 거부 | 토큰(PAT) 사용 또는 gh auth login | 10.10 |
| Pages 주소가 404 | Branch main / (root) 저장 확인 → 1~2분 뒤 새로고침 | 10.11 |

# 부록

APPENDIX · A – F

## 부록

A 치트시트 · B config.toml 전체 예시 · C AGENTS.md 템플릿 · D 용어집 · E 참고 자료 · F 마크다운 작성법

# A.1 프롬프트 엔지니어링 예시 (P · C · T · F)

네 요소(역할 · 맥락 · 작업 / 검증 · 형식)를 채워서 사용한다

### ▶ 기능 추가

P: 꼼꼼한 프론트엔드 개발자로서 작업해줘.
C: 대상 src/components/Result.tsx / 새 라이브러리 금지, 기존 디자인 토큰 사용.
T: 단축 링크 결과에 "복사" 버튼을 추가. 검증 — npm run build · npm test 통과 + 클릭 시 복사.
F: 변경 파일 목록 + 핵심 변경점 3줄.

### ▶ 버그 수정

C: 재현 — 빈 URL 입력 후 줄이기 → 앱이 흰 화면이 됨.
T: 원인을 찾아 고치고, 같은 회귀를 막을 테스트 추가.
F: 추가한 테스트가 실패 → 통과로 바뀌는 것을 보여줄 것.

### ▶ 리팩터링

C: 대상 src/lib/shorten.ts.
T: 동작 / 공개 API는 그대로, 중복만 제거해 가독성 개선.
F: 기존 테스트가 전부 통과함을 보여줄 것(동작 변화 금지).

### ▶ 계획 먼저

T: 코드를 짜기 전에 5–8개 작은 작업으로 나눈 To-Do만 제안. 각 작업에 " 완료 기준" 한 줄씩.

| 원칙 | Do | Don't |
|---|---|---|
| 역할(P) | 관점 · 전문성 부여 | 아무 맥락 없이 지시 |
| 맥락(C) | 파일 · 제약 명시 | 전부 알아서 하라고 |
| 작업(T) | 테스트 · 실행 명령 포함 | 검증 없이 '완성해줘' |
| 형식(F) | 요약 · diff · 로그 요청 | 결과 형태 미지정 |

# A.2 컨텍스트 · A.3 하네스 엔지니어링 예시

### ▶ A.2 컨텍스트 치트시트

# 파일 콕 집기 (선택)
**@src/lib/shorten.ts @tests/shorten.test.ts**
이 두 파일만 보고 makeCode의 충돌 가능성을 분석해줘.
# 이미지/스크린숏 주기 (디자인 · 에러)
이 시안(./design/result.png)대로 결과 화면 레이아웃을 맞춰줘.
# 규칙은 대화 말고 파일로 (지속성 / 작성)
반복되는 규칙은 AGENTS.md 로 옮긴다 → 매 작업 자동 로드
# 컨텍스트 다이어트 (관련성)
지금 작업과 무관한 파일은 참고하지 말고, shorten.ts 한 곳만 수정해줘.

**참고**

컨텍스트 우선순위(높음 → 낮음) — ① 현재 폴더 AGENTS.md ② 프로젝트 루트 AGENTS.md ③ 전역 AGENTS.md ④ @로 준 파일 ⑤ 대화 기록. 가까운 규칙이 먼 규칙을 덮어쓴다.

### A.3 하네스 치트시트

```
# (1) AGENTS.md — 완료의 정의
## Definition of Done
다음이 모두 통과해야 끝이다:
npm run typecheck && npm run lint && npm test
실패 시 로그를 보고 원인을 고쳐 다시 통과시킨다.

# (2) 프롬프트 — 검증을 항상 포함
기능을 구현하고, 위 3개 명령을 실행해
통과 로그를 마지막에 요약해줘.

# (3) CI — 비대화형 하네스
codex exec --json --output-last-message out.txt \
  "변경을 구현하고 typecheck · lint · test를 모두 통과시켜줘."

# (4) 권한 경계 — 안전한 자율성
codex --sandbox workspace-write --ask-for-approval on-request "..."
```

# B. config.toml 전체 예시 (주석 포함)

~/.codex/config.toml 로 저장하면 이 과정에 맞는 합리적 기본값이 된다 — 모델 이름은 본인이 쓸 수 있는 최신 모델로 바꾼다

```
# ~/.codex/config.toml
# ── 모델 ──────────────────
model = "gpt-5.6-sol"            # 사용 가능한 최신 모델로 지정
model_provider = "openai"        # 기본 제공자
# ── 안전(샌드박스 & 승인) ────────
sandbox_mode = "workspace-write"   # read-only | workspace-write | danger-full-access
approval_policy = "on-request"     # untrusted | on-request | never
approvals_reviewer = "user"        # user | auto_review (버전에 따라 없을 수 있음)
[sandbox_workspace_write]
network_access = false             # 기본 차단. 설치 등 필요 시 일시 허용
# writable_roots = ["/추가/쓰기허용/경로"]  # 작업 폴더 밖 특정 경로 허용 시
# ── 프로젝트 규칙 문서 ─────────
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536      # 합쳐 읽는 규칙 최대 크기(기본 32KiB)
# ── 선택 기능 플래그 ──────────
[features]
goals = true                       # /goal (목표 모드) 사용
# ── 외부 도구(MCP) ────────────
# [mcp_servers.github]
# url = "https://.../mcp"          # 또는 command = "..."(STDIO)
# env = { GITHUB_TOKEN = "..." }
# ── 커스텀 모델 제공자(선택) ─────
# [model_providers.proxy]
# name = "My proxy"
# base_url = "https://..."
```

## TIP — 자주 쓰는 CLI 플래그

- --sandbox / -s (범위)
- --ask-for-approval / -a (승인)
- --cd (작업 폴더) · --profile (프로필)
- -c key=value (일시 설정)
- --full-auto (자동) · exec (비대화형)

**참고** 프로젝트별로 다르게 하려면 <프로젝트>/.codex/config.toml 에 필요한 키만 적는다 (7.5).

# C. AGENTS.md 템플릿 (복사용)

새 프로젝트를 시작할 때 그대로 붙여 넣고 빈칸을 채운다

```
# AGENTS.md
## 프로젝트 개요
- 무엇을: (한 줄)
- 기술 스택: (프론트/백/DB)
## 셋업
- 설치: (예: npm install)
- 개발: (예: npm run dev)
- 빌드: (예: npm run build)
## Definition of Done (완료의 정의)
다음이 모두 통과하기 전에는 끝났다고 하지 말 것:
(예: npm run typecheck && npm run lint && npm test)
실패하면 로그를 보고 고쳐 다시 통과시키고, 통과 로그를 요약한다.
## 코드 규칙
- 들여쓰기/스타일: (예: 스페이스 2칸)
- 금지: 비밀키 하드코딩, 무분별한 의존성 추가
- 새 운영 의존성 추가 전에는 먼저 물어볼 것
## 커밋/PR 규칙
- 커밋 메시지: 한 줄 요약 + 변경 이유
- 한 작업 = 한 커밋(가능하면)
```

**▶ 프롬프트 — 템플릿 채우기**

왼쪽 템플릿으로 이 프로젝트의 AGENTS.md를 만들어줘. 빈칸은 package.json과 폴더 구조를 보고 채우고, 확인되지 않는 항목은 '확인 필요'로 남겨줘.

### 쓰는 법

- 루트에 AGENTS.md 로 저장
- Definition of Done 은 실제로 통과하는 명령으로
- 7.4 처럼 '지시 출처 요약'으로 로드 확인
- CLI 는 /init 으로 뼈대를 만들 수도 있다

# D. 용어집

| 용어 | 뜻 |
|---|---|
| Codex | 터미널 · 에디터에서 코드를 직접 읽고 고치고 명령을 실행하는 코딩 에이전트 |
| 사용 방식(Surface) | Codex 를 쓰는 창구 — App · IDE 확장 · CLI |
| 스레드(Thread) | 하나의 작업 세션. 여러 프롬프트가 이어진 대화 |
| 컨텍스트 윈도우 | 한 스레드의 정보가 들어가는 모델의 작업 기억 한도 |
| 컴팩션(Compaction) | 한도에 가까워지면 중요한 내용만 요약해 작업을 이어 가는 기능 |
| AGENTS.md | 작업 전 자동 로드되는 프로젝트 상시 규칙 문서 |
| config.toml | 모델 · 승인 · 샌드박스 등 Codex 기본 동작 설정 파일 |
| Sandbox(샌드박스) | Codex 의 파일 / 네트워크 접근 '범위' |

| 용어 | 뜻 |
|---|---|
| Approvals(승인) | 위험 행동 전 사람에게 묻는 '시점' 정책 |
| Harness(하네스) | 빌드 · 테스트 · 배포 · 로그로 결과를 검증하는 틀 |
| Skill | 언제 · 어떤 절차로 할지를 묶은 재사용 작업 지침 (SKILL.md) |
| codex exec | 사람 개입 없이 끝까지 도는 비대화형 실행 (CI 용) |
| MCP | 외부 도구(GitHub 등)를 Codex 에 연결하는 표준 규약 |
| Worktree | 같은 저장소를 여러 폴더로 동시에 체크아웃하는 Git 기능 |
| PRD | 제품 요구사항 정의서 — 무엇을 왜 만드는지 적은 문서 |
| GitHub Pages | 저장소의 정적 파일을 무료로 호스팅하는 GitHub 기능 |

공식 문서: Codex 문서(developers.openai.com/codex) · AGENTS.md(agents.md) — 제품 · 기능은 업데이트될 수 있으니 막힐 때는 공식 문서의 최신 안내를 함께 확인한다.

# E. 참고 자료 — 장별 출처

## 2–4장 Codex · App

- OpenAI Codex 문서 — developers.openai.com/codex
- Codex CLI 슬래시 명령 · Skills · AGENTS.md 가이드 (같은 문서 사이트)
- AGENTS.md 개방형 형식 — agents.md · Agent Skills 표준 — agentskills.io

## 5장 프롬프트

- Bsharat 외(2023) Principled Instructions… arXiv:2312.16171
- A. Ng & I. Fulford, 'ChatGPT Prompt Engineering for Developers' — DeepLearning.AI × OpenAI
- OpenAI「Prompt engineering」· Anthropic「Prompt engineering overview」· promptingguide.ai
- Stanford CS230 (Autumn 2025) Lecture 8 'Agents, Prompts, and RAG'

## 6장 컨텍스트

- Anthropic「Effective context engineering for AI agents」(2025.9)
- A. Karpathy, X(2025.6.25) · T. Lütke, X(2025.6)
- DeepLearning.AI「Context Hub」(2026.3)

## 7–8장 하네스

- V. Trivedy「The Anatomy of an Agent Harness」LangChain(2026.3)
- M. Hashimoto「My AI Adoption Journey」(2026.2) · R. Lopopolo, openai.com/index/harness-engineering(2026.2)
- B. Böckeler, martinfowler.com(2026.4) · Anthropic「Effective harnesses for long-running agents」(2025.11)
- codecentric (2026) · ADTmag (2026.7) — Loop Engineering

## 9–10장 배포

- GitHub Docs — 저장소 생성 · Pages 배포 (docs.github.com)
- GitHub CLI — cli.github.com
- Markdown Guide — markdownguide.org

# F. 마크다운(Markdown) 작성법 ① — 개행 · 헤더 · 코드 · 특수 기호

README.md · 위키를 쓸 때 쓰는 간결한 문법 — 에디터에 따라 결과가 조금 다르다 (markdownguide.org)

## □ 개행 — 엔터 2개, 또는 문장 뒤 공백 2개 + 엔터

```
**첫번째** 문장(엔터 1개)
두번째 문장

**첫번째** 문장(엔터 2개)

두번째 문장
```

**첫번째** 문장(엔터 1개) 두번째 문장
**첫번째** 문장(엔터 2개)
두번째 문장

## □ 헤더 — 제목 · 강조 (위키 하위 제목)

```
# h1
## h2
### h3
#### h4
##### h5
```

# h1
## h2
### h3
#### h4
##### h5

## □ 코드 — ``` 로 감싸고, 언어 이름으로 문법 강조

````
```javascript
for (let i = 0; i < 10; ++i) {
   ...
}
```
`for` 문은 문장 안에서 ` 로 감싼다
````

```javascript
for (let i = 0; i < 10; ++i) {
   ...
}
```

`for` 문은 문장 안에서 ` 로 감싼다

## □ 특수 기호 — 그대로 보이려면 앞에 \

```
\*별표 그대로\*
\# 샵 그대로
\[대괄호\]
```

*별표 그대로*
# 샵 그대로
[대괄호]

# F. 마크다운 작성법 ② — 강조 · 인용 · 목록 · 체크박스

## ☐ 강조와 구분선

```
*이텔릭체*  _이텔릭체_
**볼드체**  __볼드체__
~~취소선~~
<u>밑줄</u>
<span style="color:red">빨간색</span>
***  (구분선)
```

*이텔릭체*
**볼드체**
~~취소선~~(가로줄)
<u>밑줄</u>
빨간색

---

## ☐ 인용 — > 로 이메일 인용 형식

```
> 인용이다.
>> 들여쓰기된 인용이다.
>>
>> **마크다운** 의 용법이 그대로 적용된다.
```

> 인용이다.
>> 들여쓰기된 인용이다.
>>
>> **마크다운** 의 용법이 그대로 적용된다.

## ☐ 목록 — 숫자. / * + - · 하위 항목은 Tab 으로 들여쓰기

```
1. 목록이다.
3. 번호를 잘못 써도 순서대로 2번
* 부모 (* 뒤에 띄어쓰기)
        + 자식 (Tab 1)
                - 손자 (Tab 2)
```

1. 목록이다.
3. 번호를 잘못 써도 순서대로 2번
* 부모 (* 뒤에 띄어쓰기)
    + 자식 (Tab 1)
        - 손자 (Tab 2)

## ☐ 체크박스 — - [ ] 와 x

```
- [ ] 항목 1
- [x] 항목 2
```

- [ ] 항목 1
- [x] 항목 2

# F. 마크다운 작성법 ③ — 접기 · 링크 · 그림 · 표

## □ 접기 / 펼치기

```
<details>
<summary>접기/펼치기
</summary>
<div markdown="1">
펼쳐졌을 때 표시되는 내용
</div>
</details>
```

▶ 접기/펼치기
(펼치면) 펼쳐졌을 때 표시되는 내용

## □ 링크 — [LinkText](URL)

```
[Github](https://www.github.com)
```

Github

## □ 그림 — ![ALTText](URL)

```
![오드리햅번](https://github.com/assets/133472501/)
```



결과 — 이미지가 표시된다 (오류 시 ALTText)

## □ 표 — | 와 - · : 로 정렬

```
|왼쪽정렬|가운데정렬|오른쪽정렬|
|:--|:--:|--:|
|자동차|지상 이동수단||
|배|수상 이동수단||
|비행기|*항공* 이동수단|비고|
```

| 항목 | 설명 | 비고 |
|---|---|---|
| 자동차 | 지상 | |
| 배 | 수상 | |
| 비행기 | 항공 | ✓ |

**TIP** Tetris 의 README.md(10.12)도 이 문법으로 작성된다 — 제목(#) · 목록(-) · 코드(```) · 링크([]()) · 표(|)만 알아도 충분하다.

CLOSING

# 수고하셨는다

개념은 함께, 실습은 각자 — 이제 내 손으로 만든 결과물이 인터넷에 있다.

### 다음 단계

- 부록 A 치트시트로 P · C · T · F 를 내 프로젝트에 적용해 보기
- AGENTS.md 에 나만의 '완료의 정의' 못 박기
- 게임에 다음 블록 미리보기 · 레벨 · 속도 증가 기능 더해 보기 (10.8)

## Q & A · 질문을 환영한다